UNIVERSITÀ DELLA CALABRIA

Erasmus Mundus Joint Master's in Advanced Solid Mechanics

(STRAINS)





DIPARTIMENTO DI INGEGNERIA CIVILE

Master's Thesis

Physics-Informed and Data-Driven Neural Networks for Structural Analysis

Supervisors:

Prof. Domenico Magisano

Prof. Giovanni Garcea

Candidate:

Qurat Ul Ain Karim

Matr. 270441

ACADEMIC YEAR 2025/2026

میری امّی کے لیے۔
میں جو کچھ بھی ہوں، آپ کے دم سے ہوں۔

Abstract

Finite element analysis has been established as a robust numerical tool for structural analysis, but repeated simulations can be tedious and computationally expensive. Neural networks are an emerging tool for structural analysis that reduces the computational cost and time of repeated finite element simulations. The main objective of this thesis is to develop and evaluate data-driven and physics-informed neural-network surrogate models for predicting the displacement, stress, and modal responses of parametrised two-dimensional plane-stress structural domains, and to check their accuracy, computational efficiency, and applicability relative to finite element solutions.

A MATLAB-based finite element solver is developed to analyse random geometries and produce the training data required for data-driven neural networks. Simultaneously, physics-informed neural networks are also formulated using equilibrium equations, constitutive laws, and boundary conditions to calculate the loss function. Variational energy approaches in physics-informed networks, where the potential energy is penalised in the loss function, are also tested as an alternative. This work investigates the effects of mesh refinement, geometric parameter ranges, and proper orthogonal decomposition, and extends surrogate modelling to modal analysis.

Results revealed that data-driven neural networks can provide accurate and fast predictions but the training required finite element datasets, while physics-informed approaches do not completely rely on pre-generated data but are more sensitive to formulation and training parameters. The variational formulation requires less training time and improves accuracy compared to the strong form. This study shows the potential of neural-network surrogates for structural analysis while identifying their respective advantages and limitations. Future work can extended surrogate models for the prediction of transient dynamic responses.

Contents

1 Theoretical Background: Structural Analysis **4**

1.1 Linear Elasticity and Plane Stress Formulation 4

1.2 Derivation of the Weak Form 8

1.2.1 Relation to the Total Potential Energy 9

1.3 Finite Element Method 9

1.3.1 Domain Discretisation 10

1.3.2 Shape Functions 10

1.3.3 Element Stiffness Matrix Formulation 11

1.3.4 Bilinear Isoparametric Mapping 12

1.3.5 The Jacobian Matrix 13

1.3.6 Gauss Quadrature 14

1.3.7 Global Stiffness Matrix Assembly 15

1.3.8 Global Force Vector Assembly 16

1.3.9 Boundary Conditions 16

1.3.10 Matrix Partitioning for Boundary Conditions 17

1.3.11 Nodal Displacements Solution 18

1.4 Structural Dynamics and Modal Analysis 18

1.4.1 Global Mass Matrix Assembly 19

1.4.2 Equation of Motion and Modal Analysis 19

2 Theoretical Background: Artificial Neural Networks **22**

2.1 Feedforward Neural Networks 23

2.2 Data-Driven and Physics-Informed Networks 24
2.3 Main Features of Neural Networks 25
2.3.1 Activation Functions 25
2.3.2 Loss Functions and Error Metrics 26
2.3.3 Training and Optimization 27
2.3.4 Normalisation of Input and Output Data 28

3 Methodology **29**
3.1 Problem Definition 29
3.2 Geometry and Boundary Conditions 30
3.3 Material Properties 31
3.4 Loading Conditions 31
3.5 Mesh Generation 32
3.6 Dataset Generation Strategy 32
3.6.1 Parametrization of Geometry 32
3.6.2 Parametrisation of Loading 33
3.6.3 Storage of Output 33
3.7 Computational Time 34
3.8 MATLAB Implementation 34
3.9 Verification of the FEM Solver 35

4 Data-driven Neural Networks **36**
4.1 Problem definition 36
4.2 Preliminary Case: Single Body Force on a Rectangular Structure . . 36
4.3 Case 1: Two Independent Body Force Components on a Rectangular Structure 37
4.3.1 Dataset Generation 38
4.3.2 Finite Element Assembly Function 40
4.3.3 Neural Network Definition 43
4.3.4 Training Strategy 44
4.3.5 Inference and Superposition 46

4.3.6 Results . . . 47
4.4 Case 2: Effect of Mesh Size . . . 49
4.5 Case 3: Effect of Dimension Change . . . 51
4.6 Case 4: General Quadrilateral Geometry . . . 53
4.6.1 Geometry Parametrisation . . . 53
4.6.2 Convexity Check . . . 54
4.6.3 Network Definition . . . 55
4.6.4 Results . . . 56
4.7 Case 5: POD-Reduced Surrogate for Quadrilateral Geometry . . . 59
4.7.1 POD Reduction of Displacement Snapshots . . . 59
4.7.2 POD Truncation Error and Mode Shapes . . . 61
4.7.3 Coefficient Normalisation . . . 64
4.7.4 Network Definition . . . 64
4.7.5 Reconstruction of the Full Displacement Field . . . 64
4.7.6 Results . . . 65
4.8 Case 6: Structural Domains with Circular Holes . . . 67
4.8.1 Geometry Parametrisation . . . 67
4.8.2 Hole Representation . . . 67
4.8.3 Geometry Validity Check . . . 69
4.8.4 Network Definition . . . 69
4.8.5 Inference and Results . . . 70
4.9 Limitations of Data-Driven Models . . . 73

5 Physics-informed Neural Networks 74
5.1 Problem Definition . . . 75
5.2 Strong Form PINN Formulation . . . 75
5.2.1 Governing Equations . . . 75
5.2.2 Loss Function . . . 77
5.2.3 Preliminary Case: Fixed Geometry . . . 77

5.2.4 Parametrised Strong-Form PINN with Two Unit Body Force Case . . . 78
5.2.5 Results . . . 88
5.3 Energy-Based Variational PINN Formulation . . . 91
5.3.1 Variational Formulation . . . 91
5.3.2 Loss Function . . . 92
5.3.3 Parametrised Variational PINN with Two Unit Body Force Case . . . 93
5.3.4 Results . . . 95
5.4 Limitations of Physics-Informed Models . . . 98

6 Dynamic Extension: Modal Analysis 99
6.1 Problem Definition . . . 100
6.2 Finite Element Implementation . . . 100
6.2.1 Mass Normalisation . . . 104
6.2.2 Mode Ordering and Sign Correction . . . 104
6.3 Neural Network Architecture . . . 105
6.4 Training . . . 107
6.5 Inference . . . 108
6.6 Results . . . 108
6.6.1 Error Metrics . . . 108
6.6.2 Error Evaluation . . . 109
6.6.3 Computational Time . . . 111
6.7 Discussion . . . 112

7 Comparison of Surrogate Modelling Approaches 113
7.1 Comparison of Data-Driven Cases . . . 113
7.1.1 Prediction Accuracy . . . 114
7.1.2 Computational Cost . . . 114
7.1.3 Effect of Mesh Size . . . 115
7.1.4 Effect of Geometry Range . . . 116

7.1.5 Effect of General Quadrilateral Geometry 116
7.1.6 Effect of POD Integration 116
7.1.7 Effect of Circular Holes 117
7.2 Comparison of Physics-Informed Cases 117
7.2.1 Prediction Accuracy 118
7.2.2 Computational Time 119
7.3 Comparison of Data-driven and Physics-Informed Cases 119
7.3.1 Prediction Accuracy 120
7.3.2 Computational Time 121
7.3.3 Summary 122

Bibliography **127**

List of Figures

1.1 Isoparametric coordinate mapping between the natural reference domain (ξ, η) and the physical Cartesian domain (x, y) for a Q4 element. 13
1.2 Location of the 2×2 Gauss integration points within the natural coordinate system of a Q4 quadrilateral element. 15

2.1 Architecture of a simple fully-connected Feedforward Neural Network (Multilayer Perceptron) with one hidden layer. 23

4.1 Comparison of FEM and neural network displacement fields with absolute error plots for Case 1. 47
4.2 Effect of mesh size on the prediction error (Case 2). 50
4.3 Effect of geometry range on the prediction error (Case 3). 52
4.4 Comparison of FEM and neural network displacement fields with absolute error plots for Case 4. 58
4.5 Relative POD truncation error versus number of retained POD modes. 62
4.6 First POD modes for the horizontal body-force displacement snapshot matrix. 63
4.7 First POD modes for the vertical body-force displacement snapshot matrix. 63
4.8 Comparison of FEM and neural network displacement fields with absolute error plots for Case 5. 65
4.9 Comparison of FEM and neural network displacement fields with absolute error plots for Case 6. 72

5.1 Comparison of FEM and neural network displacement fields with absolute error plots for the strong-form PINN. 89
5.2 Recovered stress fields from the strong-form PINN prediction. . . . 90
5.3 Comparison of FEM and neural network displacement fields with absolute error plots for the variational PINN. 97
5.4 Recovered stress fields from the variational PINN prediction. 97

6.1 Comparison of FEM and neural network prediction with error plots for mode 1. 110
6.2 Comparison of FEM and neural network prediction with error plots for mode 2. 110
6.3 Comparison of FEM and neural network prediction with error plots for mode 3. 111

List of Tables

4.1 Dataset parameters for the two-body-force case (Case 1). 38
4.2 Training parameters for the two-body-force case (Case 1). 44
4.3 Test parameters for two-body-force case (Case 1). 46
4.4 Error values for Case 1. 47
4.5 Computational time for Case 1. 49
4.6 Parameters used for the mesh-effect study (Case 2). 49
4.7 Error comparison for different mesh sizes (Case 2). 50
4.8 Parameters used for the dimension-range study (Case 3). 51
4.9 Error comparison for increasing geometry ranges. 52
4.10 Parameters used for the quadrilateral geometry case (Case 4). 54
4.11 Test parameters for the quadrilateral geometry case (Case 4). 57
4.12 Prediction errors for the quadrilateral geometry case. 57
4.13 Computational time for the parameterised quadrilateral geometry case. 59
4.14 Main parameters used for the POD-reduced data-driven case (Case 5). 60
4.15 Prediction errors for the POD-reduced quadrilateral surrogate. . . . 65
4.16 Computational time for the POD-reduced quadrilateral surrogate (Case 5). 66
4.17 Parameters used for the two-hole structural domain case (Case 6). . 68
4.18 Test parameters for the two-hole structure case (Case 6). 71
4.19 Displacement prediction errors for the two-hole structure case (Case 6). 71
4.20 Computational time for the two-hole structural domain case (Case 6). 73

5.1 Collocation point parameters for the strong-form PINN. 79
5.2 Training parameters for the strong-form PINN. 86
5.3 Prediction errors for the strong-form PINN. 88
5.4 Computational time for the strong-form PINN. 90
5.5 Collocation point parameters for the variational PINN. 93
5.6 Training parameters for the variational PINN. 95
5.7 Prediction errors for the variational PINN. 96
5.8 Computational time for the variational PINN. 98

6.1 Material and model parameters used in the dynamic analysis. 101
6.2 Training parameters used for the modal neural network. 107
6.3 Modal prediction errors for the test geometry $B = 1.25$ and $H = 2.0$. 109
6.4 Average computational time for the dynamic analysis workflow. . . . 111

7.1 Comparison of selected data-driven surrogate cases. 114
7.2 Comparison of computational time for selected data-driven cases. . 115
7.3 Comparison of displacement prediction errors for the strong-form and variational PINNs. 118
7.4 Comparison of computational time for the strong-form and variational PINNs. 119
7.5 Comparison of displacement prediction errors for the selected data-driven and physics-informed models. 120
7.6 Comparison of computational times for the selected data-driven and physics-informed models. 122
7.7 Comparison summary of data-driven and physics-informed surrogate models. 123

Introduction

Structural analysis is an integral part of the engineering design process, as the performance and safety of structural components must be studied meticulously before manufacturing and construction. Analysing a structure or structural component can be a lengthy process, where several design alternatives, loading conditions, and geometric configurations are tested before selecting an optimised design. The same workflow applies in a wide range of applications, from the automotive and aerospace industry to construction and infrastructure development. The finite element method has been a reliable and widely used numerical analysis tool to solve complex structural problems. However, it comes with a computational cost that increases when many geometries, loading cases, mesh densities, or design iterations are required [1].

In recent years, engineering industries have been increasingly integrating artificial intelligence to support analysis and design. In structural mechanics, one important usage of artificial intelligence is in response prediction to relieve or reduce the cost of finite element simulations. This can be performed using surrogate models created by training neural networks to approximate the displacement, stress, or other responses of the structure without the need to perform a finite element simulation for every new case. Neural networks allow fast parametric analyses to explore the design space and provide directly the solution gradient that is useful to accelerate optimisation methods. Furthermore, neural networks have also been used in other civil and structural engineering problems, including material modelling, damage identification, and design optimisation [2, 3].

Data-driven neural networks are a common method for constructing surrogate

models. These networks are trained on preexisting or precomputed data and learn the mappings between the selected inputs and the corresponding outputs. For structural analysis, these data can be produced by finite element simulations. These neural networks learn by minimising the loss function that measures the difference between the output predicted by the neural network and the finite element solution. Once trained, the neural network can be used to predict the response of new configurations within the range of the training data.

An alternative method of surrogate model training is provided by physics-informed neural network, which do not necessarily require preexisting finite element data, rather they use the laws of physics. They predict structural response by learning the physics through governing equations, which long with the boundary conditions are integrated in the loss function. In computational mechanics, this idea is particularly applicable, as the governing equations of equilibrium, constitutive relations, and boundary conditions are already known. In addition to the strong-form formulation, variational physics-informed approaches can also be used, where the loss function is derived from an integral or energy-based form of the governing problem [4, 5].

This thesis aims to use such data-driven and physics-informed neural networks to create surrogate models for two-dimensional plane-stress problems. Initially, a finite element solver for the static and modal analysis of plates was developed. It is used to generate datasets for parametrised geometries, which is then used to train data-driven neural networks. These networks predict the displacement and the stress fields for the variations of the geometric cases of the static structural problem and the modes of the dynamic problem. Simultaneously, physics-informed neural networks are also developed to solve rectangular plane-stress plate problem using strong-form and variational formulations both. Absolute and relative error metrics and visual plots are used to evaluate the prediction accuracy of the different cases. Finally, the different approaches are compared for their relative advantages and disadvantages.

The scope of this work is limited to two-dimensional linear elastic plane stress problems under small deformation assumptions. However, the methodology can be

easily extended to other structural models. The results of finite elements are used as numerical reference solutions and the neural networks are tested within the range of geometries and loading conditions used during training. The neural-network models developed in this work should be treated as numerical surrogate models for the selected cases of structural problems rather than general replacements for full finite element analysis.

The main objective of this thesis is to develop and evaluate neural-network surrogate models for the structural analysis of parametrised two-dimensional plane-stress plates. Both data-driven and physics-informed formulations are compared for their prediction accuracy and their efficiency in terms of training time. Their performance is also assessed against reference finite element solutions in terms of prediction accuracy, computational efficiency, data requirements, and robustness. Through these comparisons, the study aims to determine the suitability and limitations of each approach to replace repeated finite element simulations.

The thesis is organised as follows. Chapter 1 presents the theoretical background of linear elasticity, plane stress assumptions, and the finite element formulation used in this work. Chapter 2 introduces the concepts of artificial neural networks that will be used in structural analysis. Chapter 3 discusses the methodology that will be followed throughout the work. Chapters 4 and Chapter 5 introduce the work done in building data driven and physics informed surrogate models, respectively. Chapter 6 extends the work to a dynamic case. Chapter 7 compares all the various cases studied, and then finally the concluding chapter summarises the main findings and discusses possible future developments.

Chapter 1

Theoretical Background: Structural Analysis

This chapter establishes the theoretical baseline for the main structural concepts used in this thesis. A plane-stress structural problem is targeted in this work, which follows the assumptions of linear elasticity. It is further extended into a dynamic problem to solve for the eigenmodes. The Finite Element Method (FEM) forms the key foundation for structural analysis in this work, serving as the comparative standard for the results of the surrogate models, developed using data-driven and physics-informed neural networks.

In this chapter and the subsequent ones, bold lowercase symbols denote vectors, while bold uppercase symbols denote matrices. Non-bold italic symbols denote scalar quantities.

1.1 Linear Elasticity and Plane Stress Formulation

For structural analysis, the focus is on the theory of linear elasticity for most of the work before extending to dynamic analysis. In **linear elasticity**, the assumption states that the material deformation is reversible under small strains and small displacements. Thus, the relationship between stress and strain is also linear [6–8].

Where $\Omega \subset \mathbb{R}^2$ is the two-dimensional domain that the structure occupies, and a boundary defined as Γ, linear elasticity can be used to define the displacement field as:

$$\mathbf{u}(x,y) = \begin{bmatrix} u(x,y) \\ v(x,y) \end{bmatrix}, \tag{1.1}$$

where u and v are the displacement components in the x and y directions, respectively.

Under small strain assumptions, the **strain tensor** is obtained from the symmetric part of the displacement gradient:

$$\boldsymbol{\varepsilon} = \frac{1}{2}\left(\nabla \mathbf{u} + \nabla \mathbf{u}^T\right) \tag{1.2}$$

For a two-dimensional problem, the strain can be calculated from the displacement and the **strain vector** can be written in Voigt notation as:

$$\boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \gamma_{xy} \end{bmatrix} = \begin{bmatrix} \dfrac{\partial u}{\partial x} \\ \dfrac{\partial v}{\partial y} \\ \dfrac{\partial u}{\partial y} + \dfrac{\partial v}{\partial x} \end{bmatrix} \tag{1.3}$$

where ε_{xx} and ε_{yy} represent the normal strain components, and γ_{xy} represents the engineering shear strain.

The corresponding in-plane stress components in the **stress vector** are:

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \tau_{xy} \end{bmatrix} \tag{1.4}$$

For a body in static equilibrium, the **strong form of governing equilibrium equations**, governed by linear momentum, are given as:

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{in } \Omega \tag{1.5}$$

where $\mathbf{b} = [b_x, b_y]^T$ represents the body force vector per unit volume (such as gravity). When expanded in their components, these **equilibrium partial differential equations** can be written as:

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \tau_{xy}}{\partial y} + b_x = 0 \tag{1.6}$$

$$\frac{\partial \tau_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + b_y = 0 \tag{1.7}$$

This creates a **boundary value problem**, that requires the specification of the boundary conditions to be complete. The total boundary Γ can be divided into a displacement (Dirichlet) boundary Γ_u and a traction (Neumann) boundary Γ_t, which are explained later in this chapter, satisfying:

$$\Gamma = \Gamma_u \cup \Gamma_t, \quad \Gamma_u \cap \Gamma_t = \emptyset \tag{1.8}$$

The boundary conditions are formally defined as:

$$\mathbf{u} = \bar{\mathbf{u}} \quad \text{on } \Gamma_u \tag{1.9}$$

$$\boldsymbol{\sigma}\mathbf{n} = \bar{\mathbf{t}} \quad \text{on } \Gamma_t \tag{1.10}$$

where $\bar{\mathbf{u}}$ is the prescribed displacement, $\bar{\mathbf{t}}$ is the prescribed surface traction, and $\mathbf{n}$ is the unit normal vector pointing outward from the boundary.

The stress and strain fields are defined by the **constitutive law**, which in case of an isotropic linear elastic material is the **Hooke's law**, given as:

$$\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon} \tag{1.11}$$

where $\mathbf{D}$ is the material's elasticity matrix. In this thesis, the continuous three-dimensional elasticity equations are reduced to two dimensions by adopting the plane-stress assumption.

The **plane-stress condition** can be assumed for thin structures where the thickness in the z-direction is negligible compared to the in-plane dimensions, and all

external loads are applied strictly parallel to the xy-plane. Under these assumptions, the out-of-plane stress components are assumed to vanish:

$$\sigma_{zz} = 0, \quad \tau_{xz} = 0, \quad \tau_{yz} = 0 \tag{1.12}$$

The out-of-plane stress is zero but the out-of-plane strain (ε_{zz}) is non-zero. Poisson's effect causes the thin structural domain to expand or contract in the thickness direction according to the in-plane deformation, governed by:

$$\varepsilon_{zz} = -\frac{\nu}{E}\left(\sigma_{xx} + \sigma_{yy}\right) \tag{1.13}$$

where E is Young's modulus and ν is Poisson's ratio [6].

For an isotropic material under plane stress conditions, the **constitutive matrix D** is formulated as:

$$\mathbf{D} = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \dfrac{1-\nu}{2} \end{bmatrix} \tag{1.14}$$

To summarise, the plane stress formulation is governed by three main laws, the **equilibrium equations**, the **strain-displacement relations**, and the **constitutive law**. These laws and the boundary conditions form a complete boundary value problem for the displacement field. In the finite element method, this continuous problem is transformed into a discrete algebraic system by approximating the displacement field over a mesh of finite elements.

Plane stress analysis has also historically been used as the baseline to develop the finite element method. Early finite element work in structural mechanics considered stiffness-based methods for complex structures [9], and Clough's work was one of the pioneering ones establishing the finite element method as an effective tool for continuum mechanics problems [10]. In this thesis, the same general idea is followed, where the plane stress elasticity problem is solved using the finite element method, and the resulting numerical data are later used to train neural network surrogate models.

1.2 Derivation of the Weak Form

Finite element discretisation uses equations in their weak form, obtained from the strong-form differential equations using the variational formulation [8]. The **weak form** serves as an alternative, providing an integral representation of the conventional governing equations, while the strong form requires the equilibrium equations to be satisfied pointwise.

An arbitrary **virtual displacement field** (or **test function**) is introduced as:

$$\delta\mathbf{u} = \begin{bmatrix} \delta u \\ \delta v \end{bmatrix}, \tag{1.15}$$

The homogeneous essential (Dirichlet) boundary condition must be satisfied by the virtual displacement field:

$$\delta\mathbf{u} = \mathbf{0} \qquad \text{on } \Gamma_u. \tag{1.16}$$

Then multiplying the strong-form equilibrium equation by the virtual displacement field and integrating over the domain gives

$$\int_\Omega \delta\mathbf{u}^T \left(\nabla \cdot \boldsymbol{\sigma} + \mathbf{b}\right) t \mathrm{d}\Omega = 0, \tag{1.17}$$

where t is the constant thickness of the two-dimensional plane-stress domain. Applying integration by parts, together with the divergence theorem, gives

$$\int_\Omega \delta\boldsymbol{\varepsilon}^T \boldsymbol{\sigma} t \mathrm{d}\Omega = \int_\Omega \delta\mathbf{u}^T \mathbf{b} t \mathrm{d}\Omega + \int_\Gamma \delta\mathbf{u}^T \left(\boldsymbol{\sigma}\mathbf{n}\right) t \mathrm{d}\Gamma, \tag{1.18}$$

where $\delta\boldsymbol{\varepsilon}$ is the virtual strain vector associated with $\delta\mathbf{u}$.

The boundary is divided into the displacement boundary Γ_u and the traction boundary Γ_t. Since $\delta\mathbf{u} = \mathbf{0}$ on Γ_u, the corresponding boundary contribution vanishes. Applying the prescribed traction condition

$$\boldsymbol{\sigma}\mathbf{n} = \bar{\mathbf{t}} \qquad \text{on } \Gamma_t \tag{1.19}$$

leads to the weak form

$$\int_{\Omega} \delta\boldsymbol{\varepsilon}^T \boldsymbol{\sigma} t, \mathrm{d}\Omega = \int_{\Omega} \delta\mathbf{u}^T \mathbf{b} t \mathrm{d}\Omega + \int_{\Gamma_t} \delta\mathbf{u}^T \bar{\mathbf{t}} t \mathrm{d}\Gamma. \tag{1.20}$$

This equation shows the **principle of virtual work** which states the internal virtual work equals the external virtual work for every admissible virtual displacement field. The displacement boundary condition is imposed directly on the displacement field, but the prescribed traction comes naturally through the boundary integral.

1.2.1 Relation to the Total Potential Energy

For a conservative linear elastic problem, the weak equilibrium statement requires the total potential energy to be stationary with respect to all admissible virtual displacements. The total potential energy is

$$\Pi(\mathbf{u}) = \frac{1}{2} \int_{\Omega} \boldsymbol{\varepsilon}^T \boldsymbol{\sigma}\, t \, \mathrm{d}\Omega - \int_{\Omega} \mathbf{u}^T \mathbf{b}\, t \, \mathrm{d}\Omega - \int_{\Gamma_t} \mathbf{u}^T \bar{\mathbf{t}}\, t \, \mathrm{d}\Gamma. \tag{1.21}$$

The equilibrium solution makes the first variation of this functional vanish:

$$\delta\Pi(\mathbf{u}) = 0. \tag{1.22}$$

For a stable linear elastic system, the equilibrium displacement field also minimises the total potential energy. This energy principle is used later in the variational physics-informed neural network formulation.

1.3 Finite Element Method

The weak form derived in the previous section is used to build the finite element formulation. The continuous weak-form formulation is converted into a finite algebraic system of equations, as the finite element method approximates the displacement field over a discretised mesh using nodal values and shape functions.

In engineering mechanics, the finite element method is a powerful numerical tool to solve boundary value problems. It can discretise a continuous domain

into smaller elements, allowing complex geometries, material properties, loading conditions, and boundary conditions to be represented using a computational framework [8, 11, 12]

1.3.1 Domain Discretisation

The finite element method uses domain discretisation to solve the continuous structural domain. It divides the domain into several subdomains called **elements**, formed with discrete points called **nodes** [11]. The finite element model solves for the unknown values at the nodes and this system of nodes and elements is called a **mesh**. The accuracy of the solution heavily depends on the mesh density, element type (triangular, quadrilateral, etc.) and the shape functions which approximate the solution inside each element. [11, 12]

1.3.2 Shape Functions

Shape functions are mathematical interpolation functions used to approximate the spatial variation of field variables within a finite element. In this thesis, the primary field variable is the displacement field.

The finite element approximation of a scalar field variable u within an element is:

$$u(\mathbf{x}) \approx \sum_{i=1}^{n_e} N_i(\mathbf{x}) u_i, \tag{1.23}$$

where $\mathbf{x}$ is the spatial position vector of a point within the element, n_e is the number of nodes in the element, u_i is the nodal value of the field variable, and $N_i(\mathbf{x})$ is the shape function associated with node i, analysed at position $\mathbf{x}$.

For a two-dimensional displacement field, the displacement vector is written as:

$$\mathbf{u}(\mathbf{x}) = \begin{bmatrix} u(\mathbf{x}) \\ v(\mathbf{x}) \end{bmatrix}, \tag{1.24}$$

where u and v are the displacement components in the x and y directions, respectively.

Another property of shape functions is the partition of unity:

$$\sum_{i=1}^{n_e} N_i(\mathbf{x}) = 1. \tag{1.25}$$

For standard nodal finite elements, the shape functions also satisfy the Kronecker delta property at the nodes:

$$N_i(\mathbf{x}_j) = \delta_{ij}, \tag{1.26}$$

i.e., a shape function associated with a particular node will be equal to one at that specific node and zero at all the other nodes [8, 11].

1.3.3 Element Stiffness Matrix Formulation

The finite element approximation is built for elements initially by substituting the displacement approximations into the continuous weak form. Within an element, the displacement and virtual displacement fields are approximated as

$$\mathbf{u}^e = \mathbf{N}\mathbf{u}_e, \qquad \delta\mathbf{u}^e = \mathbf{N}\delta\mathbf{u}_e, \tag{1.27}$$

where $\mathbf{N}$ is the shape-function matrix, $\mathbf{u}_e$ is the element nodal displacement vector, and $\delta\mathbf{u}_e$ is the corresponding arbitrary virtual nodal displacement vector. The strain and virtual strain fields are then

$$\boldsymbol{\varepsilon}^e = \mathbf{B}\mathbf{u}_e, \qquad \delta\boldsymbol{\varepsilon}^e = \mathbf{B}\delta\mathbf{u}_e, \tag{1.28}$$

and the stress field is

$$\boldsymbol{\sigma}^e = \mathbf{D}\mathbf{B}\mathbf{u}_e, \tag{1.29}$$

where **B** is the **strain-displacement matrix** containing spatial derivatives of shape functions, **D** is the constitutive matrix, and $\mathbf{u}_e$ is the vector of real nodal displacements.

Substituting these approximations into the left-hand side (internal virtual work) of the weak form yields:

$$\int_{\Omega_e} \delta\boldsymbol{\varepsilon}^T \boldsymbol{\sigma} t \, \mathrm{d}\Omega = \delta\mathbf{u}_e^T \left[\left(\int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} t \, \mathrm{d}\Omega \right) \mathbf{u}_e \right] = \delta\mathbf{u}_e^T \mathbf{K}_e \mathbf{u}_e \tag{1.30}$$

Since the virtual nodal displacements $\delta\mathbf{u}_e^T$ are arbitrary, this identifies the standard **element stiffness matrix** $\mathbf{K}_e$:

$$\mathbf{K}_e = \int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} t \, \mathrm{d}\Omega \tag{1.31}$$

where Ω_e is the element domain and t is the element thickness. This formulation is derived using the virtual work method for linear elasticity [8, 12].

1.3.4 Bilinear Isoparametric Mapping

In finite element analysis, the physical geometries to be solved are mapped onto standardised reference domains before the analysis begins, in the process called **isoparametric mapping**. Isoparametric elements use the same interpolation functions, or shape functions N_i, to describe both the element geometry and the internal displacement fields [12].

Coordinates in the physical domain (x, y) of a two-dimensional, four-node bilinear quadrilateral (Q4) element are mapped on a natural (parent) coordinate system $(\xi, \eta) \in [-1, 1]^2$ as follows:

$$x(\xi, \eta) = \sum_{i=1}^{4} N_i(\xi, \eta) x_i, \quad y(\xi, \eta) = \sum_{i=1}^{4} N_i(\xi, \eta) y_i \tag{1.32}$$

where (x_i, y_i) are the physical coordinates of the element's nodes, and the bilinear shape functions N_i are defined as:

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta) \quad (i = 1, 2, 3, 4) \tag{1.33}$$

where (ξ_i, η_i) are the natural coordinates of node i. When the element geometry is valid and the Jacobian determinant is positive, this mapping sets up a unique, one-to-one spatial relationship between the natural and physical domains, ensuring that displacement fields remain compatible across element boundaries. Figure 1.1 shows isoparametric mapping for a Q4 element.

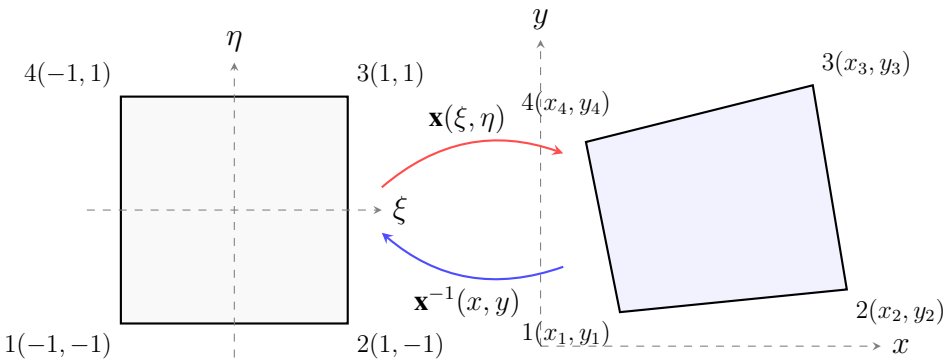


Figure 1.1. Isoparametric coordinate mapping between the natural reference domain (ξ, η) and the physical Cartesian domain (x, y) for a Q4 element.

1.3.5 The Jacobian Matrix

To evaluate spatial derivatives and perform numerical integration over the physical element, coordinate derivatives are related to the natural coordinates using the Jacobian matrix, $\mathbf{J}$ [8]. This Jacobian matrix is given as:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} \end{bmatrix} = \sum_{i=1}^{4} \begin{bmatrix} \frac{\partial N_i}{\partial \xi} x_i & \frac{\partial N_i}{\partial \xi} y_i \\ \frac{\partial N_i}{\partial \eta} x_i & \frac{\partial N_i}{\partial \eta} y_i \end{bmatrix} \tag{1.34}$$

The Jacobian acts as a local transformation factor, indicating how much the master element is stretched, rotated, or distorted to fit the physical geometry. Its determinant $\det(\mathbf{J})$ scales the differential area from Cartesian to natural coordinates:

$$\mathrm{d}x\mathrm{d}y = \det(\mathbf{J})\, \mathrm{d}\xi\mathrm{d}\eta \tag{1.35}$$

The value of $\det(\mathbf{J})$ is a critical measure for the finite element mesh quality:

- $\det(\mathbf{J}) > 0$: The mapping is valid and preserves orientation.
- $\det(\mathbf{J}) \approx 0$: The element becomes distorted, leading to unreliable derivative transformations.
- $\det(\mathbf{J}) \leq 0$: The mapping is invalid. The element is inverted, violating physical geometry.

1.3.6 Gauss Quadrature

The element matrices require numerical integration over each element domain. After isoparametric mapping, the integrals are evaluated in the natural coordinate system using Gauss quadrature.

The integrals appearing in the element stiffness and force expressions are evaluated numerically. Using coordinate mapping, the element stiffness matrix $\mathbf{K}_e$ is converted to the natural coordinate system $(\xi, \eta) \in [-1, 1]^2$ from physical coordinates:

$$\mathbf{K}_e = \int_{-1}^{1} \int_{-1}^{1} \mathbf{B}(\xi, \eta)^T \mathbf{D}\mathbf{B}(\xi, \eta)\, t\, \det\left(\mathbf{J}(\xi, \eta)\right)\, \mathrm{d}\xi\, \mathrm{d}\eta. \tag{1.36}$$

where $\mathbf{B}$ is the strain-displacement matrix and $\mathbf{D}$ is the constitutive material matrix.

To solve this integral numerically, Gauss-Legendre quadrature is applied. This method approximates the continuous integral by summing the function values at specific optimal points, called **Gauss points**, multiplied by the corresponding weight factors [11]:

$$\mathbf{K}_e \approx \sum_{i=1}^{n_{\text{gp}}} \sum_{j=1}^{n_{\text{gp}}} w_i w_j\, \mathbf{B}(\xi_i, \eta_j)^T \mathbf{D}\mathbf{B}(\xi_i, \eta_j)\, t\, \det\left(\mathbf{J}(\xi_i, \eta_j)\right). \tag{1.37}$$

where (ξ_i, η_j) are the natural coordinates of the Gauss integration points, w_i and w_j are their weights, and n_{gp} is the number of integration points per direction.

For standard 4-node bilinear quadrilateral (Q4) elements, a 2×2 integration scheme ($n_{gp} = 2$) is used. The integration coordinates are located at $\xi_i, \eta_j =$

$\pm 1/\sqrt{3} \approx \pm 0.5774$ with weights $w_i, w_j = 1.0$, as shown in Figure 1.2. In addition to their role in numerical integration, during element calculations, strains and stresses are evaluated at these integration points [8].

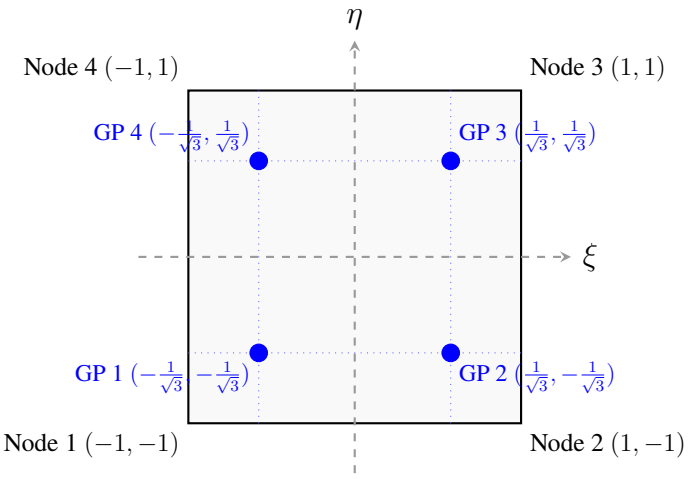


Figure 1.2. Location of the 2×2 Gauss integration points within the natural coordinate system of a Q4 quadrilateral element.

1.3.7 Global Stiffness Matrix Assembly

The global stiffness matrix is formed by assembling the element stiffness matrix of each element based on its contribution to the global degrees of freedom. The element connectivity matrix plays a key role in connecting the respective local element nodes to their corresponding global element nodes in the mesh.

The assembled system of equations for a **static linear elastic problem** is written as:

$$\mathbf{Ku} = \mathbf{F}, \tag{1.38}$$

where $\mathbf{K}$ is the global stiffness matrix, $\mathbf{u}$ is the global nodal displacement vector, and $\mathbf{F}$ is the global force vector.

1.3.8 Global Force Vector Assembly

To resolve the system of equations, external loads acting on the continuous domain must also be mapped to the discrete global nodes. The element force vector due to a body force per unit volume, **b**, is calculated in the weak form as:

$$\mathbf{F}_e = \int_{\Omega_e} \mathbf{N}^T \mathbf{b} t d\Omega, \tag{1.39}$$

where **N** is the matrix of shape functions and t is the element thickness.

Similarly, the equivalent nodal force vector due to a prescribed boundary traction $\bar{\mathbf{t}}$ acting along an element boundary Γ_e is expressed as:

$$\mathbf{F}_e = \int_{\Gamma_e} \mathbf{N}^T \bar{\mathbf{t}} t d\Gamma. \tag{1.40}$$

These element-level vectors are then assembled into the global force vector using the same connectivity matrix used for the stiffness matrix. The global force vector may include concentrated nodal forces, distributed surface tractions, and body forces. [11, 12].

1.3.9 Boundary Conditions

Each finite element problem has defined boundary conditions that explains how the structure interacts with its surroundings. Two main kinds of boundary conditions mainly used are Dirichlet and Neumann boundary conditions.

Dirichlet boundary condition is also called the essential boundary condition, and it defines the value of primary field variable on the boundary. In structural mechanics, this usually means prescribed displacement. **Neumann boundary condition**, also known as the natural boundary condition, prescribes flux-type quantities. In elasticity, where the primary unknown is displacement, the corresponding Neumann condition is expressed as a prescribed traction.

1.3.10 Matrix Partitioning for Boundary Conditions

Matrix partitioning method for boundary conditions is one of the ways of imposing the Dirichlet (essential) boundary conditions. This thesis uses the method to separate the constrained degrees of freedom from the unknown, free degrees of freedom allowing for a strong imposition of essential boundary conditions and reducing the system's dimensionality before solving [8, 12].

The unconstrained global static system of equations is:

$$\mathbf{K}\mathbf{u} = \mathbf{F} \tag{1.41}$$

Designating the subscript f for free degrees of freedom and c for constrained degrees of freedom, the global displacement vector $\mathbf{u}$ and global force vector $\mathbf{F}$ are partitioned into their respective subvectors:

$$\mathbf{u} = \begin{bmatrix} \mathbf{u}_f \\ \mathbf{u}_c \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} \mathbf{F}_f \\ \mathbf{F}_c \end{bmatrix} \tag{1.42}$$

Applying this same partitioning to the global stiffness matrix $\mathbf{K}$ produces the fully partitioned block matrix system:

$$\begin{bmatrix} \mathbf{K}_{ff} & \mathbf{K}_{fc} \\ \mathbf{K}_{cf} & \mathbf{K}_{cc} \end{bmatrix} \begin{bmatrix} \mathbf{u}_f \\ \mathbf{u}_c \end{bmatrix} = \begin{bmatrix} \mathbf{F}_f \\ \mathbf{F}_c \end{bmatrix} \tag{1.43}$$

Expanding the first row of this block matrix system results in the following:

$$\mathbf{K}_{ff}\mathbf{u}_f + \mathbf{K}_{fc}\mathbf{u}_c = \mathbf{F}_f \tag{1.44}$$

A reduced system of linear equations with only the unknown free degrees of freedom can be formed, as the prescribed displacement values at the boundary constraints ($\mathbf{u}_c$) are known:

$$\mathbf{K}_{ff}\mathbf{u}_f = \mathbf{F}_f - \mathbf{K}_{fc}\mathbf{u}_c \tag{1.45}$$

In the case of homogeneous Dirichlet boundary conditions where the constrained boundary is fully fixed:

$$\mathbf{u}_c = \mathbf{0} \tag{1.46}$$

the partitioned system simplifies directly to:

$$\mathbf{K}_{ff}\mathbf{u}_f = \mathbf{F}_f \tag{1.47}$$

After solving this reduced linear system for the free displacement vector $\mathbf{u}_f$, the complete global displacement vector $\mathbf{u}$ is reconstructed by inserting the prescribed displacement values ($\mathbf{u}_c$) back into their corresponding constrained degree-of-freedom positions.

1.3.11 Nodal Displacements Solution

Using the reduced system from matrix partitioning method, the unknown nodal displacements are solved for, once the global stiffness matrix, force vector, and boundary conditions are assembled:

$$\mathbf{u}_f = \mathbf{K}_{ff}^{-1}\mathbf{F}_f. \tag{1.48}$$

In actual finite element solvers, the inverse of the stiffness matrix is typically not computed explicitly. Instead, numerical linear solvers are used to solve the system efficiently. The computed nodal displacements can then be used to evaluate strains and stresses inside each element through:

$$\boldsymbol{\varepsilon} = \mathbf{B}\mathbf{u}_e, \qquad \text{and} \qquad \boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}. \tag{1.49}$$

1.4 Structural Dynamics and Modal Analysis

After completing the static finite element formulation, this thesis extends the work to a dynamic analysis, specifically to a modal analysis. Modal analysis determines the natural vibration characteristics of a structure, its natural frequencies and the corresponding mode shapes. These characteristics describe how a structure vibrates

from its equilibrium position when disturbed. This is called the **free vibration state**. Natural frequencies are the key to understanding structural dynamics as they provide insight into the deformation patterns that dominate the dynamic behaviour of a system [13–15].

1.4.1 Global Mass Matrix Assembly

For dynamic problems, the inertia of the structure must be considered; therefore, the finite element formulation also requires the construction of a mass matrix. The **element mass matrix** is calculated as:

$$\mathbf{M}_e = \int_{\Omega_e} \rho \mathbf{N}^T \mathbf{N}\, t\, d\Omega, \tag{1.50}$$

where ρ is the material density.

The global mass matrix $\mathbf{M}$ is constructed in a way similar to the assembly of the stiffness matrix by assembling the individual element mass matrices $\mathbf{M}_e$ using the connectivity relations.

1.4.2 Equation of Motion and Modal Analysis

For a linear elastic structure, the dynamic equilibrium equation can be written as:

$$\mathbf{M}\ddot{\mathbf{u}}(t) + \mathbf{C}\dot{\mathbf{u}}(t) + \mathbf{K}\mathbf{u}(t) = \mathbf{F}(t), \tag{1.51}$$

where $\mathbf{M}$ is the mass matrix, $\mathbf{C}$ is the damping matrix, $\mathbf{K}$ is the stiffness matrix, $\mathbf{u}(t)$ is the displacement vector, $\dot{\mathbf{u}}(t)$ and $\ddot{\mathbf{u}}(t)$ are the velocity and acceleration vectors, and $\mathbf{F}(t)$ is the external force vector. This equation represents the balance between inertial forces, damping forces, internal elastic forces, and externally applied forces [13, 14].

In free vibration analysis, there is no external force causing the structure to vibrate. The equation of motion changes to this if damping is also disregarded:

$$\mathbf{M}\ddot{\mathbf{u}}(t) + \mathbf{K}\mathbf{u}(t) = \mathbf{0}. \tag{1.52}$$

This undamped free vibration problem is the basis of classical modal analysis and is used later in this thesis for the prediction of mode shapes.

The generalized eigenvalue problem can be represented as:

$$\mathbf{K}\boldsymbol{\phi} = \omega^2 \mathbf{M}\boldsymbol{\phi}. \tag{1.53}$$

The eigenvalues of this problem are ω_i^2, where ω_i is the i-th natural circular frequency. The corresponding eigenvectors $\boldsymbol{\phi}_i$ are the mode shapes. The natural frequency f_i, expressed in cycles per second, is obtained as:

$$f_i = \frac{\omega_i}{2\pi}. \tag{1.54}$$

The **mode shapes** represent the characteristic deformation patterns associated with the natural frequencies of the structure. The modes are ordered according to increasing natural frequency, where the lowest mode refers to the lowest natural frequency. Lower modes generally represent smooth, global deformation patterns, whereas higher modes exhibit greater spatial variations [14, 15].

For an undamped linear system with symmetric positive definite mass and stiffness matrices, the mode shapes satisfy **orthogonality properties**. For two different modes i and j, the following relations are satisfied:

$$\boldsymbol{\phi}_i^T \mathbf{M} \boldsymbol{\phi}_j = 0 \qquad i \neq j, \tag{1.55}$$

and

$$\boldsymbol{\phi}_i^T \mathbf{K} \boldsymbol{\phi}_j = 0 \qquad i \neq j. \tag{1.56}$$

These orthogonality properties are important because they allow the dynamic response of a structure to be expressed as a combination of independent modal contributions. Essentially, the displacement response can be approximated as:

$$\mathbf{u}(t) \approx \sum_{i=1}^{m} q_i(t) \boldsymbol{\phi}_i, \tag{1.57}$$

where $q_i(t)$ is the modal coordinate associated with mode i, and m is the number of retained modes. Representing modes like this is favored in structural dynamics as it provides a physical interpretation of the vibration behavior and reduces the complexity of modal analysis results [13, 14].

The magnitude of an eigenvector is arbitrary, therefore it is common to normalize modes before comparison or further interpretation. **Mass normalization** is normally performed where each mode is scaled according to the equation:

$$\boldsymbol{\phi}_i^T \mathbf{M} \boldsymbol{\phi}_i = 1. \tag{1.58}$$

Alternatively, mode shapes can be normalized with respect to their maximum displacement component. Normalization method does not change the actual physical deformation of the mode, but only the numerical values in the eigenvector. Before training neural networks, it is important to account for inconsistent signs or scaling of the eigenvectors and to apply appropriate normalization and sign corrections for consistent modal data.

The static and modal finite element solutions developed in this chapter form the reference data used in the later chapters for training and evaluating neural network surrogate models.

Chapter 2

Theoretical Background: Artificial Neural Networks

This chapter continues the theoretical background by introducing the artificial intelligence concepts used in this thesis, particularly artificial neural networks. Finite element method provides accurate numerical solutions but repeated finite element calculations can become computationally expensive, especially when trial-and-error or multiple structural configurations are required in design. As artificial intelligence is increasingly being used in engineering to improve workflows, structural analysis is also moving toward surrogate modelling approaches.

Artificial neural networks can be used as surrogate models to approximate the results of structural analysis using either finite element data or physics integration. This work focuses on training various neural networks to learning the mapping between inputs, such as geometric parameters, and outputs, such as displacements, stresses, or mode shapes. Once the network is trained, it can predict the structural behaviour under specific conditions for newer configurations of that structure at a much lower computational cost than a finite element solver. The use of surrogate models in such applications is supported by the approximation capability of feedforward neural networks for nonlinear functions under suitable conditions [16–18].

2.1 Feedforward Neural Networks

Artificial neural networks are inspired by the neurological connections of biological brains and are designed to learn the mappings or underlying relationships in a set of data the same way a brain does. The information passes through simple processors called **neurons**, which are organized in structured groups called **layers**.

The architecture used in this work is the feedforward neural network where information flows in one direction, also known as a **multilayer perceptron**. It consists of an input layer, which takes in raw data, one or more hidden layers, which perform the transformation to map the input data and an output layer, which presents the network's prediction.

Figure 2.1 shows a simple multi-layer perceptron, with one hidden layer, showing input features (x_i), predicted outputs ($\bar{y}_k$), and the path of weights (w) and biases (b). Weights provide a quantitative strength to the relationship between two neurons and biases provide an offset or threshold to the weighted inputs.

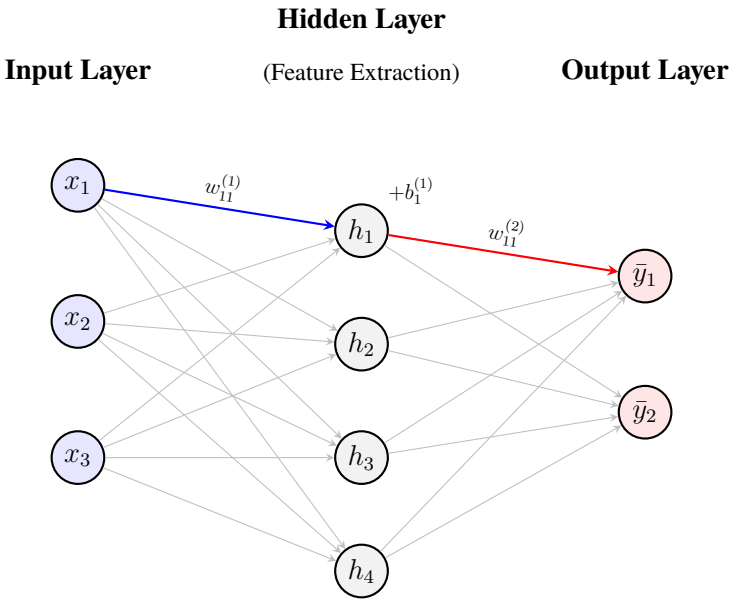


Figure 2.1. Architecture of a simple fully-connected Feedforward Neural Network (Multilayer Perceptron) with one hidden layer.

The hidden layer extracts and recombines the features from the input data using

the assigned weights. For a hidden layer l, the operation can be written as:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \tag{2.1}$$

$$\mathbf{a}^{(l)} = f\left(\mathbf{z}^{(l)}\right), \tag{2.2}$$

where $\mathbf{W}^{(l)}$ is the weight matrix, $\mathbf{b}^{(l)}$ is the bias vector, f is the activation function (described in a following subsection), and $\mathbf{a}^{(l)}$ is the output layer. The complete network prediction can be expressed as:

$$\hat{\mathbf{y}} = \mathcal{N}(\mathbf{x}; \boldsymbol{\theta}), \tag{2.3}$$

where $\mathbf{x}$ is the input vector, $\hat{\mathbf{y}}$ is the predicted output, and $\boldsymbol{\theta}$ contains all trainable weights and biases.

2.2 Data-Driven and Physics-Informed Networks

There has been research on various kinds of neural network, but for the purpose of this research, the focus remains on two broad categories: data-driven neural networks and physics-informed neural networks.

Data-driven neural networks work as interpolative surrogate models, which undergo supervised learning by being trained on pre-generated datasets, for instance, the ones generated through finite element simulations. These networks aim to predict structural outputs, such as displacement or stress fields, by studying inputs and minimising the loss function over the training data set. While they are flexible during training and fast during inference, they rely heavily on the availability of high-quality and large datasets for training and can be faulty in interpolation when dealing with unseen geometries or evaluating out-of-range data.

These data-dependency and generalisation limitations can be overcome by **physics-informed neural networks (PINNs)**, which integrate the governing physical laws and differential equations of structural mechanics directly into the training process [19]. The differential equations of equilibrium, boundary conditions, and

material constitutive relations are incorporated directly into the PINNs' loss function rather than depending only on pre-generated data. The network determines the physics loss based on the violation of physical principles by using automatic differentiation tools to get partial derivatives. This integration of domain knowledge allows PINNs to achieve high generalisation accuracy, enforce physical consistency, and train successfully even in scenarios where the data set is scarce or not available [4, 20].

2.3 Main Features of Neural Networks

2.3.1 Activation Functions

Activation functions introduce nonlinearity into neural networks. Without this nonlinearity, neural networks would just be capable of creating a simple linear transformation. Therefore, these activation functions allow the network to map complex relationships between inputs and outputs [18]. Some commonly used activation functions that are mentioned or used in this work are discussed below.

The **sigmoid activation function** is a very basic and known activation function. It can be defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \tag{2.4}$$

with output in the interval $(0, 1)$. However, it may suffer from vanishing gradients for very large or very small inputs [21].

The **hyperbolic tangent activation function** is defined as:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}, \tag{2.5}$$

with output in $(-1, 1)$. It is zero-centred and is therefore often more suitable than the sigmoid function.

The **rectified linear unit (ReLU)** is defined as:

$$\text{ReLU}(x) = \max(0, x). \tag{2.6}$$

Its output range is $[0, \infty)$, and it is widely used because of its simplicity and computational efficiency [22].

2.3.2 Loss Functions and Error Metrics

The network learns from its mistakes, which is characterized as the loss function. The **loss function** defines the difference between the neural network prediction and the reference solution, which in case of data-driven networks are FEM solutions.

One of the most widely used loss function for regression problems is the **mean squared error (MSE)**:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 , \tag{2.7}$$

where y_i is the reference value, $\hat{y}_i$ is the predicted value, and N is the number of output components.

In this work, after training, additional error metrics are used to compare the neural network prediction with the finite element reference solution.

The **Root Mean Squared Error (RMSE)** is used to evaluate the average absolute discrepancy:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \bar{y}_i)^2} \tag{2.8}$$

The **relative** L^2 **error** measures the normalised global prediction error over all response components:

$$e_{L^2} = \frac{|\mathbf{y} - \hat{\mathbf{y}}|_2}{|\mathbf{y}|_2}, \tag{2.9}$$

where $\mathbf{y}$ is the reference finite element response vector, $\hat{\mathbf{y}}$ is the predicted response vector, and the L^2 norm of a vector $\mathbf{z}$ is defined as

$$|\mathbf{z}|_2 = \sqrt{\sum_{i=1}^{n} z_i^2}. \tag{2.10}$$

The relative L^2 error therefore represents the overall difference between the predicted and reference responses relative to the magnitude of the reference solution.

The **relative L^∞ error** measures the largest absolute discrepancy among all response components:

$$e_\infty = \frac{|\mathbf{y} - \hat{\mathbf{y}}|_\infty}{|\mathbf{y}|_\infty}, \tag{2.11}$$

where the L^∞ norm of a vector $\mathbf{z}$ is defined as

$$|\mathbf{z}|_\infty = \max_{1 \le i \le n} |z_i| . \tag{2.12}$$

2.3.3 Training and Optimization

Training a neural network involves iterations through loss function calculation and optimising the predicted output to be close to the finite element reference output. This is done by minimising the loss function with respect to the trainable parameters.

Backpropagation is a process through which the gradients of the loss function are calculated. It applies the chain rule to calculate the derivative of the loss with respect to each weight and bias [18, 23]. A basic **gradient descent** update is:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L} \left(\boldsymbol{\theta}_k \right), \tag{2.13}$$

where α is the learning rate.

In this work, the **Adam optimiser** is used. Adam combines momentum-based gradient descent with adaptive learning rates by estimating the first and second moments of the gradients [24].

The learning rate, number of hidden layers, and number of neurons affects the accuracy and stability of the training process. Larger networks can approximate

more complex mappings, but they may also overfit the training data if the architecture is unnecessarily large [18].

2.3.4 Normalisation of Input and Output Data

Input and output data are typically normalised in a preprocessing step prior to neural network training. Different input variables fed to a network may have different physical units and numerical scales. Without normalisation, features with larger values are likely to dominate the training process, leading to bias and incorrect results.

In this thesis, input quantities are commonly normalised using min-max scaling to the interval $[-1, 1]$, using the formula:

$$\tilde{x} = 2\frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}} - 1. \tag{2.14}$$

where $\tilde{x}$ is the normalized input.

Output normalization is also important because finite element response vectors may contain components with very different magnitudes. For example, displacements near constrained boundaries may be much smaller than those farther from the support. To treat all output components more evenly, z-score normalization is used:

$$\tilde{y}_i = \frac{y_i - \mu_i}{\sigma_i}, \tag{2.15}$$

where μ_i and σ_i are the mean and standard deviation of the i-th output component in the training data. After prediction, the output is transformed back to the physical scale using:

$$y_i = \tilde{y}_i \sigma_i + \mu_i. \tag{2.16}$$

Because eigenvectors have arbitrary magnitude and sign, normalisation for modal analysis requires extra caution. As a result, before training, a sign convention is used and mode shapes scaled consistently. This prevents the network from receiving modal data that is numerically inconsistent but physically equivalent.

This concludes the theoretical background of the thesis.

Chapter 3

Methodology

3.1 Problem Definition

The aim of this thesis is to build surrogate models for structural analysis. The reference problem considered in this work is a two-dimensional rectangular or quadrilateral structure under plane stress conditions, following the formulation introduced in Chapter 1.

The finite element solver is used to generate the reference solutions. For all data-driven networks in this thesis, for each combination of geometric and loading parameters, the finite element problem is solved to obtain the complete displacement field over the domain and in some cases the corresponding stress field. The resulting displacement field is used to train the models. The main purpose of data-driven surrogate models in this work is to approximate the finite element mapping from input parameters to structural response.

In the static case, the governing finite element equation can be written as:

$$\mathbf{K}(\boldsymbol{\mu})\mathbf{U}(\boldsymbol{\mu}) = \mathbf{F}(\boldsymbol{\mu}), \tag{3.1}$$

where $\mathbf{K}$ is the global stiffness matrix, $\mathbf{U}$ is the global nodal displacement vector, $\mathbf{F}$ is the global force vector, and $\boldsymbol{\mu}$ represents the set of input parameters, such as geometry. Instead of repeatedly solving the costly finite element model for

every new parameter combinations, the surrogate model is trained to directly predict the structural response based on the input parameters.

3.2 Geometry and Boundary Conditions

The geometry of the structural domain is varied throughout the work as a parametrised problem is targeted for structural analysis. The domain remains quadrilateral in its general form, yet for most applications it is specifically a rectangular structure.

For a **rectangular structure**, the geometric parameters are defined with width B and height H. The structure occupies a two-dimensional domain $\Omega \subset \mathbb{R}^2$, where the x-direction corresponds to the width and the y-direction corresponds to the height. The geometric domain for a rectangular structure is:

$$\Omega = [0, B] \times [0, H]. \tag{3.2}$$

The geometric parameters B and H are varied randomly during training within a prescribed range. Later, the work expands to accommodate parametrised structures, and a more generic quadrilateral domain is chosen. Where **parametrised quadrilateral structures** are introduced, additional spatial and geometric parameters are introduced to create complex geometries, which will be discussed in their relevant sections.

Further, the work includes **circular holes or openings inside the structural domain**. Where circular openings are added, they are described by the coordinates for their centroid and their diameter, creating additional parameters for the holes case:

$$x_c, \quad y_c, \quad d, \tag{3.3}$$

where (x_c, y_c) denotes the centre of the opening and d denotes its diameter.

The **boundary conditions** remain constant throughout the study. The bottom

boundary of the rectangular or the boundary lying on the $y = 0$ axis for generic quadrilateral structures remains fixed. It imposes zero displacement in both x- and y-directions:

$$u_x = 0, \qquad u_y = 0 \quad \text{on } y = 0. \tag{3.4}$$

The remaining boundaries are assumed to be traction-free, either explicitly defined or implicitly implied.

3.3 Material Properties

The structure is assumed to be homogeneous, isotropic, and linearly elastic. The material behaviour for the linear static case is defined by Young's modulus E and Poisson's ratio ν. For the dynamic case, the density is also defined. Wherever needed, the thickness t is assumed to be 1 for this plane stress formulation unless stated otherwise. The required material properties are summarised as:

$$E = 1, \qquad \nu = 0.3, \qquad \rho = 1, \qquad t = 1. \tag{3.5}$$

The use of generic material properties is deliberate because the study is focused on the response of the surrogate models and the comparison between the various models, rather than on material behaviour.

3.4 Loading Conditions

For most cases, the static analysis considers body forces per unit volume applied in the two principal directions, x and y directions. Two separate loading cases are defined as:

$$\mathbf{b}_x = \begin{bmatrix} b_x \\ 0 \end{bmatrix}, \qquad \mathbf{b}_y = \begin{bmatrix} 0 \\ b_y \end{bmatrix}. \tag{3.6}$$

These are referred to as the body force in the x-direction and the body force in the y-direction, respectively.

The two loading directions are treated separately during data generation. This allows the surrogate model to learn the displacement response associated with each independent load component. Therefore, two separate neural networks are trained for the two unit load cases.

3.5 Mesh Generation

The finite element mesh is generated by discretising the structure into quadrilateral (Q4) elements. This mesh is defined by the number of nodes in the two principal directions, x and y. Each node has two degrees of freedom, corresponding to the displacement in the x- and y-directions:

$$\mathbf{u}_i = \begin{bmatrix} u_{x,i} \\ u_{y,i} \end{bmatrix}. \tag{3.7}$$

Therefore, for a mesh with n_{nodes} total nodes, the total number of displacement degrees of freedom is $n_{\text{dof}} = 2n_{\text{nodes}}$.

The number of nodes and hence, the mesh density is varied throughout this study. The reason for the variations is to study the effect of mesh density, improve the finite element data, or to boost the performance of the neural network, depending on its case application.

3.6 Dataset Generation Strategy

3.6.1 Parametrization of Geometry

For the rectangular cases, the input dataset is compiled by randomly generating the geometric parameters B and H, within a defined range:

$$B \in [B_{\text{min}}, B_{\text{max}}], \qquad H \in [H_{\text{min}}, H_{\text{max}}]. \tag{3.8}$$

For the number of samples, n_{samples}, the random geometric parameters are generated in MATLAB as shown in Listing 3.1.

Listing 3.1. Random generation of rectangular plane-stress structure dimensions

```
for i = 1:nSamples
    B = B_min + (B_max - B_min) * rand();
    H = H_min + (H_max - H_min) * rand();
end
```

These geometric configurations are solved using the MATLAB-based finite element solver for data-driven cases, and their respective structural response is stored as the output. In most cases, that structural response is the displacement field for the particular geometry under the given boundary and loading conditions.

Additional parameters are similarly generated at random for more complex geometries, like quadrilateral plane-stress structures or structures with circular openings. The extra coordinates are added to the input vector.

3.6.2 Parametrisation of Loading

For static analysis, body forces are applied separately in the x- and y-directions by creating separate networks for each case. In linear elastic cases, unit body forces are used for training. Since the problem is linear, during inference the displacement field can be obtained by scaling for the required force magnitude the response of the network and superposing both the responses.

This strategy reduces the number of finite element simulations required for different load cases, while still allowing the model to represent different loading magnitudes.

3.6.3 Storage of Output

For the static case, each finite element simulation produces the primary output as the nodal displacement vector, except for the POD (Proper Orthogonal Decomposition) case discussed later. The displacement vector contains both displacement components (x and y) at every node:

$$\mathbf{U}_{FEM} = \begin{bmatrix} u_{x,1} & u_{y,1} & u_{x,2} & u_{y,2} & \cdots & u_{x,n} & u_{y,n} \end{bmatrix}^T. \tag{3.9}$$

In some cases, the stress field is also computed and stored. The stress output contains the in-plane stress components:

$$\sigma_{xx}, \qquad \sigma_{yy}, \qquad \tau_{xy}. \tag{3.10}$$

For modal analysis, the output consists of the selected number of mode shapes. Each mode shape is stored as a global vector with the same displacement degree-of-freedom ordering as the static displacement field.

3.7 Computational Time

Throughout this thesis, the term *data generation time* refers to the total time required to generate finite element training samples for data-driven models. For physics-informed neural networks, *geometry and collocation points generation time* refers to the time required to generate the training geometries and their associated collocation points. *Training time* denotes the time required to train the neural network based, either based on the labelled data or physics-based governing equations. *Inference time* denotes the time required by a trained network to predict the response of a new geometry. Collectively, the generation and training times are treated as the initial computational time, while the inference time represents the computational time required for each subsequent prediction.

3.8 MATLAB Implementation

The work of this thesis has been executed in MATLAB. The finite element solver is developed in MATLAB and consequently, the data generated and stored. The solver is used to assemble the stiffness and force matrices (and the mass matrices where required), apply the boundary conditions, and solve the systems for the required displacement and stress fields or mode shapes.

For the neural network implementation, the training, inference and post-processing is done in MATLAB. The MATLAB Deep Learning Toolbox has been a key tool for this study. The toolbox allows the creation of custom training loops using objects such as `dlarray`. The automatic differentiation functions are useful for computing gradients of loss functions during training and for implementing physics-informed terms when required [25].

The MATLAB implementation allows for random data generation for parametrised finite element datasets, training data-driven neural networks, computing loss functions and error metrics, visualizing displacement, stress, and modal fields, and comparison of finite element results with the network responses.

3.9 Verification of the FEM Solver

The custom solver is checked to make sure it generates results that are both numerically consistent and physically correct before using the finite element results as training data. Verifying the symmetry and conditioning of the stiffness matrix, examining the deformed shapes, comparing displacement and stress patterns with the anticipated structural behaviour, and confirming the displacement boundary conditions are all part of the verification process.

In the static case, the displacement field is examined to make sure that the deformation pattern is in line with the applied body force and that the fixed boundary has zero displacement. The computed mode shapes for the modal analysis are examined to confirm that they represent in-plane vibration patterns consistent with the adopted plane stress formulation and the specified boundary conditions.

This verification step is critical as the neural network learns directly from the finite element data. Any error in the finite element solver could be transferred to the surrogate model, if not identified and corrected earlier.

The methodology explained in this chapter is used in the subsequent ones to build the surrogate models using the approaches defined.

Chapter 4

Data-driven Neural Networks

This chapter discusses the implementation of data-driven neural surrogate models for plane-stress rectagular or quadrilateral structural problems, their prediction, and the accuracy of their results by comparison with reference finite element solutions.

4.1 Problem definition

For the data-driven neural network study, this work considers plane-stress structures with different geometrical configurations, including rectangular, general quadrilateral, and some containing circular holes in their domain. The main objective is to investigate how changing different modelling factors affects the accuracy of the surrogate models, to evaluate the performance of the models, and to compare the different cases discussed.

4.2 Preliminary Case: Single Body Force on a Rectangular Structure

Firstly, a simplified case of a plane-stress rectangular structure being subjected to a single body force in the $x-$direction is studied. The purpose of this is to build a finite element solver and a neural network, and check if the network can learn

from the finite element data the mappings between the geometric parameters of the rectangular structure and the displacement response.

The finite element solution is computed for randomly generated rectangular geometries defined by the base B and height H. In the preliminary case, a fixed unit body force per unit volume is applied in the x-direction. The input to the surrogate model is therefore the vector of geometric parameters,

$$\mathbf{x} = \begin{bmatrix} B \\ H \end{bmatrix}, \tag{4.1}$$

while the output is the complete nodal displacement vector,

$$\mathbf{u} = \begin{bmatrix} u_{x,1} & u_{y,1} & u_{x,2} & u_{y,2} & \cdots & u_{x,n} & u_{y,n} \end{bmatrix}^T. \tag{4.2}$$

This case helps with a preliminary check of whether the complete displacement field can be predicted directly from only two geometric inputs.

4.3 Case 1: Two Independent Body Force Components on a Rectangular Structure

To create a surrogate model that is more flexible with respect to loading direction, the work is extended to include two independent and orthogonal unit body force cases:

$$\mathbf{b}_x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{b}_y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \tag{4.3}$$

The first case corresponds to a unit body force per unit volume in the x-direction, while the second corresponds to a unit body force per unit volume in the y-direction.

4.3.1 Dataset Generation

For each geometry sample, the parameters B and H are randomly generated within the set range. The parameters used for this case are summarised in Table 4.1.

Table 4.1. Dataset parameters for the two-body-force case (Case 1).

Parameter	Code notation	Value
Number of sampled geometries	`nSamples`	1000
Number of nodes in the x-direction	`nNodesX`	8
Number of nodes in the y-direction	`nNodesY`	8
Degrees of freedom per node	`ndofNode`	2
Geometry range for B	$[B_{\text{min}}, B_{\text{max}}]$	$[1, 3]$
Geometry range for H	$[H_{\text{min}}, H_{\text{max}}]$	$[1, 3]$

For each generated geometry, two finite element systems are assembled, one for the unit body force in the x-direction and one for the unit body force in the y-direction. The corresponding physical nodal coordinates are obtained by scaling a reference unit mesh, as shown in Listing 4.1.

Listing 4.1. Dataset generation for the two-body-force case (Case 1).

```
nNodesX = 8;
nNodesY = 8;
nNodes  = nNodesX * nNodesY;

[xGrid, yGrid] = meshgrid(linspace(0,1,nNodesX), ...
                          linspace(0,1,nNodesY));
coords0 = [xGrid(:), yGrid(:)];

ndofNode = 2;
nDOF = nNodes * ndofNode;

Xdata = zeros(2, nSamples);
Uall1 = zeros(nDOF, nSamples);
Uall2 = zeros(nDOF, nSamples);

bodyForcex = [1; 0];
bodyForcey = [0; 1];

for i = 1:nSamples

```

```
    B = B_min + (B_max - B_min) * rand();
    H = H_min + (H_max - H_min) * rand();
    Xdata(:,i) = [B; H];

    coords = [coords0(:,1)*B, coords0(:,2)*H];

    [K1, F1] = buildPlaneStressFEM(coords, nNodesX, nNodesY, E, nu
        , bodyForcex);
    [K2, F2] = buildPlaneStressFEM(coords, nNodesX, nNodesY, E, nu
        , bodyForcey);

    tol = 1e-12;
    baseNodes = find(abs(coords(:,2)) < tol);

    fixedDOFs = reshape([(baseNodes-1)'*ndofNode + 1; ...
                         (baseNodes-1)'*ndofNode + 2], [], 1);

    allDOFs  = (1:nDOF)';
    freeDOFs = setdiff(allDOFs, fixedDOFs);

    u1 = zeros(nDOF,1);
    u1(freeDOFs) = K1(freeDOFs, freeDOFs) \ F1(freeDOFs);

    u2 = zeros(nDOF,1);
    u2(freeDOFs) = K2(freeDOFs, freeDOFs) \ F2(freeDOFs);

    Uall1(:,i) = u1;
    Uall2(:,i) = u2;

end
```

The geometric parameters for all samples (N_s is the total number of sampled geometries) are stored in the input matrix

$$\mathbf{X}_{\text{data}} = \begin{bmatrix} B_1 & B_2 & \cdots & B_{N_s} \\ H_1 & H_2 & \cdots & H_{N_s} \end{bmatrix}. \tag{4.4}$$

The displacement solutions for the two unit body force cases are stored separately as

$$\mathbf{U}_x = \begin{bmatrix} \mathbf{u}_x^{(1)} & \mathbf{u}_x^{(2)} & \cdots & \mathbf{u}_x^{(N_s)} \end{bmatrix}, \qquad \mathbf{U}_y = \begin{bmatrix} \mathbf{u}_y^{(1)} & \mathbf{u}_y^{(2)} & \cdots & \mathbf{u}_y^{(N_s)} \end{bmatrix}. \tag{4.5}$$

Here, $\mathbf{U}_x$ corresponds to the displacement responses stored in `Uall1`, while $\mathbf{U}_y$ corresponds to the displacement responses stored in `Uall2`. The i-th column of

each displacement matrix corresponds to the geometry stored in the i-th column of $\mathbf{X}_{\text{data}}$.

Each displacement vector for a single sample i contains both horizontal and vertical displacement components at all nodes,

$$\mathbf{u}^{(i)} = \begin{bmatrix} u_{x,1}^{(i)} & u_{y,1}^{(i)} & u_{x,2}^{(i)} & u_{y,2}^{(i)} & \cdots & u_{x,n}^{(i)} & u_{y,n}^{(i)} \end{bmatrix}^T. \tag{4.6}$$

4.3.2 Finite Element Assembly Function

The function displayed in Listing 4.2 assembles the finite element outputs used for dataset generation. The mesh size, material properties, applied body force vector, and nodal coordinates of the current geometry are all passed to this function. The global force vector **F** and global stiffness matrix **K** are returned.

Listing 4.2. Finite element assembly function used for the data-driven dataset generation.

```
function [K,F] = buildPlaneStressFEM(coords,nx,ny,E,nu,bodyForce)

    nNodes = nx*ny;
    nElemX = nx - 1; nElemY = ny - 1;
    nElem  = nElemX * nElemY;

    ndofNode = 2;
    nDOF = nNodes * ndofNode;

    conn = zeros(nElem,4);

    for ix = 1:nElemX
        for jy = 1:nElemY
            n1 = (ix-1)*ny + jy;
            n2 = ix*ny + jy;
            n3 = ix*ny + (jy+1);
            n4 = (ix-1)*ny + (jy+1);

            e = (ix-1)*nElemY + jy;
            conn(e,:) = [n1 n2 n3 n4];
        end
    end

    K = zeros(nDOF);
    F = zeros(nDOF,1);

    D = E/(1-nu^2) * [1,  nu, 0;
```

```
                    nu, 1,  0;
                    0,  0,  (1-nu)/2];

gp  = [-1/sqrt(3), 1/sqrt(3)]; wgp = [1, 1];

for e = 1:nElem

    nodes = conn(e,:);
    xe = coords(nodes,1); ye = coords(nodes,2);

    Ke = zeros(8,8);
    Fe = zeros(8,1);

    xi = 0; eta = 0;
    [~, dNdxi0, dNdeta0] = shape4(xi,eta);

    J0 = [dNdxi0'; dNdeta0'] * [xe, ye];
    dN0 = [dNdxi0, dNdeta0] / J0;

    B0 = zeros(3,8);

    for a = 1:4
        dNx = dN0(a,1); dNy = dN0(a,2);
        col = (a-1)*2;

        B0(1,col+1) = dNx;
        B0(2,col+2) = dNy;
        B0(3,col+1) = dNy;
        B0(3,col+2) = dNx;
    end

    for i = 1:2
        for j = 1:2

            xi = gp(i); eta = gp(j);

            [N, dNdxi, dNdeta] = shape4(xi,eta);

            J = [dNdxi'; dNdeta'] * [xe, ye];
            detJ = det(J);
            dN = [dNdxi, dNdeta] / J;

            B = zeros(3,8);

            for a = 1:4
                dNx = dN(a,1); dNy = dN(a,2);
                col = (a-1)*2;

                B(1,col+1) = dNx;
                B(2,col+2) = dNy;
                B(3,col+1) = dNy;
                B(3,col+2) = dNx;
            end
```

```
81
82                  B(3,:) = B0(3,:);
83
84                  Ke = Ke + B' * D * B * detJ * wgp(i) * wgp(j);
85
86                  Nmat = zeros(2,8);
87
88                  for a = 1:4
89                      Nmat(1,(a-1)*2 + 1) = N(a);
90                      Nmat(2,(a-1)*2 + 2) = N(a);
91                  end
92
93                  Fe = Fe + Nmat' * bodyForce * detJ * wgp(i) * wgp(
                        j);
94
95              end
96          end
97
98          gdofs = zeros(8,1);
99
100         for a = 1:4
101             node = nodes(a);
102             gdofs((a-1)*2 + (1:2)) = (node-1)*2 + (1:2);
103         end
104
105         K(gdofs,gdofs) = K(gdofs,gdofs) + Ke;
106         F(gdofs) = F(gdofs) + Fe;
107
108     end
109 end
110
111 function [N, dNdxi, dNdeta] = shape4(xi,eta)
112
113     N = 0.25*[(1-xi)*(1-eta); (1+xi)*(1-eta);
114               (1+xi)*(1+eta); (1-xi)*(1+eta)];
115
116     dNdxi = 0.25*[-(1-eta); (1-eta);
117                   (1+eta);-(1+eta)];
118
119     dNdeta = 0.25*[-(1-xi); -(1+xi);
120                    (1+xi); (1-xi)];
121 end
```

This uses the concepts from Chapter 1 for the definition of the element connectivity matrix, calculation and assembly of local and global stiffness and force matrices, while applying the boundary condition where the bottom edge is fixed. For each sampled geometry, this function is called twice for the body force in each direction, as indicated in Listing 4.1.

4.3.3 Neural Network Definition

The first network, denoted by $\mathcal{N}_x$, predicts the displacement response due to the unit body force in the x-direction. The second network, denoted by $\mathcal{N}_y$, predicts the displacement response due to the unit body force in the y-direction. The mappings are indicated as:

$$\mathcal{N}_x : \begin{bmatrix} \bar{B} \\ \bar{H} \end{bmatrix} \mapsto \bar{\mathbf{u}}_{\text{NN}}^{(x)}, \qquad \mathcal{N}_y : \begin{bmatrix} \bar{B} \\ \bar{H} \end{bmatrix} \mapsto \bar{\mathbf{u}}_{\text{NN}}^{(y)}. \tag{4.7}$$

Here, the input is the normalised geometric parameters $\bar{B}$ and $\bar{H}$ and the output layer contains n_{dof} neurons, corresponding to the full displacement vector of the finite element model.

The MATLAB implementation of the two networks is shown in Listing 4.3. Since the inputs are already normalised before training, no normalisation is applied in the input layer.

Two hidden layers with 64 neurons each are included in each network. The rectified linear unit (ReLU) activation function is used in the hidden layers. For this data-driven case, ReLU offers better convergence and lesser prediction error as compared to the hyperbolic tangent and sigmoid activation functions, which were also tested. ReLU is a piecewise-linear activation function that helps reduce gradient saturation during training and is ideal for approximating such direct regression mapping and response.

Listing 4.3. Definition of the two neural networks for the two-body-force case (Case 1).

```
nOut1 = nDOF;
nOut2 = nDOF;

layers1 = [
    featureInputLayer(2,"normalisation","none")
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(nOut1)
];

```

```
dlnet1 = dlnetwork(layerGraph(layers1));

layers2 = [
    featureInputLayer(2,"normalisation","none")
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(nOut2)
];

dlnet2 = dlnetwork(layerGraph(layers2));
```

Here, `dlnet1` refers to the network trained with the displacement dataset `Uall1`, and `dlnet2` to the network trained with `Uall2`. Though the two networks are trained separately, they share the same normalised input data `Xnorm`.

4.3.4 Training Strategy

The main training parameters are summarised in Table 4.2.

Table 4.2. Training parameters for the two-body-force case (Case 1).

Parameter	Code notation	Value
Number of epochs	`numEpochs`	1000
Initial learning rate	`learnRate`	10^{-3}
Mini-batch size	`miniBatchSize`	200
Optimizer	`adamupdate`	Adam
Learning rate reduction epoch	$0.7\,$`numEpochs`	700
Learning rate reduction factor	–	0.2
Loss function	–	Mean squared error

The training is performed using mini-batches. At each iteration, the same input batch is used for both networks, while the target displacement batch is different for the two body force cases. The relevant part of the MATLAB implementation is shown in Listing 4.4.

Listing 4.4. Mini-batch training loop for the two neural networks.

```
numEpochs = 1000;
learnRate = 1e-3;
miniBatchSize = 200;

numIterationsPerEpoch = ceil(nSamples / miniBatchSize);

for epoch = 1:numEpochs

    idx = randperm(nSamples);

    for it = 1:numIterationsPerEpoch

        batchStart = (it-1)*miniBatchSize + 1;
        batchEnd = min(it*miniBatchSize, nSamples);
        batchIdx = idx(batchStart:batchEnd);

        Xbatch  = dlarray(Xn(:,batchIdx), "CB");
        Ybatch1 = dlarray(Un1(:,batchIdx), "CB");
        Ybatch2 = dlarray(Un2(:,batchIdx), "CB");

        [loss1, gradients1] = dlfeval(@modelLoss, dlnet1, Xbatch,
            Ybatch1);
        [loss2, gradients2] = dlfeval(@modelLoss, dlnet2, Xbatch,
            Ybatch2);

        iteration = (epoch-1)*numIterationsPerEpoch + it;

        [dlnet1, avgGrad1, avgSqGrad1] = adamupdate(dlnet1, ...
            gradients1, avgGrad1, avgSqGrad1, iteration, learnRate
                );

        [dlnet2, avgGrad2, avgSqGrad2] = adamupdate(dlnet2, ...
            gradients2, avgGrad2, avgSqGrad2, iteration, learnRate
                );
    end

    if epoch == round(0.7*numEpochs)
        learnRate = learnRate * 0.2;
    end
end
```

The loss function is the mean squared error (MSE) between the normalised finite element displacement vector and the normalised neural network prediction. The two networks are updated independently using the Adam optimizer, but they are trained using the same set of geometry samples.

4.3.5 Inference and Superposition

After training, the unit-load displacement responses for a specific test geometry are predicted using the two networks. The same input normalisation used for training is used to normalize the test geometry. The test geometry parameters and loading condition used are summarised in Table 4.3.

Table 4.3. Test parameters for two-body-force case (Case 1).

Parameter	Value
B	1.25
H	2.00
Body force in x-direction	$[2, 0]^T$
Body force in y-direction	$[0, 3]^T$

After prediction, the stored z-score normalisation parameters are used to convert the network outputs back to the physical displacement scale.

For a general constant body force,

$$\mathbf{b} = \begin{bmatrix} b_x \\ b_y \end{bmatrix}, \tag{4.8}$$

the final neural network displacement prediction is obtained by linear superposition of the two unit-load responses:

$$\mathbf{u}_{\text{NN}} = b_x \mathbf{u}_{\text{NN}}^{(x)} + b_y \mathbf{u}_{\text{NN}}^{(y)}. \tag{4.9}$$

For comparision and verification of the neural network prediction, a finite element solution is also computed for the same test geometry, boundary conditions, and body forces.

4.3.6 Results

Error Evaluation

The quantitative errors checked against the finite element solutions are summarised in Table 4.4, while Figure 4.1 shows the contour plots of the two displacement components used to compare the finite element and neural network solutions on the test structure and the absolute error fields.

Table 4.4. Error values for Case 1.

Error measure	Value
RMSE	8.373×10^{-2}
Relative L^2 error	4.597×10^{-3}
Relative infinity norm error	9.784×10^{-3}

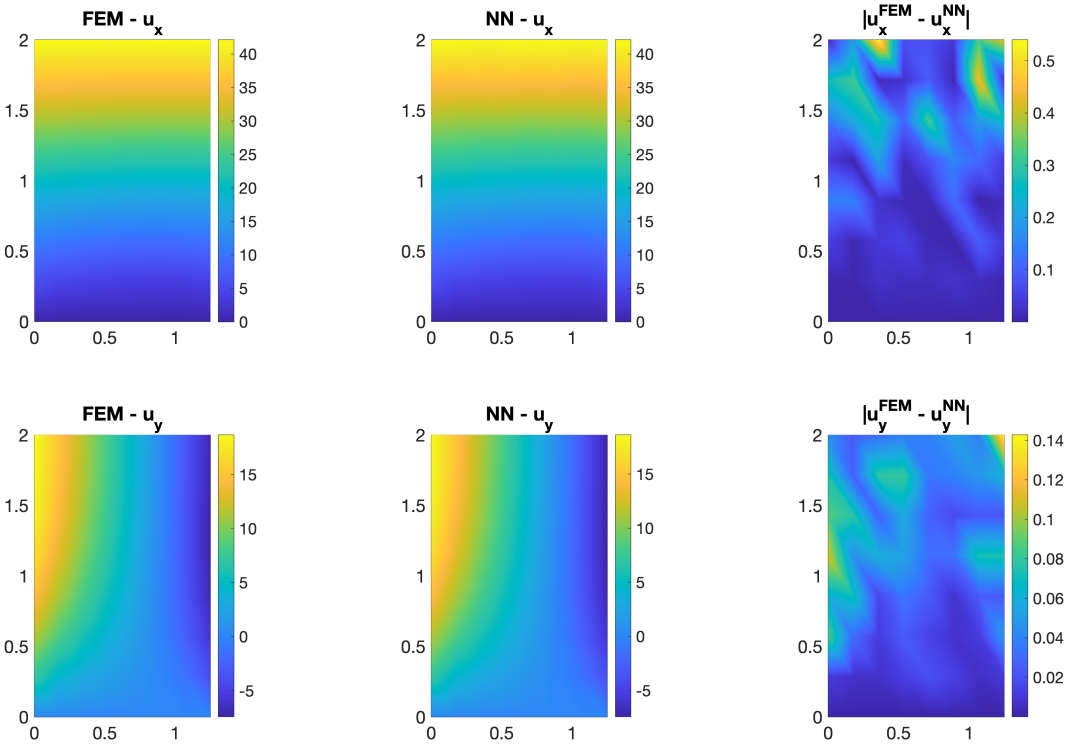


Figure 4.1. Comparison of FEM and neural network displacement fields with absolute error plots for Case 1.

The neural networks reproduce the response prediction magnitude and spatial distribution quite closely. As summarised, the combined displacement vector has an RMSE of 8.373×10^{-2}, a relative L^2 error of 4.597×10^{-3}, and a relative infinity norm error of 9.784×10^{-3}. The relative L^2 error indicates that the global difference between the predicted and referenced finite element displacement vectors is approximately 0.46% of the magnitude of the finite element solution. Similarly, the relative infinity norm error remains below 1%, indicating that the largest nodal discrepancy is small relative to the maximum displacement in the reference solution.

These results are obtained using two relatively simple neural networks, each with two hidden layers with 64 neurons per layer, trained using 1000 finite element samples. The results therefore demonstrate that, for the rectangular geometries and parameter range considered in Case 1, a comparatively compact network and small dataset is sufficient to approximate the displacement response accurately. However, the accuracy of the prediction is limited to the range of training inputs and may not necessarily be the same if the test geometry is beyond this range.

The error is not constant over the structural domain. The plots show that the higher absolute errors are concentrated near the free boundaries, approximately near the top edge for the horizontal displacement component and near the side edges for the vertical displacement component. This is because the free boundaries are less constrained and usually experience larger displacements and hence larger prediction errors. In contrast, the bottom edge is fixed, so the displacement is directly constrained there and is easier for the network to reproduce. Overall, the prediction plots show no major discontinuities or incorrect patterns, and the network can be said to capture both the overall response and the main displacement patterns of the finite element solution.

Computational Time

The computational time for the two-body-force data-driven case is summarised in Table 4.5. Training took the highest time here, but it is a once-off time cost. Once trained, the surrogate predicts the displacement field without assembling and solving

a new finite element system in a much shorter time. The practical benefit of surrogate models is more evident when many geometries have to be evaluated repeatedly, as in most industrial cases.

Table 4.5. Computational time for Case 1.

Step	Time [s]
FEM data generation	11.876
Training of neural networks	73.547
Inference	0.755

4.4 Case 2: Effect of Mesh Size

In this case, the effect of mesh size on the performance of the model is studied. The workflow for Case 1 is repeated, but for five different mesh sizes on the same 2000 geometric samples. The mesh sizes considered in this study are summarised in Table 4.6.

Table 4.6. Parameters used for the mesh-effect study (Case 2).

Parameter	Value
Mesh sizes checked	$[4 \times 4, 6 \times 6, 8 \times 6, 12 \times 12, 16 \times 16]$
Number of sampled geometries	2000
Test base	1.25
Test height	2.0
Test body force	$[2, 3]^T$

Error Evaluation

The errors obtained for each mesh are reported in Table 4.7 and plotted as a function of the number of nodes per direction in Figure 4.2.

Table 4.7. Error comparison for different mesh sizes (Case 2).

Mesh	n_{dof}	RMSE	Relative L^2 error	Relative L^∞ error
4×4	32	5.735×10^{-2}	3.091×10^{-3}	4.916×10^{-3}
6×6	72	7.554×10^{-2}	4.118×10^{-3}	7.890×10^{-3}
8×8	128	4.199×10^{-2}	2.305×10^{-3}	3.391×10^{-3}
12×12	288	4.169×10^{-2}	2.308×10^{-3}	5.904×10^{-3}
16×16	512	4.942×10^{-2}	2.749×10^{-3}	6.549×10^{-3}

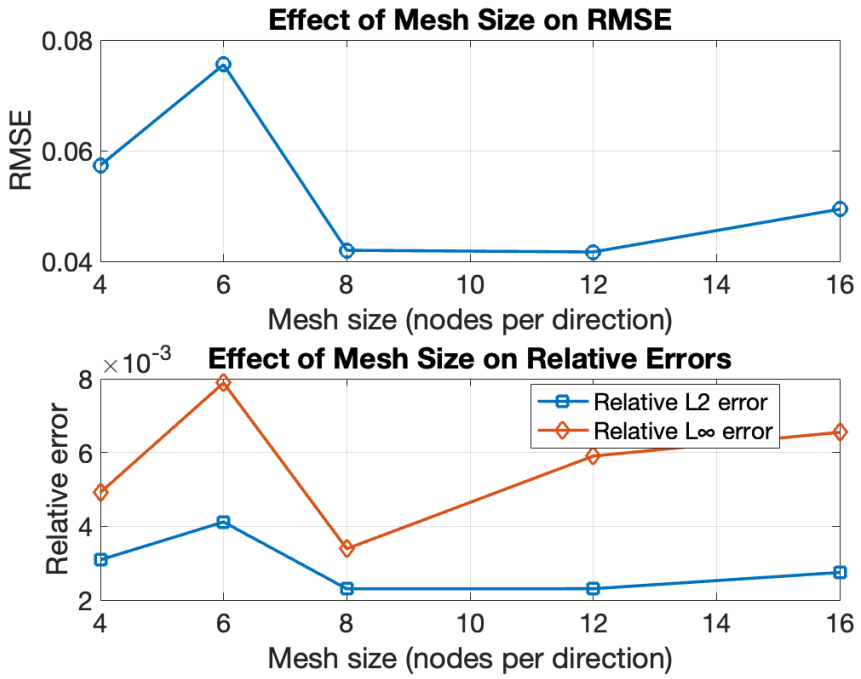


Figure 4.2. Effect of mesh size on the prediction error (Case 2).

The results show that the error generally decreases when the mesh is refined from 4×4 to 8×8. However, the reduction is consistent for all decreasing mesh sizes. This is expected since the neural network is retrained separately for each mesh, and the output dimension changes. For increasing mesh sizes, the number of degrees of freedom increases hence the output dimension increases, make it more prone to error. Therefore, the final error is affected not only by mesh improvement,

but also the network architecture and the increased size of the displacement vector to be learned.

The 8×8 and 12×12 meshes give the lowest errors overall, seemingly due to a good balance of the mesh size and the prediction output dimension. The 12×12 mesh gives the smallest RMSE, while the 8×8 mesh gives the smallest relative errors. The 4×4 and 6×6 meshes show larger errors, which are likely due to the heavily coarse mesh.

4.5 Case 3: Effect of Dimension Change

This case studies the effect of increasing the geometric dimension range for the input geometries. The workflow stays the same, but the maximum values of the base and height of the geometries are increased while the minimum value remains the same and the mesh is fixed at 8×8 nodes. The study parameters are shown in Table 4.8.

Table 4.8. Parameters used for the dimension-range study (Case 3).

Parameter	Value
Number of nodes	8×8
Minimum base and height	1
Maximum base/height list	$[3, 6, 12, 24]$
Number of sampled geometries	1000
Test base	$0.7B_{\text{max}}$
Test height	$0.6H_{\text{max}}$
Test body force	$[1, 1]^T$

Error Evaluation

The errors are reported in Table 4.9, and then plotted against the maximum dimension value of the base and height in Figure 4.3.

Table 4.9. Error comparison for increasing geometry ranges.

Maximum B, H	Geometry range	RMSE	Relative L^2 error	Relative L^∞ error
3	$[1, 3]$	3.374×10^{-2}	8.562×10^{-3}	1.817×10^{-2}
6	$[1, 6]$	2.007×10^{-1}	1.273×10^{-2}	3.128×10^{-2}
12	$[1, 12]$	2.337×10^{0}	3.707×10^{-2}	7.185×10^{-2}
24	$[1, 24]$	9.578×10^{1}	3.798×10^{-1}	6.818×10^{-1}

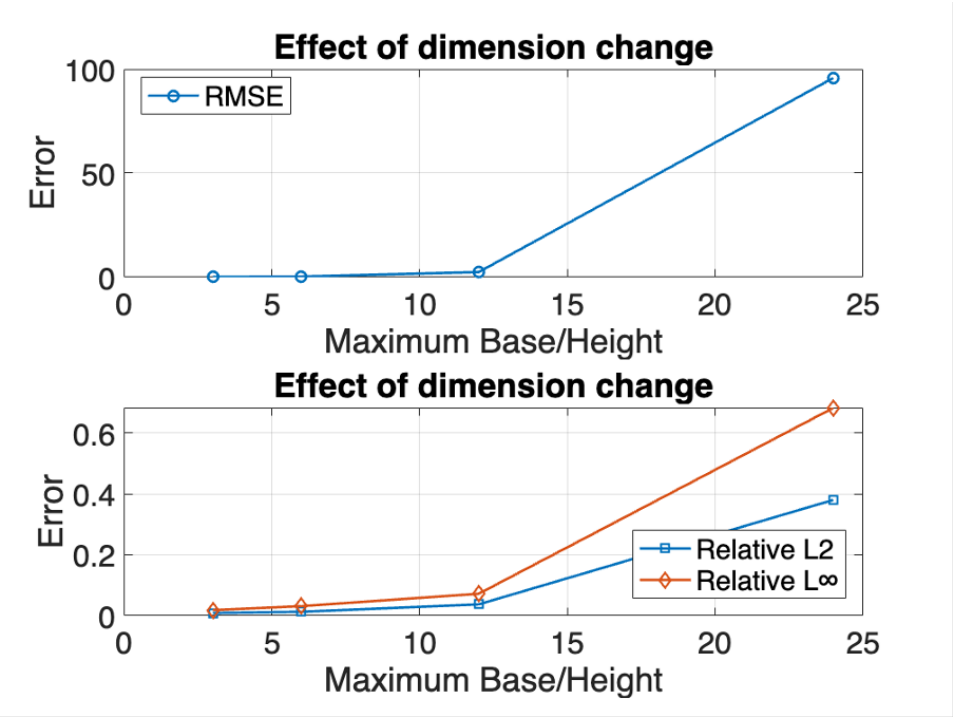


Figure 4.3. Effect of geometry range on the prediction error (Case 3).

The absolute RMSE error increases slightly as the maximum dimension increases from 3 to 12, but more steeply from 12 to 24. This higher increase is because the same number of training samples is now distributed over a larger geometric parameter range between $[1, 24]$. The training data is now less dense with respect to other ranges, leading to worse response prediction.

The general increase in error is because larger structures generally produce larger displacement magnitudes under the same body force. Therefore, the absolute

displacement error also increases as the geometry size increases.

The relative errors also increase slightly, especially for the $[1, 24]$ range. This shows that the prediction error grows faster than the reference displacement norm. The increase in error is due to the larger displacement magnitudes and also due to the reduced learning accuracy over the wider parameter range.

4.6 Case 4: General Quadrilateral Geometry

The goal of this case is expand the scope of the work from rectangular plane-stress structures to more generic quadrilateral shapes, thereby increasing the range of shapes that the model can represent. It also investigates whether the neural network can learn the more complex relationship between the geometric parameters and the resulting displacement response. The two-body-force strategy and much of the workflow is the same as in Case 1. However, the input vector is expanded to accommodate the complex shape of a quadrilateral geometry.

4.6.1 Geometry Parametrisation

The geometric parametrisation case uses five geometric parameters.

$$\mathbf{x} = \begin{bmatrix} B_1 & B_2 & H_1 & H_2 & d \end{bmatrix}^T. \tag{4.10}$$

These parameters define the four vertices of the quadrilateral as:

$$\mathbf{v}_1 = (0, 0), \qquad \mathbf{v}_2 = (B_1, 0), \qquad \mathbf{v}_3 = (d + B_2, H_1), \qquad \mathbf{v}_4 = (d, H_2). \tag{4.11}$$

For this case the bottom edge is fixed at $y = 0$, while allowing the upper edge to translate and rotate. The physical quadrilateral shape is warped onto the reference mesh using the bilinear shape functions discussed in Chapter 1.

The main parameters used in this case are summarised in Table 4.10.

Table 4.10. Parameters used for the quadrilateral geometry case (Case 4).

Parameter	Code notation	Value
Number of nodes in the x-direction	`nNodesX`	8
Number of nodes in the y-direction	`nNodesY`	8
Number of sampled geometries	`nSamples`	2000
Range of B_1	$[B_{1,\text{min}}, B_{1,\text{max}}]$	$[1, 3]$
Range of B_2	$[B_{2,\text{min}}, B_{2,\text{max}}]$	$[1, 3]$
Range of H_1	$[H_{1,\text{min}}, H_{1,\text{max}}]$	$[1, 3]$
Range of H_2	$[H_{2,\text{min}}, H_{2,\text{max}}]$	$[1, 3]$
Range of d	$[d_{\text{min}}, d_{\text{max}}]$	$[-3, 3]$

4.6.2 Convexity Check

For each sample, five geometric parameters of the vertices of the quadrilateral are randomly generated in their given ranges. Random combinations can also create distorted, inverted or non-convex shapes, so it's important to ensure that the convexity of the randomly generated geometry.

The cross product of consecutive edge vectors is checked, as a valid quadrilateral should have the same turning direction between consecutive edges around the whole boundary. That means, all four cross products must have the same sign. This also avoids cases where the quadrilateral becomes locally inverted or where three consecutive vertices become collinear, creating a triangle. The implementation is shown in Listing 4.5.

Listing 4.5. Generation of valid quadrilateral geometries (Case 4).

```
B1 = B1_min + (B1_max - B1_min) * rand();
B2 = B2_min + (B2_max - B2_min) * rand();
H1 = H1_min + (H1_max - H1_min) * rand();
H2 = H2_min + (H2_max - H2_min) * rand();
d  = d_min  + (d_max  - d_min)  * rand();

ver1 = [0, 0];
ver2 = [B1, 0];
ver3 = [d+B2, H1];
```

```
ver4 = [d, H2];

while ~checkConvexity(ver1, ver2, ver3, ver4)

    B2 = B2_min + (B2_max - B2_min) * rand();
    H1 = H1_min + (H1_max - H1_min) * rand();
    H2 = H2_min + (H2_max - H2_min) * rand();
    d  = d_min  + (d_max  - d_min)  * rand();

    ver3 = [d+B2, H1];
    ver4 = [d, H2];

end

Xdata(:,i) = [B1; B2; H1; H2; d];

function flag = checkConvexity(v1, v2, v3, v4)

    v = [v1; v2; v3; v4; v1; v2];
    crossVals = zeros(4,1);

    for i = 1:4
        a = v(i+1,:) - v(i,:);
        b = v(i+2,:) - v(i+1,:);
        crossVals(i) = a(1)*b(2) - a(2)*b(1);
    end

    flag = all(crossVals > 0) || all(crossVals < 0);

end
```

4.6.3 Network Definition

The same normalisation is applied as earlier and the resultant normalised input vector is:

$$\bar{\mathbf{x}} = \begin{bmatrix} \bar{B}_1 & \bar{B}_2 & \bar{H}_1 & \bar{H}_2 & \bar{d} \end{bmatrix}^T . \tag{4.12}$$

The output normalisation uses the z-score normalisation again.

Since the geometry description is more complex, a deeper network is used. It uses five hidden layers with 64 neurons each and ReLU activation functions. The network architecture is shown in Listing 4.6.

Listing 4.6. Neural network definition for the quadrilateral case (Case 4).

```
nOut1 = nDOF;
nOut2 = nDOF;

layers1 = [
    featureInputLayer(5,"normalisation","none")
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(nOut1)
];

dlnet1 = dlnetwork(layerGraph(layers1));

layers2 = [
    featureInputLayer(5,"normalisation","none")
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(nOut2)
];

dlnet2 = dlnetwork(layerGraph(layers2));
```

For training, the same parameters that are used for Case 1 are used here.

4.6.4 Results

For inference, the same steps as the last cases are repeated. The test geometry is checked for convexity and then normalised. The geometric parameters and loading condition used for this test case are summarised in Table 4.11.

Table 4.11. Test parameters for the quadrilateral geometry case (Case 4).

Parameter	Value
B_1	2.3533
B_2	2.1457
H_1	2.1889
H_2	1.6395
d	-0.7824
Body force in x-direction	$[2, 0]^T$
Body force in y-direction	$[0, 3]^T$

Error Evaluation

The displacement responses for the two unit body force cases are predicted by the trained networks. The numerical errors are reported in Table 4.12 and Figure 4.4 shows the visual comparison of the displacement fields and the absolute error fields.

Table 4.12. Prediction errors for the quadrilateral geometry case.

Error measure	Value
RMSE	3.995×10^{-1}
Relative L^2 error	7.773×10^{-2}
Relative L^∞ error	9.588×10^{-2}

This case has higher errors than the rectangular structure case, in Case 1. The relative L^2 error corresponds to an overall displacement error of approximately 7.8% relative to the magnitude of the finite element response. The relative L^∞ error indicates that the largest nodal discrepancy is approximately 9.6% of the maximum finite element displacement.

The neural networks reproduce the displacement field and its spatial distribution in this case. Compared with the rectangular case, the errors are higher, which could likely be due to the increased complexity of the input-output relationship. In

this case, five geometric parameters define the shape coordinates, allowing variations in width, height, and angle. Consequently, the network must represent a wider range of geometric transformations than in the rectangular cases. These changes also affect the finite element mesh geometry and the distribution of the displacement field. Although the number of training samples is increased to 2000, the dataset is distributed over a larger five-dimensional parameter space, resulting in a lower sampling density than for the simpler rectangular case.

The results indicate that data-driven model can approximate the structural response of general quadrilateral geometries within the prescribed parameter range, although with lower accuracy than for the simpler rectangular cases. Further improvements could include a larger or better distributed training dataset, additional network tuning, or a reduced-order output representation, such as Proper Orthogonal Decomposition, which is tested in the next case.

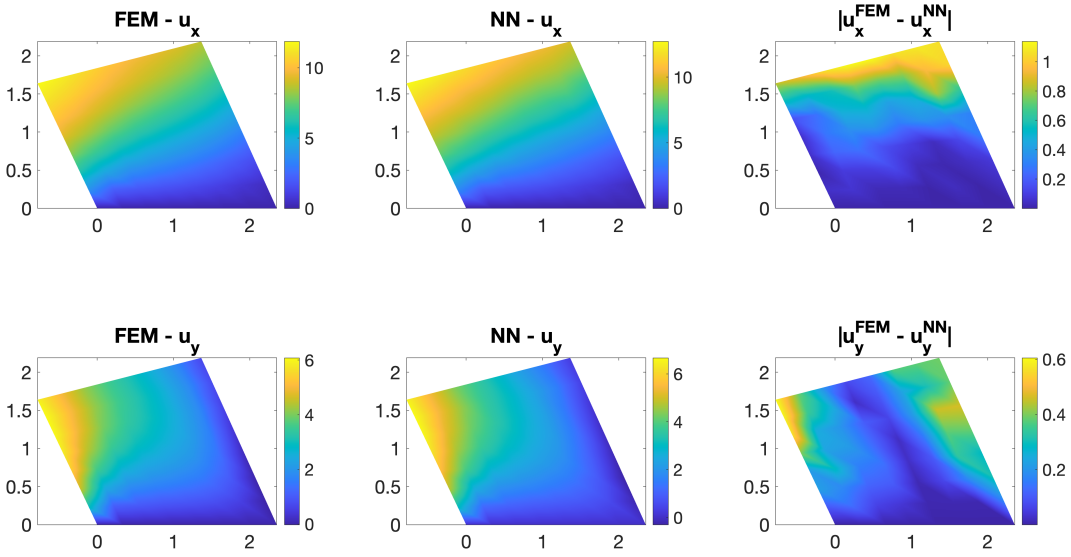


Figure 4.4. Comparison of FEM and neural network displacement fields with absolute error plots for Case 4.

Computational Time

The computational times for the parametrised quadrilateral case are summarised in Table 4.13. The increase in data-generation and training times relative to the rectan-

gular case is mainly associated with the larger training dataset, the increased number of geometric input parameters, and the deeper neural network architecture. These are initial computational costs incurred during model training. Once the model is trained, the inference time remains comparatively small and does not require the assembly and solution of a new finite element system.

Table 4.13. Computational time for the parameterised quadrilateral geometry case.

Step	Time [s]
FEM data generation	30.264
Training of neural networks	155.259
Inference	0.673

4.7 Case 5: POD-Reduced Surrogate for Quadrilateral Geometry

In this case, the generic quadrilateral geometry formulation utilises Proper Orthogonal Decomposition (POD) to if a more efficient analysis can be done by reducing the number of outputs. POD is a dimensionality reduction technique, where instead of predicting the full displacement field, the network now predicts a few displacement patterns or modes that represent the dominant energy of the displacement snapshot dataset.

The geometric parametrisation, convexity check, finite element assembly, and two-body force strategy remain the same as in Case 4. The main parameters used in this case are summarised in Table 4.14.

4.7.1 POD Reduction of Displacement Snapshots

For each body force direction, the finite element displacement snapshots (displacement vectors for all samples) are collected in separate matrices, $\mathbf{U}^{(x)}$ and $\mathbf{U}^{(y)}$. The

Table 4.14. Main parameters used for the POD-reduced data-driven case (Case 5).

Parameter	Code notation	Value
Number of nodes in the x-direction	`nNodesX`	8
Number of nodes in the y-direction	`nNodesY`	8
Number of sampled geometries	`nSamples`	10000
POD energy threshold	`energyThreshold`	0.999
Maximum retained POD modes	`rMax1`, `rMax2`	200
Number of epochs	`numEpochs`	1000
Initial learning rate	`learnRate`	10^{-3}
Learning rate reduction factor	–	0.2

mean displacement field is first removed from each dataset to create a centred displacement snapshot:

$$\mathbf{U}_c^{(x)} = \mathbf{U}^{(x)} - \boldsymbol{\mu}_u^{(x)}, \qquad \mathbf{U}_c^{(y)} = \mathbf{U}^{(y)} - \boldsymbol{\mu}_u^{(y)}. \tag{4.13}$$

Singular value decomposition is then applied to the centred displacement snapshot matrix:

$$\mathbf{U}_c = \boldsymbol{\Phi}\mathbf{S}\mathbf{V}^T, \tag{4.14}$$

where $\boldsymbol{\Phi}$ contains the left singular vectors, which form the POD modes or basis vectors, $\mathbf{S}$ is a diagonal matrix containing the singular values, and $\mathbf{V}$ contains the right singular vectors, which describe the contribution of each POD mode to the individual displacement snapshots.

Depending on the cumulative energy of the singular values intended to be retained, the number of POD modes are selected. In this implementation, the number of retained modes is chosen so that at least 99.9% of the snapshot energy is preserved, where the maximum allowable modes are 200.

The relevant MATLAB implementation is shown in Listing 4.7.

Listing 4.7. POD reduction of displacement snapshots (Case 5).

```
mu_U1 = mean(Uall1,2);
Uc1 = Uall1 - mu_U1;

mu_U2 = mean(Uall2,2);
Uc2 = Uall2 - mu_U2;

[Phi1, S1, ~] = svd(Uc1, 'econ');
singvals1 = diag(S1);
energy1 = cumsum(singvals1.^2) / sum(singvals1.^2);

[Phi2, S2, ~] = svd(Uc2, 'econ');
singvals2 = diag(S2);
energy2 = cumsum(singvals2.^2) / sum(singvals2.^2);

energyThreshold = 0.999;

r1 = find(energy1 >= energyThreshold, 1, 'first');
r2 = find(energy2 >= energyThreshold, 1, 'first');

r1 = min(r1, min(200, size(Phi1,2)));
r2 = min(r2, min(200, size(Phi2,2)));

Phi_r1 = Phi1(:,1:r1);
Phi_r2 = Phi2(:,1:r2);

A1 = Phi_r1' * Uc1;
A2 = Phi_r2' * Uc2;
```

Here, $\mathbf{\Phi}_r^{(x)}$ and $\mathbf{\Phi}_r^{(y)}$ are the reduced POD bases for the two unit body force cases. The matrices $\mathbf{A}^{(x)}$ and $\mathbf{A}^{(y)}$ contain the POD coefficients corresponding to all training samples.

4.7.2 POD Truncation Error and Mode Shapes

This work calculates the number of POD modes needed to represent a certain threshold of the cumulative energy of the displacement field, which is set to 99.9%. For the horizontal body-force case, $r = 8$ POD modes are retained, while for the vertical body-force case, $r = 10$ POD modes are retained. These retained modes capture more than 99.9% of the total snapshot energy in both cases.

The relative POD truncation error is computed from the discarded energy as

$$e_r = \sqrt{1 - E_r}, \tag{4.15}$$

where E_r is the cumulative energy retained by the first r POD modes. As the number of retained modes increases, more cumulative energy can be captured, and the truncation error decreases. Figure 4.5 shows the variation of the relative POD truncation error with the number of retained modes.

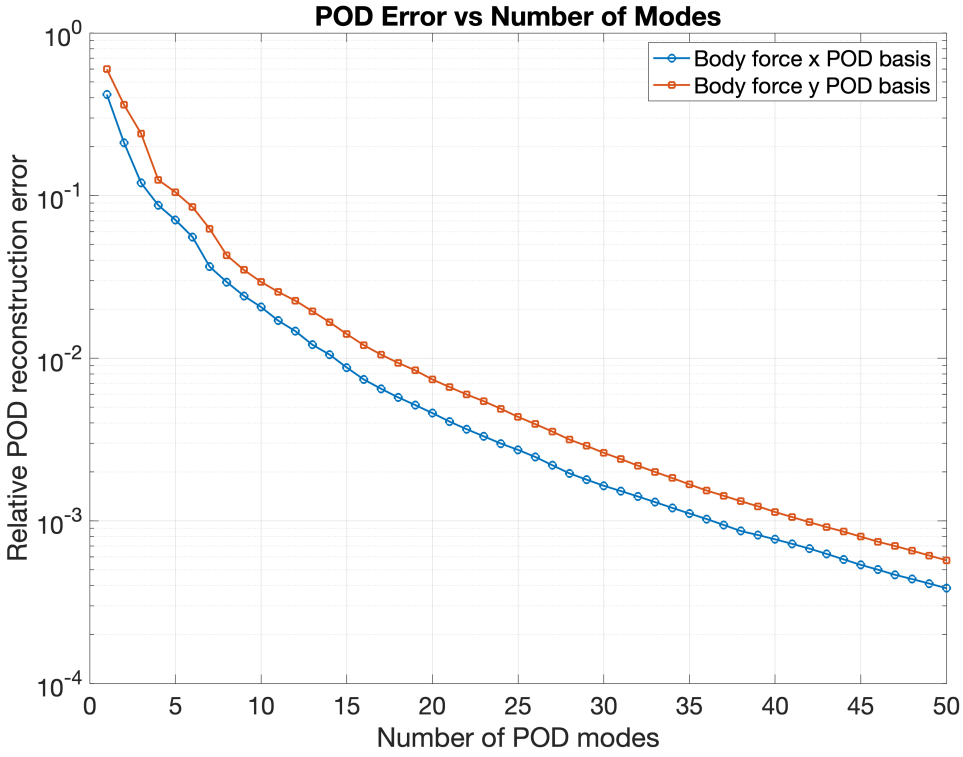


Figure 4.5. Relative POD truncation error versus number of retained POD modes.

The first few POD modes for the horizontal and vertical body-force cases are also shown in Figures 4.6 and 4.7. Since separate displacement snapshot matrices are used for the two loading directions, separate POD bases are obtained. The POD modes are plotted as deformed mesh patterns on a representative rectangular geometry with $B = 2$ and $H = 2$. However, the POD modes themselves are extracted from displacement snapshots over many randomly generated quadrilateral geometries. Therefore, individual POD modes should not be interpreted as physical deformation modes of the representative geometry. Instead, they represent dominant spatial variations in the displacement dataset.

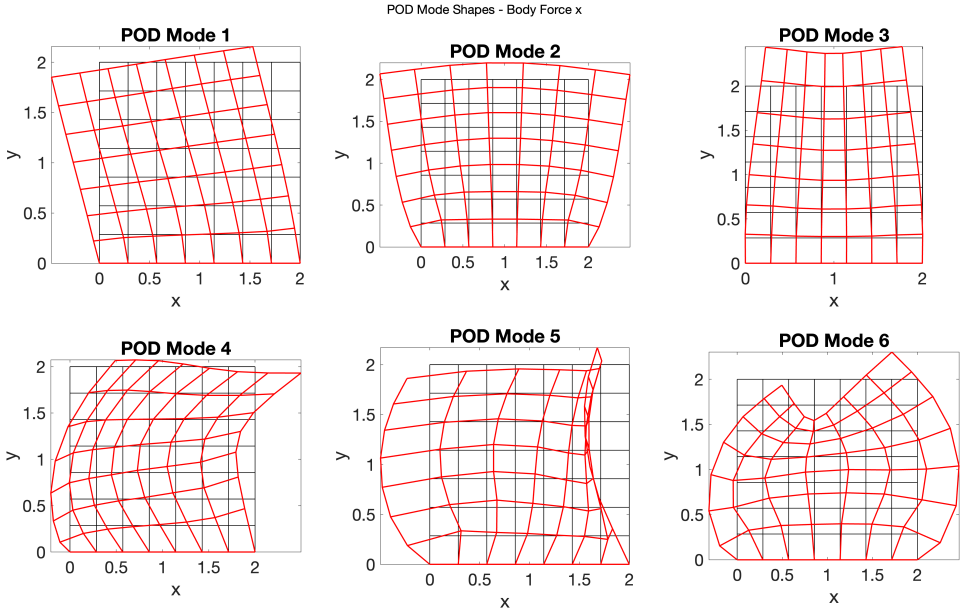


Figure 4.6. First POD modes for the horizontal body-force displacement snapshot matrix.

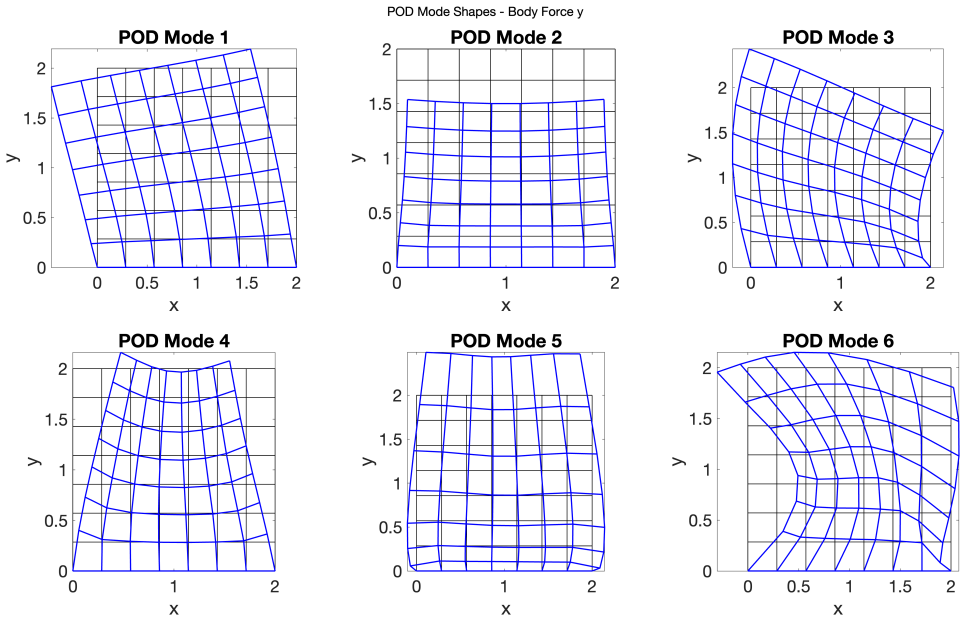


Figure 4.7. First POD modes for the vertical body-force displacement snapshot matrix.

4.7.3 Coefficient Normalisation

In this case, the networks predict the normalised POD coefficients instead of the full displacement vector, reducing the size of the learning problem. Therefore, the output dimension of each network becomes the number of retained POD modes:

$$\mathcal{N}_x : \bar{\mathbf{x}} \mapsto \bar{\mathbf{a}}^{(x)}, \qquad \mathcal{N}_y : \bar{\mathbf{x}} \mapsto \bar{\mathbf{a}}^{(y)}, \tag{4.16}$$

where $\mathcal{N}_x$ and $\mathcal{N}_y$ denote the neural networks associated with the body-force cases in the x- and y-directions, respectively. The vector $\bar{\mathbf{x}}$ contains the normalised geometric input parameters, while $\bar{\mathbf{a}}^{(x)}$ and $\bar{\mathbf{a}}^{(y)}$ contain the corresponding normalised POD coefficients for the retained modes. The POD coefficients are normalised using z-score normalisation.

4.7.4 Network Definition

For comparison to Case 4, the network architecture is kept the same. The only difference is that the number of output neurons is equal to the number of retained modes. Therefore, the networks predict the reduced POD coefficient vectors instead of the full displacement vectors.

4.7.5 Reconstruction of the Full Displacement Field

During prediction, the networks first output the normalised POD coefficients. These coefficients are denormalised and then used to reconstruct the full displacement vector:

$$\mathbf{u}_{\mathrm{NN}}^{(x)} = \boldsymbol{\mu}_u^{(x)} + \boldsymbol{\Phi}_r^{(x)} \mathbf{a}_{\mathrm{NN}}^{(x)}, \tag{4.17}$$

$$\mathbf{u}_{\mathrm{NN}}^{(y)} = \boldsymbol{\mu}_u^{(y)} + \boldsymbol{\Phi}_r^{(y)} \mathbf{a}_{\mathrm{NN}}^{(y)}. \tag{4.18}$$

The final displacement for a general body force is again obtained by scaling and superposition of the two reconstructed responses.

4.7.6 Results

The test parameters used in this analysis are the same as the ones used in Case 4.

Error Evaluation

The numerical errors for the POD-reduced surrogate are reported in Table 4.15.

Table 4.15. Prediction errors for the POD-reduced quadrilateral surrogate.

Error measure	Value
RMSE	4.989×10^{-1}
Relative L^2 error	9.708×10^{-2}
Relative L^∞ error	1.190×10^{-1}

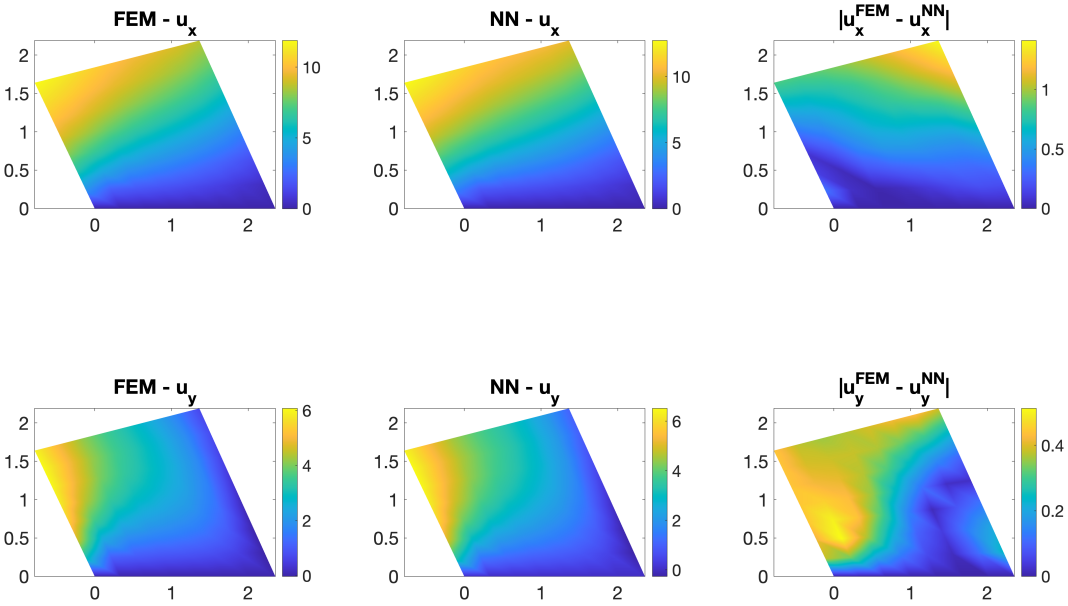


Figure 4.8. Comparison of FEM and neural network displacement fields with absolute error plots for Case 5.

The results are moderately lower yet still comparable to the direct displacement prediction used in Case 4, although the networks in this case predict a reduced number of POD coefficients instead of the complete displacement field. For compar-

ison, the network architecture is kept the same as in Case 4, except that the output layer is changed according to the number of retained POD modes.

The relative L^2 error of 9.708×10^{-2} indicates that the overall difference between the reconstructed POD prediction and the finite element displacement vector is approximately 9.7% of the reference response magnitude. The relative L^∞ error of 1.190×10^{-1} shows that the largest nodal discrepancy is approximately 11.9% of the maximum finite element displacement.

Compared with Case 4, a larger number of FEM samples is used in this case. This increases the data generation time, but the POD reduction decreases the output dimension of the learning problem. Therefore, the neural networks learn the mapping from the geometric parameters to the reduced POD coefficient space rather than directly learning all displacement degrees of freedom.

The final prediction error is a little higher because in POD-based models the error comes from two sources, POD truncation error and neural network prediction error. If more POD modes are involved, the truncation error is reduced. However, the neural network then has to predict a larger number of coefficients. Therefore, the number of retained modes must balance accuracy and learning complexity, which it does in this case.

Computational Time

The computational time for the POD-reduced quadrilateral case is summarised in Table 4.16.

Table 4.16. Computational time for the POD-reduced quadrilateral surrogate (Case 5).

Step	Time [s]
FEM data generation	115.571
Training of neural networks	91.343
Inference	0.811

The data generation time for a higher number of samples is more than in Case 4, but the training time is lesser. This indicates that reducing the output dimension has a significant effect on the computational requirements of training. Although the data-generation time remains an important initial cost because the POD basis must be constructed from a sufficiently representative collection of finite element snapshots.

4.8 Case 6: Structural Domains with Circular Holes

A lot of topological optimisation cases introduce voids to improve structure efficiency, while minimising weight. This case intends to check how the network solves for rectangular structures with circular holes. A preliminary version is first tested with one circular hole. The formulation is then extended to two circular holes, which is the main case discussed in this section. This case also evaluates the ability of the network to handle a larger geometric parameter space, since the hole sizes and positions are included as additional inputs. The rest of the workflow stays the same as in the previous cases.

4.8.1 Geometry Parametrisation

The two-hole rectangular structure is described using eight input parameters:

$$\mathbf{x} = \begin{bmatrix} B & H & d_1 & c_{x1} & c_{y1} & d_2 & c_{x2} & c_{y2} \end{bmatrix}^T, \tag{4.19}$$

where B and H are the structure dimensions, d_1 and d_2 are the hole diameters, and (c_{x1}, c_{y1}) and (c_{x2}, c_{y2}) are the coordinates of the two hole centers.

The main parameters used in this case are summarised in Table 4.17.

4.8.2 Hole Representation

The holes are represented using a low-stiffness material approach. Instead of removing elements from the mesh completely, elements associated with the holes are

Table 4.17. Parameters used for the two-hole structural domain case (Case 6).

Parameter	Code notation	Value
Number of nodes in the x-direction	`nNodesX`	16
Number of nodes in the y-direction	`nNodesY`	16
Number of sampled geometries	`nSamples`	1000
Number of input parameters	`inputs`	8
Geometry range for B	$[B_{\text{min}}, B_{\text{max}}]$	$[1, 3]$
Geometry range for H	$[H_{\text{min}}, H_{\text{max}}]$	$[1, 3]$
Hole diameter range	$[d_{\text{min}}, d_{\text{max}}]$	$[0.1, 0.4]$
Hole centre coordinate range	$[c_{\text{min}}, c_{\text{max}}]$	$[0.3, 0.7]$
Hole Young's modulus	E_{hole}	$10^{-6}E$

assigned a very small Young's modulus of $E_{\text{hole}} = 10^{-6}E$.

In addition, the body force is set to zero inside the hole regions. This allows the same structured rectangular mesh to be used for all the samples, while approximately representing void regions.

The function `elementInHole` in Listing 4.8 checks whether an element is located inside either circular hole. If any node of the element lies inside the corresponding circular region, it is treated as a part of the hole. Therefore, a denser mesh is better in this case.

Listing 4.8. Element check for circular hole regions (Case 6).

```
function isInside = elementInHole(elemCoords, cx, cy, radius)

    distances = sqrt((elemCoords(:,1)-cx).^2 + ...
                     (elemCoords(:,2)-cy).^2);

    isInside = any(distances < radius);

end
```

4.8.3 Geometry Validity Check

As hole parameters are generated randomly, there may be invalid geometries formed. Checks to ensure that the holes lie completely inside the structural domain and the two holes do not overlap are applied, as shown in Listing 4.9.

Listing 4.9. Boundary and overlap checks for two circular holes (Case 6).

```
margin = 0.2;

while true

    dia1 = dia_min + (dia_max - dia_min) * rand();
    cx1 = (cx_min + (cx_max - cx_min) * rand()) * B;
    cy1 = (cy_min + (cy_max - cy_min) * rand()) * H;

    dia2 = dia_min + (dia_max - dia_min) * rand();
    cx2 = (cx_min + (cx_max - cx_min) * rand()) * B;
    cy2 = (cy_min + (cy_max - cy_min) * rand()) * H;

    boundaryViolation = ...
        (cx1 - dia1/2 <= margin) || (cx1 + dia1/2 + margin >= B)
           || ...
        (cy1 - dia1/2 <= margin) || (cy1 + dia1/2 + margin >= H)
           || ...
        (cx2 - dia2/2 <= margin) || (cx2 + dia2/2 + margin >= B)
           || ...
        (cy2 - dia2/2 <= margin) || (cy2 + dia2/2 + margin >= H);

    dist_centers = sqrt((cx1 - cx2)^2 + (cy1 - cy2)^2);
    overlap = dist_centers < (dia1 + dia2)/2;

    if ~(boundaryViolation || overlap)
        break;
    end

end
```

4.8.4 Network Definition

The two networks used here follow the same strategy as before and predict the displacement response for the unit body force cases. Listing 4.10 shows the network definition.

Listing 4.10. Network definition for Case 6.

```
nOut1 = nDOF;
```

```
nOut2 = nDOF;
nOut3 = 3*nNodes;

layers1 = [
    featureInputLayer(inputs,"normalisation","none")
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(nOut1)
];

layers2 = [
    featureInputLayer(inputs,"normalisation","none")
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(nOut2)
];
```

4.8.5 Inference and Results

For a test geometry, the eight input parameters are normalised using the scaling used for training. The geometric parameters and loading condition used for this test case are summarised in Table 4.18.

The two displacement networks predict the responses for the unit body force cases, which are then scaled and superposed for the selected loading condition. The two hole outlines are added to the displacement plots to show the position of the void regions.

Error Evaluation

The displacement prediction errors for the two-hole case are reported in Table 4.19.

The relative L^2 error of 4.613×10^{-2} corresponds to an overall displacement

Table 4.18. Test parameters for the two-hole structure case (Case 6).

Parameter	Value
B	1.25
H	2.00
d_1	0.30
c_{x1}	0.50
c_{y1}	1.50
d_2	0.15
c_{x2}	0.65
c_{y2}	0.40
Body force	$[2, 3]^T$

error of approximately 4.6% relative to the finite element solution, indicating that the network captures the overall displacement response. The relative L^∞ error is higher, at approximately 7.6%, indicating that the largest local deviations are visible near the hole.

Table 4.19. Displacement prediction errors for the two-hole structure case (Case 6).

Error measure	Value
RMSE	8.630×10^{-1}
Relative L^2 error	4.613×10^{-2}
Relative L^∞ error	7.608×10^{-2}

Figure 4.9 shows the comparison between the finite element and neural network displacement fields for the two-hole structural domain. The L^∞ error values are consistent with the localised response around the holes. The low-stiffness regions produce sharp changes in the effective stiffness distribution and stronger displacement gradients near the hole boundaries and between the two holes. These local features are more difficult for the network to reproduce than the smoother global displacement field. In addition, this case has a larger input space because the model

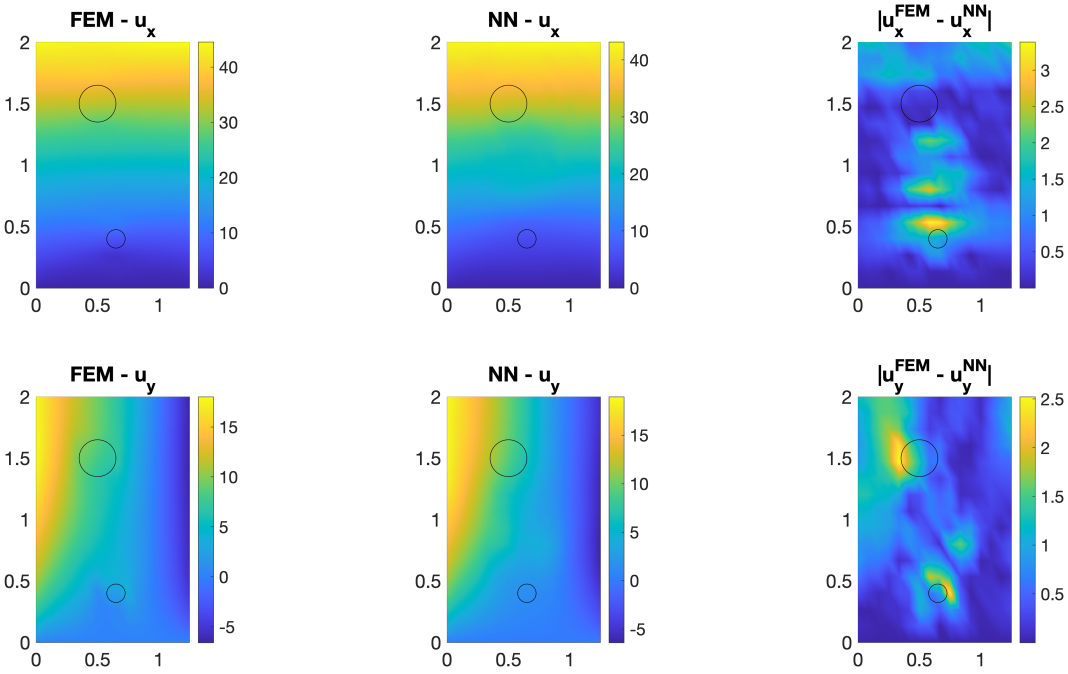


Figure 4.9. Comparison of FEM and neural network displacement fields with absolute error plots for Case 6.

depends on eight geometric parameters, which may further contribute to the higher errors.

Although the geometry is more complex than the rectangular and quadrilateral cases, the updated errors show that the network captures the global displacement field with reasonable accuracy. The larger local errors remain concentrated near the hole boundaries and between the two holes.

Computational Time

The computational time for the two-hole structural domain case is summarised in Table 4.20. The higher data-generation and training times compared with the simpler cases is because of the larger parameter space and the more complex displacement fields produced by the two-hole geometry. The relatively higher time is due to a deeper network being used to capture the complex mappings of this case.

Table 4.20. Computational time for the two-hole structural domain case (Case 6).

Step	Time [s]
FEM data generation	75.613
Training of neural networks	346.100
Inference	0.916

4.9 Limitations of Data-Driven Models

The cases studied reveal the advantages of data-driven neural networks, but there are still some limitations that must be discussed. These models perform well for geometries or loading conditions that are within the range of the training dataset. For a test geometry that lies outside the trained parameter range, the network may not generalise the output as accurately.

A major obstacle comes in training data-driven models, as they require a large number of pre-computed finite element solutions. The generation of this dataset can be computationally expensive, especially as the mesh gets denser or the geometry gets more complex.

These limitations encourage the use of physics-informed neural networks in Chapter 5, where the equilibrium equations, constitutive relations, and boundary conditions are incorporated directly into the loss function, rather than the dependence on finite element training data.

Chapter 5

Physics-informed Neural Networks

This chapter provides an alternative way of solving boundary value problems using physics-informed neural networks (PINNs). Instead of training the network on data from finite element solver, this network trains on physics-based governing equations and boundary conditions imposed through the loss function for the training. Therefore, while minimising the loss function during training, the network learns to satisfy the mechanics of the structural problem.

In this chapter, two PINN formulations are studied. In the strong-form PINN, the loss function contains the equilibrium equation residuals and the boundary condition terms. In the energy-based variational PINN, the loss is instead based on the total potential energy of the system. In both cases, the governing mechanics are introduced directly into the training process.

PINNs can be useful when limited training data is available or when physical constraints are required to guide the learning process. Since the displacement field is represented as a differentiable function, strains and stresses can also be obtained using automatic differentiation. However, the satisfaction of the governing equations is not exact in a soft-constrained formulation. The equations and boundary conditions are satisfied approximately by minimising their corresponding loss terms.

5.1 Problem Definition

In this work, the PINN is used to approximate the displacement response of a two-dimensional rectanhgular plane-stress structure. The network takes the spatial coordinates and the structural dimensions as inputs and predicts the displacement field in the domain.

The structure is rectangular, with variable width B and height H. The bottom edge is fixed, while the remaining boundaries are traction-free. Body forces are applied in the domain, and the response is studied for independent body force directions. For the parametrised cases, the network input is written as:

$$\mathbf{X} = \begin{bmatrix} \hat{x} & \hat{y} & \hat{B} & \hat{H} \end{bmatrix}, \tag{5.1}$$

where $\hat{x}$ and $\hat{y}$ are the normalised spatial coordinates, and $\hat{B}$ and $\hat{H}$ are the normalised geometry parameters.

The aim of this chapter is to test whether PINNs can approximate the displacement and stress fields of the structure without using finite element displacement data during training. The finite element solution is only used as a reference for testing.

5.2 Strong Form PINN Formulation

5.2.1 Governing Equations

The rectangular structure is modelled under plane stress conditions. Initially, PINN performance is tested for a fixed-geometry case, where the inputs are only (x, y), the spatial coordinates in the domain. Then, the parametrised case is solved, where the geometric parameters B and H of the rectangular structure are also included as inputs. Therefore, for the parametrised case, the neural network approximates the displacement field based on four inputs as:

$$\mathbf{u}_\theta(x, y, B, H) = \begin{bmatrix} u_x(x, y, B, H) \\ u_y(x, y, B, H) \end{bmatrix}, \tag{5.2}$$

where θ denotes the trainable parameters of the neural network.

The PINN formulation is developed based on the physical governing equations of the plane-stress problem. The strain-displacement relations are defined as:

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}, \qquad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}, \qquad \gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}. \tag{5.3}$$

The constitutive equations are:

$$\sigma_{xx} = \frac{E}{1-\nu^2}\left(\varepsilon_{xx} + \nu\varepsilon_{yy}\right), \tag{5.4}$$

$$\sigma_{yy} = \frac{E}{1-\nu^2}\left(\varepsilon_{yy} + \nu\varepsilon_{xx}\right), \tag{5.5}$$

$$\sigma_{xy} = G\gamma_{xy}, \qquad G = \frac{E}{2(1+\nu)}. \tag{5.6}$$

The strong form equilibrium equations are:

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + f_x = 0, \tag{5.7}$$

$$\frac{\partial \sigma_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + f_y = 0. \tag{5.8}$$

The corresponding residuals are defined as:

$$r_x = \frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + f_x, \tag{5.9}$$

$$r_y = \frac{\partial \sigma_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + f_y. \tag{5.10}$$

By reducing these residuals at the defined collocation points in the rectangular structure domain, the PINN is trained.

5.2.2 Loss Function

In the strong-form PINN formulation, the loss function helps impose the governing equations and boundary conditions. The total loss used in this work is composed of three main terms:

$$\mathcal{L} = \mathcal{L}_{\text{PDE}} + W_{\text{bc}}\mathcal{L}_{\text{bc}} + W_{\text{tr}}\mathcal{L}_{\text{tr}}, \tag{5.11}$$

where $\mathcal{L}_{\text{PDE}}$ penalises the residuals of the differential equations, $\mathcal{L}_{\text{bc}}$ penalises the violation of the fixed displacement boundary condition, and $\mathcal{L}_{\text{tr}}$ penalises the violation of the traction-free boundary conditions. The weighing factors, W_{bc} and W_{tr}, are constants set to balance the different loss contributions.

The PDE loss is evaluated at the interior collocation points as:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_\Omega} \sum_{i=1}^{N_\Omega} \left[r_x^2(\mathbf{x}_i) + r_y^2(\mathbf{x}_i) \right], \tag{5.12}$$

where N_Ω is the number of interior collocation points, and r_x and r_y are the equilibrium residuals defined previously.

The boundary terms are evaluated at boundary collocation points. The bottom edge is fixed, while the top, left, and right edges are traction-free. In the strong-form PINN case, these conditions are included as soft constraints, meaning that the network is trained to reduce their residuals rather than satisfying them exactly.

5.2.3 Preliminary Case: Fixed Geometry

Preliminary studies are conducted to smoothly transition to physics-informed networks. Before studying the parametrised problem, a fixed-geometry case is considered with one body force. In this preliminary case, the structure dimensions are fixed as:

$$B = 2, \qquad H = 2. \tag{5.13}$$

The neural network inputs are only the spatial coordinates x and y. The network predicted the two displacement components as:

$$\mathbf{u}_\theta(x, y) = \begin{bmatrix} u_x(x, y) \\ u_y(x, y) \end{bmatrix}. \tag{5.14}$$

Since the dimensions of the structure are not included as inputs, this case does not represent a geometric surrogate model. However, it is useful as an initial verification step where it allows the strong-form PINN formulation to be tested on a simple problem before introducing the geometry parameters B and H.

5.2.4 Parametrised Strong-Form PINN with Two Unit Body Force Case

This strong-form PINN formulation is developed for a rectangular plane stress structure with variable dimensions B and H, aiming to approximate the displacement field for two independent unit body force cases with a single neural network.

The two unit body force cases are defined as:

$$\mathbf{f}^{(x)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{f}^{(y)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \tag{5.15}$$

The network is therefore trained to predict the displacement response for both unit load cases. Since the problem is linear elastic, the response for any constant body force can be obtained by scaling the corresponding unit body force responses and then superposing the responses in both directions, as done for data-driven cases. It is important to note that the model is not fully load-parametrised, as the body force components f_x and f_y are not direct inputs to the neural network.

Training Domain and Collocation Points

A range is set from which the geometry parameters are randomly sampled. Interior collocation points, for each geometry, are also sampled randomly inside that domain.

These points are used to evaluate the equilibrium residuals. Boundary collocation points are sampled separately on the bottom, top, left, and right edges.

The bottom boundary is fixed, while the remaining three boundaries are traction-free. The number of sampled geometries and collocation points used in this study are summarized in Table 5.1.

Table 5.1. Collocation point parameters for the strong-form PINN.

Parameter	Value
Number of sampled geometries	20
Interior collocation points per geometry	1000
Boundary points per edge per geometry	50
Geometry range for B	$[1, 3]$
Geometry range for H	$[1, 3]$

The spatial coordinates x and y and the geometry parameters are normalised in the interval $[-1, 1]$, as done for inputs in the data-driven cases. As the network inputs are normalised and the equilibrium equations are written in the physical coordinates x and y, the derivatives are scaled using the chain rule:

$$\frac{\partial}{\partial x} = \frac{2}{B}\frac{\partial}{\partial \hat{x}}, \qquad \frac{\partial}{\partial y} = \frac{2}{H}\frac{\partial}{\partial \hat{y}}. \tag{5.16}$$

This scaling is included when computing strains and stress derivatives.

Network Architecture

The network takes four inputs and approximates four outputs. The inputs are the normalised coordinates and geometry parameters. The outputs are the two displacement components for the two unit body force cases:

$$\mathbf{u}_\theta = \begin{bmatrix} u_x^{(x)} \\ u_y^{(x)} \\ u_x^{(y)} \\ u_y^{(y)} \end{bmatrix}. \tag{5.17}$$

The first two outputs correspond to the displacement field caused by the unit body force in the x-direction. The last two outputs correspond to the displacement field caused by the unit body force in the y-direction.

Listing 5.1 shows the network architecture. The hyperbolic tangent activation function is used because the strong-form PINN requires derivatives of the network output. Therefore, smooth activation functions are suitable for this formulation.

Listing 5.1. Network architecture for the strong-form PINN.

```
layers = [
    featureInputLayer(4,"Normalization","none")
    fullyConnectedLayer(128)
    tanhLayer
    fullyConnectedLayer(128)
    tanhLayer
    fullyConnectedLayer(128)
    tanhLayer
    fullyConnectedLayer(128)
    tanhLayer
    fullyConnectedLayer(4)
];

dlnet = dlnetwork(layerGraph(layers));
```

Loss Function

In the strong-form case, the total loss is defined as:

$$\mathcal{L} = \mathcal{L}_{\text{PDE}}^{(x)} + \mathcal{L}_{\text{PDE}}^{(y)} + W_{\text{bc}}\mathcal{L}_{\text{bc}} + W_{\text{tr}}\mathcal{L}_{\text{tr}}, \tag{5.18}$$

where $\mathcal{L}_{\text{PDE}}^{(x)}$ is the PDE loss for the unit body force in the x-direction and $\mathcal{L}_{\text{PDE}}^{(y)}$ is the PDE loss for the unit body force in the y-direction. The other terms penalise the fixed displacement boundary condition and the traction-free boundary conditions.

For the first load case in the x-direction, the residuals are

$$r_x^{(x)} = \frac{\partial \sigma_{xx}^{(x)}}{\partial x} + \frac{\partial \sigma_{xy}^{(x)}}{\partial y} + 1, \qquad r_y^{(x)} = \frac{\partial \sigma_{xy}^{(x)}}{\partial x} + \frac{\partial \sigma_{yy}^{(x)}}{\partial y}. \tag{5.19}$$

For the second load case in the y-direction, the residuals are

$$r_x^{(y)} = \frac{\partial \sigma_{xx}^{(y)}}{\partial x} + \frac{\partial \sigma_{xy}^{(y)}}{\partial y}, \qquad r_y^{(y)} = \frac{\partial \sigma_{xy}^{(y)}}{\partial x} + \frac{\partial \sigma_{yy}^{(y)}}{\partial y} + 1. \tag{5.20}$$

Here, the value 1 represents the unit body force in the corresponding direction. The two PDE loss terms are then written as

$$\mathcal{L}_{\text{PDE}}^{(x)} = \frac{1}{N_\Omega} \sum_{i=1}^{N_\Omega} \left[\left(r_x^{(x)}(\mathbf{x}_i) \right)^2 + \left(r_y^{(x)}(\mathbf{x}_i) \right)^2 \right], \tag{5.21}$$

$$\mathcal{L}_{\text{PDE}}^{(y)} = \frac{1}{N_\Omega} \sum_{i=1}^{N_\Omega} \left[\left(r_x^{(y)}(\mathbf{x}_i) \right)^2 + \left(r_y^{(y)}(\mathbf{x}_i) \right)^2 \right]. \tag{5.22}$$

The bottom boundary is fixed for both load cases. Therefore, all four network outputs are penalised on $y = 0$.

$$u_x^{(x)} = u_y^{(x)} = u_x^{(y)} = u_y^{(y)} = 0 \qquad \text{on } y = 0. \tag{5.23}$$

The displacement boundary loss is computed as

$$\mathcal{L}_{\text{bc}} = \frac{1}{N_{\Gamma_u}} \sum_{i=1}^{N_{\Gamma_u}} \left[\left(u_x^{(x)} \right)^2 + \left(u_y^{(x)} \right)^2 + \left(u_x^{(y)} \right)^2 + \left(u_y^{(y)} \right)^2 \right]_{\mathbf{x}_i}. \tag{5.24}$$

The remaining boundaries are traction-free for both load cases. On the top boundary, the penalized components are

$$\sigma_{yy}^{(x)}, \quad \sigma_{xy}^{(x)}, \quad \sigma_{yy}^{(y)}, \quad \sigma_{xy}^{(y)}. \tag{5.25}$$

On the right and left boundaries, the penalized components are

$$\sigma_{xx}^{(x)}, \quad \sigma_{xy}^{(x)}, \quad \sigma_{xx}^{(y)}, \quad \sigma_{xy}^{(y)}. \tag{5.26}$$

Therefore, the traction loss is written as

$$\mathcal{L}_{\text{tr}} = \mathcal{L}_{\text{tr,top}} + \mathcal{L}_{\text{tr,right}} + \mathcal{L}_{\text{tr,left}}. \tag{5.27}$$

For example, the loss contribution from the vertical boundaries is

$$\begin{aligned}\mathcal{L}_{\text{tr,vertical}} = \frac{1}{N_\Gamma}\sum_{i=1}^{N_\Gamma}\Big[&\left(\sigma_{xx}^{(x)}(\mathbf{x}_i)\right)^2 + \left(\sigma_{xy}^{(x)}(\mathbf{x}_i)\right)^2 \\ &+ \left(\sigma_{xx}^{(y)}(\mathbf{x}_i)\right)^2 + \left(\sigma_{xy}^{(y)}(\mathbf{x}_i)\right)^2\Big].\end{aligned} \tag{5.28}$$

For the top boundary, the same expression is used with σ_{yy} replacing σ_{xx}.

Implementation of the Loss Function

The four network displacement outputs are first predicted and then separated into the two unit load cases, as shown in Listing 5.2. Automatic differentiation is then used to compute the displacement gradients, strain components, stress components, and equilibrium residuals. Since the inputs are normalised, the chain-rule scaling factors $2/B$ and $2/H$ are applied to all derivatives with respect to x and y.

The displacement boundary condition is applied to all four outputs on the fixed bottom boundary. Since the prescribed displacement is zero, the boundary loss is computed directly from the predicted boundary displacement. The final loss is computed based on the equations given earlier. It includes the boundary condition weight W_{bc} and traction loss weight W_{tr}. In this study, both weights are set equal to 1.

Listing 5.2. Loss calculation for the strong-form PINN.

```
function [loss, loss_pde_x, loss_pde_y, loss_bc, loss_tr,
    gradients] = pinnLoss( ...
    dlnet, X, Xbc, UBoundary, Xtr, E, nu, fx, fy, ...
    B_min, B_max, H_min, H_max)

    Bn = X(3,:);
    Hn = X(4,:);

    B = (Bn + 1)/2 * (B_max - B_min) + B_min;
    H = (Hn + 1)/2 * (H_max - H_min) + H_min;

    U = forward(dlnet, X);

    ux_fx = U(1,:);
    uy_fx = U(2,:);
```

```
15
16      ux_fy = U(3,:);
17      uy_fy = U(4,:);
18
19      gradUx_fx = dlgradient(sum(ux_fx), X, ...
20          'EnableHigherDerivatives', true);
21      gradUy_fx = dlgradient(sum(uy_fx), X, ...
22          'EnableHigherDerivatives', true);
23
24      gradUx_fy = dlgradient(sum(ux_fy), X, ...
25          'EnableHigherDerivatives', true);
26      gradUy_fy = dlgradient(sum(uy_fy), X, ...
27          'EnableHigherDerivatives', true);
28
29      dux_dx_fx = gradUx_fx(1,:) .* (2 ./ B);
30      dux_dy_fx = gradUx_fx(2,:) .* (2 ./ H);
31      duy_dx_fx = gradUy_fx(1,:) .* (2 ./ B);
32      duy_dy_fx = gradUy_fx(2,:) .* (2 ./ H);
33
34      dux_dx_fy = gradUx_fy(1,:) .* (2 ./ B);
35      dux_dy_fy = gradUx_fy(2,:) .* (2 ./ H);
36      duy_dx_fy = gradUy_fy(1,:) .* (2 ./ B);
37      duy_dy_fy = gradUy_fy(2,:) .* (2 ./ H);
38
39      exx_fx = dux_dx_fx;
40      eyy_fx = duy_dy_fx;
41      gxy_fx = dux_dy_fx + duy_dx_fx;
42
43      exx_fy = dux_dx_fy;
44      eyy_fy = duy_dy_fy;
45      gxy_fy = dux_dy_fy + duy_dx_fy;
46
47      C = E/(1 - nu^2);
48      G = E/(2*(1 + nu));
49
50      sxx_fx = C*(exx_fx + nu*eyy_fx);
51      syy_fx = C*(eyy_fx + nu*exx_fx);
52      sxy_fx = G*gxy_fx;
53
54      sxx_fy = C*(exx_fy + nu*eyy_fy);
55      syy_fy = C*(eyy_fy + nu*exx_fy);
56      sxy_fy = G*gxy_fy;
57
58      gradSxx_fx = dlgradient(sum(sxx_fx), X, ...
59          'EnableHigherDerivatives', true);
60      gradSxy_fx = dlgradient(sum(sxy_fx), X, ...
61          'EnableHigherDerivatives', true);
62      gradSyy_fx = dlgradient(sum(syy_fx), X, ...
63          'EnableHigherDerivatives', true);
64
65      gradSxx_fy = dlgradient(sum(sxx_fy), X, ...
66          'EnableHigherDerivatives', true);
67      gradSxy_fy = dlgradient(sum(sxy_fy), X, ...
```

```
        'EnableHigherDerivatives', true);
    gradSyy_fy = dlgradient(sum(syy_fy), X, ...
        'EnableHigherDerivatives', true);

    dsxx_dx_fx = gradSxx_fx(1,:) .* (2 ./ B);
    dsxy_dy_fx = gradSxy_fx(2,:) .* (2 ./ H);
    dsxy_dx_fx = gradSxy_fx(1,:) .* (2 ./ B);
    dsyy_dy_fx = gradSyy_fx(2,:) .* (2 ./ H);

    dsxx_dx_fy = gradSxx_fy(1,:) .* (2 ./ B);
    dsxy_dy_fy = gradSxy_fy(2,:) .* (2 ./ H);
    dsxy_dx_fy = gradSxy_fy(1,:) .* (2 ./ B);
    dsyy_dy_fy = gradSyy_fy(2,:) .* (2 ./ H);

    res1_x = dsxx_dx_fx + dsxy_dy_fx + fx;
    res2_x = dsxy_dx_fx + dsyy_dy_fx;

    res1_y = dsxx_dx_fy + dsxy_dy_fy;
    res2_y = dsxy_dx_fy + dsyy_dy_fy + fy;

    loss_pde_x = mean(res1_x.^2,'all') + mean(res2_x.^2,'all');
    loss_pde_y = mean(res1_y.^2,'all') + mean(res2_y.^2,'all');

    Ubc = forward(dlnet, Xbc);
    loss_bc = mean(Ubc.^2,'all');

    loss_tr = 0;

    if size(Xtr.top,2) > 0
        [~, syy_x, sxy_x] = computeStressCase( ...
            dlnet, Xtr.top, E, nu, B_min, B_max, H_min, H_max, 1);
        [~, syy_y, sxy_y] = computeStressCase( ...
            dlnet, Xtr.top, E, nu, B_min, B_max, H_min, H_max, 2);

        loss_tr = loss_tr ...
            + mean(syy_x.^2,'all') + mean(sxy_x.^2,'all') ...
            + mean(syy_y.^2,'all') + mean(sxy_y.^2,'all');
    end

    if size(Xtr.right,2) > 0
        [sxx_x, ~, sxy_x] = computeStressCase( ...
            dlnet, Xtr.right, E, nu, B_min, B_max, H_min, H_max,
                1);
        [sxx_y, ~, sxy_y] = computeStressCase( ...
            dlnet, Xtr.right, E, nu, B_min, B_max, H_min, H_max,
                2);

        loss_tr = loss_tr ...
            + mean(sxx_x.^2,'all') + mean(sxy_x.^2,'all') ...
            + mean(sxx_y.^2,'all') + mean(sxy_y.^2,'all');
    end

    if size(Xtr.left,2) > 0
```

```
        [sxx_x, ~, sxy_x] = computeStressCase( ...
            dlnet, Xtr.left, E, nu, B_min, B_max, H_min, H_max, 1)
                ;
        [sxx_y, ~, sxy_y] = computeStressCase( ...
            dlnet, Xtr.left, E, nu, B_min, B_max, H_min, H_max, 2)
                ;

        loss_tr = loss_tr ...
            + mean(sxx_x.^2,'all') + mean(sxy_x.^2,'all') ...
            + mean(sxx_y.^2,'all') + mean(sxy_y.^2,'all');
    end

    Wbc = 1;
    Wtr = 1;

    loss = loss_pde_x + loss_pde_y + Wbc*loss_bc + Wtr*loss_tr;

    gradients = dlgradient(loss, dlnet.Learnables);

end
```

A separate helper function, shown in Listing 5.3, is used to reduce repetition. It computes the stress components for either the x-direction or y-direction unit body force case. The argument `caseID` selects the required output stress for either body force direction.

Listing 5.3. Stress calculation helper function for the strong-form PINN.

```
function [sxx, syy, sxy] = computeStressCase( ...
    dlnet, X, E, nu, B_min, B_max, H_min, H_max, caseID)

    Bn = X(3,:);
    Hn = X(4,:);

    B = (Bn + 1)/2 * (B_max - B_min) + B_min;
    H = (Hn + 1)/2 * (H_max - H_min) + H_min;

    U = forward(dlnet, X);

    if caseID == 1
        ux = U(1,:);
        uy = U(2,:);
    elseif caseID == 2
        ux = U(3,:);
        uy = U(4,:);
    end

    gradUx = dlgradient(sum(ux), X, ...
        'EnableHigherDerivatives', true);
    gradUy = dlgradient(sum(uy), X, ...
        'EnableHigherDerivatives', true);
```

```
24
25      dux_dx = gradUx(1,:) .* (2 ./ B);
26      dux_dy = gradUx(2,:) .* (2 ./ H);
27
28      duy_dx = gradUy(1,:) .* (2 ./ B);
29      duy_dy = gradUy(2,:) .* (2 ./ H);
30
31      exx = dux_dx;
32      eyy = duy_dy;
33      gxy = dux_dy + duy_dx;
34
35      C = E/(1 - nu^2);
36      G = E/(2*(1 + nu));
37
38      sxx = C*(exx + nu*eyy);
39      syy = C*(eyy + nu*exx);
40      sxy = G*gxy;
41
42  end
```

Training Strategy

Before training the network, the generated geometric parameters and collocation points are converted to `dlarray` format, because for MATLAB to compute derivatives using `dlgradient`, the inputs and network variables must be in a differentiable deep-learning format.

The training parameters are summarized in Table 5.2.

Table 5.2. Training parameters for the strong-form PINN.

Parameter	Value
Number of epochs	6000
Initial learning rate	10^{-3}
Reduced learning rate	10^{-4} after 5000 epochs
Optimizer	Adam
Boundary loss weight, W_{bc}	1
Traction loss weight, W_{tr}	1

The custom training loop is shown in Listing 5.4. The loss function is evaluated using `dlfeval`, which allows automatic differentiation inside the custom train-

ing loop. The gradients of the total loss are then used by the Adam optimizer to update the learnable network parameters.

Listing 5.4. Training loop for the strong-form PINN.

```
numEpochs = 6000;
learnRate = 1e-3;

avgGrad = [];
avgSqGrad = [];

for epoch = 1:numEpochs

    if epoch > 5000
        learnRate = 1e-4;
    end

    [loss, loss_pde_x, loss_pde_y, loss_bc, loss_tr, gradients] = ...
        dlfeval(@pinnLoss, dlnet, dlXInt, dlXB, dlUB, dlXtr, ...
        E, nu, fx, fy, B_min, B_max, H_min, H_max);

    [dlnet, avgGrad, avgSqGrad] = adamupdate( ...
        dlnet, gradients, avgGrad, avgSqGrad, epoch, learnRate);

    if mod(epoch,500) == 0
        fprintf("Epoch %d | Loss %.6f | PDEx %.6f | PDEy %.6f | BC %.6f | TR %.6f\n", ...
            epoch, extractdata(loss), extractdata(loss_pde_x), ...
            extractdata(loss_pde_y), extractdata(loss_bc), ...
            extractdata(loss_tr));
    end

end
```

Inference

After training, the network is tested on a geometry with the dimensions

$$B_{\text{test}} = 1.25, \qquad H_{\text{test}} = 2. \tag{5.29}$$

The test body force is selected as

$$f_x = 2, \qquad f_y = 3. \tag{5.30}$$

The network first predicts the two unit body force responses. The combined displacement field is then obtained by superposition.

Finite Element Reference Solution

The finite element solution is used only for validation after training. It is not used in the strong-form PINN loss function. This reference solution is compared with the superposed strong-form PINN prediction on the same structured grid.

5.2.5 Results

Error Evaluation

The strong-form PINN captures the general displacement pattern of the finite element solution. The prediction errors are reported in Table 5.3. The model gives an RMSE of 4.718×10^{0}, a relative L^2 error of 2.645×10^{-1}, and a relative L^{∞} error of 2.877×10^{-1}. These values correspond to a global displacement error of approximately 26.5% and a maximum relative difference of approximately 28.8% with respect to the finite element solution.

Table 5.3. Prediction errors for the strong-form PINN.

Error measure	Value
RMSE	4.718×10^{0}
Relative L^2 error	2.645×10^{-1}
Relative L^{∞} error	2.877×10^{-1}

The resultant displacement fields u_x and u_y for FEM and PINN together with the absolute error fields are plotted in Figure 5.1. The predicted solution captures the general deformation pattern and the overall directions of the two displacement components. However, noticeable differences in magnitude remain between the PINN prediction and the finite element reference solution.

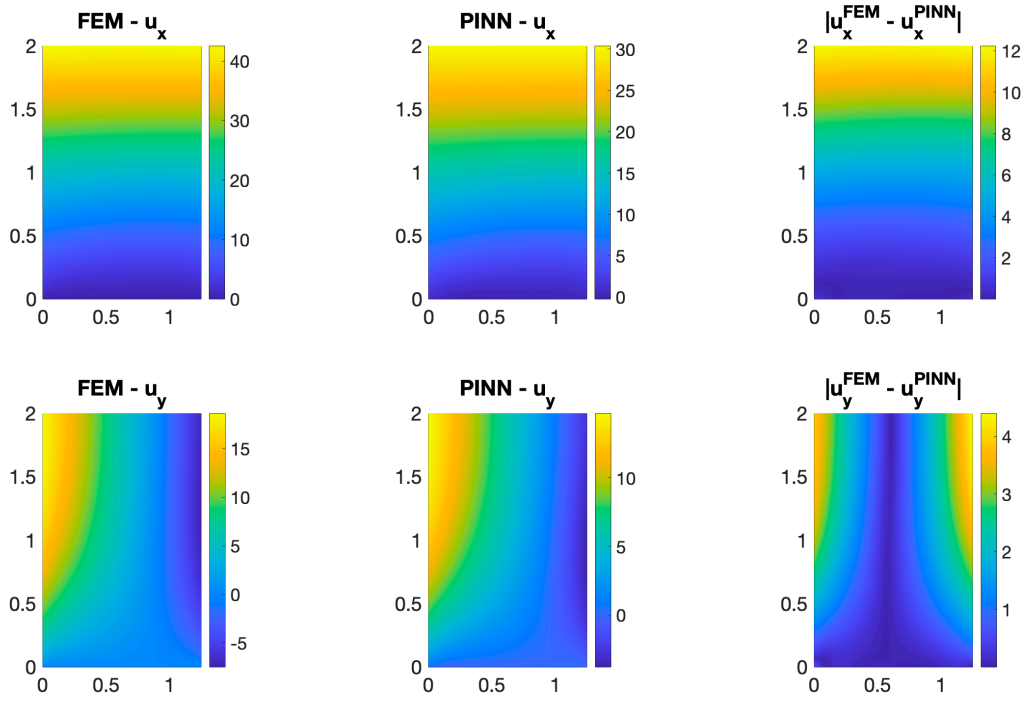


Figure 5.1. Comparison of FEM and neural network displacement fields with absolute error plots for the strong-form PINN.

These errors show that the strong-form PINN approximates the displacement field, but the accuracy is still limited for this case. This may be due to the smaller number of collocation points used, the selected network architecture, the weights of the loss terms, or the difficulty of optimising the strong-form residual loss. Increasing the number of collocation points, using a deeper or wider network, tuning the loss weights, or extending the training may improve the prediction accuracy. However, these improvements would also increase the training cost.

The stress components σ_{xx}, σ_{yy}, and σ_{xy} are also approximated from the strong-form PINN prediction using automatic differentiation. The recovered stress fields are shown in Figure 5.2.

The imposed boundary conditions are captured in the predicted stress field shown in the plots. The normal stress σ_{xx} is around zero on the left and right traction-free boundaries, and σ_{yy} is close to zero on the top traction-free boundary. The shear stress σ_{xy} is close to zero on the top, left, and right boundaries. Higher stresses can

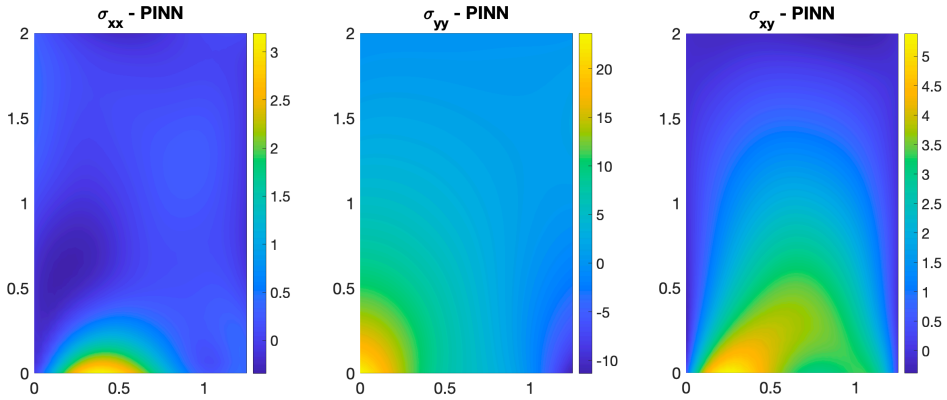


Figure 5.2. Recovered stress fields from the strong-form PINN prediction.

be seen near the fixed bottom boundary because the fixed boundary constraints create the higher stress concentrations.

Computational Time

The computational times for the strong-form PINN are reported in Table 5.4.

Table 5.4. Computational time for the strong-form PINN.

Step	Elapsed time
Geometry and collocation point generation	3.80 s
Training	14907.170 s
Inference	0.363 s

The training stage is the most computationally extensive part of the strong-form PINN. The high training time is due to the repeated calculation of the equilibrium residuals and the derivatives required to evaluate them at the collocation points. In particular, the strong-form formulation requires higher-order automatic differentiation, which increases both the computational cost and the complexity of loss computation.

The lower inference time shows that, once trained, the PINN can give predictions quickly. However, the training is costly. The cost could be reduced by using a

more efficient collocation strategy, such as combining random and structured points, reducing unnecessary boundary loss evaluations, tuning the network size, or using the variational formulation, which avoids the stress-derivative residuals required in the strong-form PINN, which is evaluated in the next section.

5.3 Energy-Based Variational PINN Formulation

In the strong-form PINN approach, the loss function contains the residuals of the equilibrium equations. Since the residuals require the derivatives of the stress components, second-order spatial derivatives of the displacement field are involved. This increases the computational cost of training.

There is another alternative approach, called the energy-based variational formulation. The method used is to train the network by minimising the total potential energy of the elastic body rather than the pointwise equilibrium residuals. The order of spatial differentiation used in the loss function is decreased because the strain energy is only dependent on the first derivatives of the displacement field.

The fixed edge boundary condition is still enforced as a penalty term. The traction-free boundaries do not, however, need to be specifically penalized, as they are natural boundary conditions for the variational problem when no external surface traction is applied.

5.3.1 Variational Formulation

For a linear elastic body, the total potential energy is given by

$$\Pi(\mathbf{u}) = \int_{\Omega} \left[\frac{1}{2} \boldsymbol{\sigma} : \boldsymbol{\varepsilon} - \mathbf{b} \cdot \mathbf{u} \right] d\Omega, \tag{5.31}$$

where $\boldsymbol{\sigma}$ is the stress tensor, $\boldsymbol{\varepsilon}$ is the strain tensor, $\mathbf{b}$ is the body force vector, and $\mathbf{u}$ is the displacement field.

For the two-dimensional plane stress problem, the energy density can be written as

$$\pi = \frac{1}{2}\left(\sigma_{xx}\varepsilon_{xx} + \sigma_{yy}\varepsilon_{yy} + \sigma_{xy}\gamma_{xy}\right) - f_x u_x - f_y u_y. \tag{5.32}$$

The strain-displacement and plane stress constitutive relations are the same as those defined for the strong-form PINN. The main difference is that the loss function is now based on the energy functional instead of the equilibrium residuals.

5.3.2 Loss Function

The variational PINN is tested for the same two independent unit body force cases used in the strong-form formulation. For the unit body force in the x-direction $f_x = 1$, the energy functional is

$$\Pi^{(x)} = \int_\Omega \left[\frac{1}{2}\left(\sigma_{xx}^{(x)}\varepsilon_{xx}^{(x)} + \sigma_{yy}^{(x)}\varepsilon_{yy}^{(x)} + \sigma_{xy}^{(x)}\gamma_{xy}^{(x)}\right) - f_x u_x^{(x)}\right] d\Omega, \tag{5.33}$$

For the unit body force in the y-direction, $f_y = 1$, the energy functional is

$$\Pi^{(y)} = \int_\Omega \left[\frac{1}{2}\left(\sigma_{xx}^{(y)}\varepsilon_{xx}^{(y)} + \sigma_{yy}^{(y)}\varepsilon_{yy}^{(y)} + \sigma_{xy}^{(y)}\gamma_{xy}^{(y)}\right) - f_y u_y^{(y)}\right] d\Omega, \tag{5.34}$$

The total variational loss is then defined as

$$\mathcal{L} = \Pi^{(x)} + \Pi^{(y)} + W_{\text{bc}}\mathcal{L}_{\text{bc}}, \tag{5.35}$$

where $\mathcal{L}_{\text{bc}}$ penalizes the displacement violation on the fixed bottom boundary. No traction loss term is required as in the case of the strong-form PINN, .

Randomly sampled collocation points are used to estimate the domain integrals. A Monte Carlo-type average multiplied by the domain's area is used to approximate the integral as the points are uniformly sampled inside each rectangular domain:

$$\Pi \approx BH\frac{1}{N_\Omega}\sum_{i=1}^{N_\Omega}\pi(\mathbf{x}_i). \tag{5.36}$$

5.3.3 Parametrised Variational PINN with Two Unit Body Force Case

The variational formulation is tested on the same parametrised rectangular plane stress problem. The same four network inputs and the four outputs representing the displacement fields for the two independent unit body force cases are used. Subsequently, the same input normalization and chain-rule scaling are used for the spatial derivatives.

Training Domain and Collocation Points

The same type of parametrised rectangular geometry is used. As traction-free boundary points are not required, only interior collocation points and fixed-boundary points are sampled. They are generated the same way as in the strong form PINN case. More details can be seen in Table 5.5.

Table 5.5. Collocation point parameters for the variational PINN.

Parameter	Value
Number of sampled geometries	20
Interior collocation points per geometry	1000
Boundary points on fixed edge per geometry	50
Geometry range for B	$[1, 3]$
Geometry range for H	$[1, 3]$

Network Architecture

The same network architecture used for the strong-form PINN, shown in Listing 5.1, is also used in the variational formulation. The network has four inputs and four outputs, corresponding to the two displacement components for the two unit body force cases.

Implementation of the Energy Loss

The loss function first evaluates the four displacement outputs. The displacement gradients are then computed using automatic differentiation. These gradients are used to calculate the strain and stress components for both unit body force cases.

The potential energy densities are computed as

Listing 5.5. Potential energy density for the variational PINN.

```
W_fx = 0.5*(sxx_fx.*exx_fx + syy_fx.*eyy_fx ...
     + sxy_fx.*gxy_fx) - fx*ux_fx;

W_fy = 0.5*(sxx_fy.*exx_fy + syy_fy.*eyy_fy ...
     + sxy_fy.*gxy_fy) - fy*uy_fy;
```

The mean energy density is multiplied by the area of each geometry to approximate the domain integral:

Listing 5.6. Approximation of the energy integral for the variational PINN.

```
W_fx = W_fx .* B .* H;
W_fy = W_fy .* B .* H;

loss_e_x = mean(W_fx,'all');
loss_e_y = mean(W_fy,'all');
```

The fixed bottom boundary is included using a penalty term:

Listing 5.7. Fixed-boundary penalty for the variational PINN.

```
Ubc = forward(dlnet, Xbc);
loss_bc = mean(Ubc.^2,'all');
```

The final loss is then computed as

Listing 5.8. Final loss for the variational PINN.

```
Wbc = 100;
loss = loss_e_x + loss_e_y + Wbc*loss_bc;
gradients = dlgradient(loss, dlnet.Learnables);
```

A large value of W_{bc} helps in the penalisation of the fixed boundary condition as it is not naturally imposed in the energy method.

Training Strategy

The training procedure is also similar to the strong-form PINN. The collocation points are converted to `dlarray` format, and the loss is evaluated using `dlfeval` inside a custom training loop. The training parameters are displayed in Table 5.6.

Table 5.6. Training parameters for the variational PINN.

Parameter	Value
Number of epochs	6000
Initial learning rate	10^{-3}
Reduced learning rate	10^{-4} after 5000 epochs
Optimizer	Adam
Boundary loss weight, W_{bc}	100

Inference

The inference procedure is the same as in the strong-form PINN. The trained network predicts the two unit body force responses, and the total response is obtained by superposition. This model is also tested on the same the test case.

5.3.4 Results

Error Evaluation

The variational PINN gives a closer approximation to the finite element displacement field than the strong-form PINN. The displacement prediction errors are summarised in Table 5.7. The model achieves an RMSE of 4.802×10^{-1}, a relative L^2 error of 2.692×10^{-2}, and a relative L^∞ error of 3.260×10^{-2}. These values correspond to a global displacement error of approximately 2.7% and a maximum relative difference of approximately 3.3% with respect to the finite element solution.

Table 5.7. Prediction errors for the variational PINN.

Error measure	Value
RMSE	4.802×10^{-1}
Relative L^2 error	2.692×10^{-2}
Relative L^∞ error	3.260×10^{-2}

Figure 5.3 compares the finite element and variational PINN displacement fields and presents the corresponding absolute error distributions. The predicted fields reproduce the overall magnitude and spatial variation of the finite element response closely. The errors remain comparatively small over most of the domain, although local deviations are visible in some regions.

The variational formulation gives better results than the strong-form PINN in this work. One possible reason is that as variational loss requires only the first derivatives of the displacement field, they can be improved and are less sensitive to derivative errors. Meanwhile, the strong-form formulation evaluates equilibrium residuals involving higher-order displacement derivatives, which are generally more sensitive to local approximation and automatic-differentiation errors. Additionally, this method integrates traction free boundary conditions naturally, therefore, it does not have an additional traction loss to penalise, leading to better prediction. However, the two cases use different weights for boundary loss, which can also lead to differences in accuracy, so the differences can not solely be linked to one particular reason.

The stress components σ_{xx}, σ_{yy}, and σ_{xy} are also recovered from the variational PINN prediction using automatic differentiation. The recovered stress fields are shown in Figure 5.4.

The stress fields are consistent with the expected behaviour of the structure. Larger stress values are observed near the fixed bottom boundary where the displacement constraint is applied. But since the stresses are obtained from displacement gradients, they remain more sensitive to local errors in the displacement field than

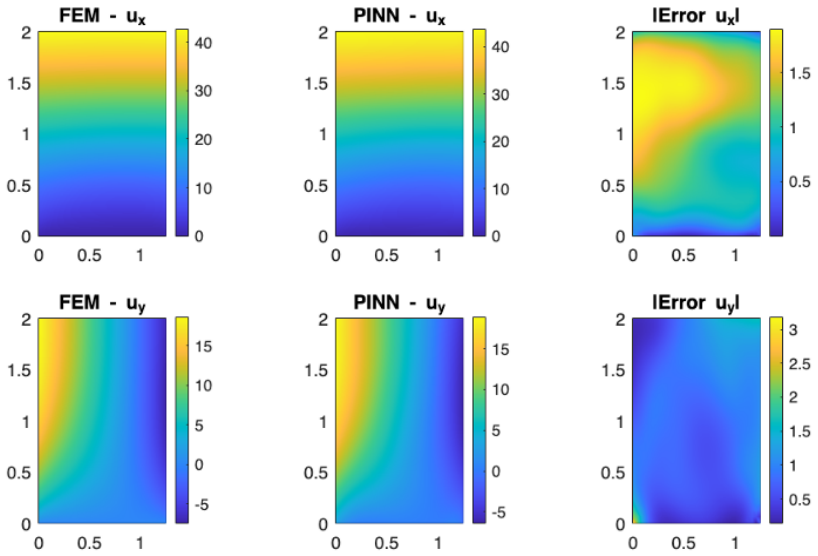


Figure 5.3. Comparison of FEM and neural network displacement fields with absolute error plots for the variational PINN.

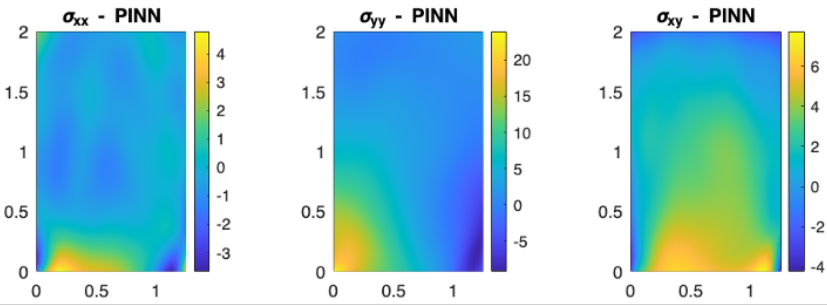


Figure 5.4. Recovered stress fields from the variational PINN prediction.

the displacement prediction itself. Small local displacement errors can be amplified during differentiation, leading to less smooth fields.

Computational Time

The computational times for the variational PINN are reported in Table 5.8.

The training stage is again the most computationally expensive part of the procedure due to repeated evaluation of the energy functional and its gradients over the collocation points during minimisation of loss function. However, this is an ini-

Table 5.8. Computational time for the variational PINN.

Step	Elapsed time
Geometry and collocation point generation	7.19 s
Training	3010.14 s
Inference	0.194 s

tial computational cost, while the low inference time allows the trained model to be evaluated rapidly for new input geometries within the prescribed parameter range.

5.4 Limitations of Physics-Informed Models

Training in physics-informed models depends on many factors, such as distribution of the points of aggregation over the domain, the network architecture, the weights of the loss functions, the number of epochs, the learning rate and more.

In this work, it is observed that PINNs are very sensitive to derivative calculations, which makes training strong-form PINNs very computationally expensive and extensive. Increasing this computational power may help increase the accuracy of the model prediction. Variational PINNs are a good alternative to reduce the derivative calculation work.

Another limitation of this work is that the simple rectangular plane-stress structures are studied. More complex geometries, material models, boundary conditions, and loading cases need to be tested before the general reliability of PINNs can be confirmed.

A suggested improvement is creating the model using hybrid collocation points generation methods, which would include both random and grid-based generation.

Chapter 6

Dynamic Extension: Modal Analysis

The thesis work now expands from the linear static case to a dynamic case. This chapter encompasses the modal analysis of a free vibration case of a two-dimensional plane-stress rectangular structure with unit thickness. The mode shapes obtained in this case show in-plane vibration modes of the structure and the out-of-plane modes are not represented. The analysis is based on modal data obtained from the finite element solver and a data-driven neural network is trained to compute the mode shapes.

In this case study, modal analysis is used to obtain reference mode shapes from the finite element model. The data is then given to the neural network for training to predict neural responses for various structural geometries in terms of the mode of deformation. Unlike in static analysis, no external force is required for modal analysis because the mode shapes are obtained from the mass and stiffness properties of the structure.

As the structure is two-dimensional, though a unit thickness is introduced and uses in-plane displacement degrees of freedom, the computed modes describe in-plane vibration patterns. Out-of-plane bending or torsional modes of a real three-dimensional structure are not represented in this case. Therefore, the modal analysis in this thesis should be interpreted within the assumptions of the plane stress model used [8, 12].

6.1 Problem Definition

This dynamic problem is solved on a rectangular plane-stress structure with a unit thickness. The geometry is parametrised by the width B and height H. These two geometric parameters are used as input variables in the neural network.

The finite element mesh used contains 8×8 nodes. Therefore, the total number of nodes is:

$$n_{\text{nodes}} = 64. \tag{6.1}$$

Since each node has two in-plane displacement degrees of freedom, u_x and u_y, the total number of degrees of freedom of the system is:

$$n_{\text{DOF}} = 2n_{\text{nodes}} = 128. \tag{6.2}$$

The geometric parameters are sampled within the following ranges:

$$B \in [1, 3], \qquad H \in [1, 3]. \tag{6.3}$$

A total of 2000 samples are generated. For each sample, a finite element modal analysis is carried out and the first 3 mode shapes are stored. The first three modes are selected because they represent the lowest-frequency global in-plane vibration patterns of the structure. This is sufficient for the simple structure selected for this modal analysis. The material and model parameters used in the analysis are listed in Table 6.1.

6.2 Finite Element Implementation

The finite element model is used to generate the reference modal data similar to the previous cases. The random geometric parameters of the rectangular structure, B and H, are provided by the finite element solver as input. For every pair of values, the stiffness matrix $\mathbf{K}$ and mass matrix $\mathbf{M}$ are assembled. The output of the solver is the set of mode shapes $\boldsymbol{\phi}$ for each node and each degree of freedom.

Table 6.1. Material and model parameters used in the dynamic analysis.

Parameter	Symbol	Value
Young's modulus	E	1
Poisson's ratio	ν	0.3
Density	ρ	1
Thickness	t	1
Number of modes	n_{modes}	3

The stiffness matrix depends on the material stiffness defined by E and the geometry of the structure. The mass matrix depends on the density, thickness, and geometry. For this dynamic case, the density ρ is introduced because inertia effects are required in the free-vibration equation.

The stiffness and mass matrices are calculated using the function shown in Listing 6.1.

Listing 6.1. Assembly of stiffness and mass matrices for modal analysis.

```
function [K, M] = buildPlaneStressModalFEM(coords, nx, ny, E, ...
    nu, rho, t)

    nNodes = nx * ny;
    nElemX = nx - 1;nElemY = ny - 1;
    nElem = nElemX * nElemY;

    ndofNode = 2;
    nDOF = nNodes * ndofNode;

    conn = zeros(nElem, 4);

    for ix = 1:nElemX
        for jy = 1:nElemY
            n1 = (ix-1) * ny + jy;
            n2 = ix * ny + jy;
            n3 = ix * ny + (jy+1);
            n4 = (ix-1) * ny + (jy+1);

            e = (ix-1) * nElemY + jy;
            conn(e,:) = [n1 n2 n3 n4];
        end
    end

    K = zeros(nDOF, nDOF);
```

```
    M = zeros(nDOF, nDOF);

    D = E / (1 - nu^2) * [1,  nu, 0;
                          nu, 1,  0;
                          0,  0,  (1-nu)/2];

    gp  = [-1/sqrt(3), 1/sqrt(3)];
    wgp = [1, 1];

    for e = 1:nElem
        nodes = conn(e,:);
        xe = coords(nodes,1);
        ye = coords(nodes,2);

        Ke = zeros(8,8);
        Me = zeros(8,8);

        [~, dNdxi0, dNdeta0] = shape4(0, 0);
        J0 = [dNdxi0'; dNdeta0'] * [xe, ye];
        invJ0 = inv(J0);

        dN0 = [dNdxi0, dNdeta0] * invJ0;
        B0 = zeros(3,8);
        for a = 1:4
            dNx = dN0(a,1);
            dNy = dN0(a,2);
            col = (a-1) * 2;

            B0(1, col+1) = dNx;
            B0(2, col+2) = dNy;
            B0(3, col+1) = dNy;
            B0(3, col+2) = dNx;
        end

        for i = 1:2
            for j = 1:2
                xi  = gp(i);
                eta = gp(j);

                [N, dNdxi, dNdeta] = shape4(xi, eta);

                J = [dNdxi'; dNdeta'] * [xe, ye];
                detJ = det(J);
                invJ = inv(J);

                dN = [dNdxi, dNdeta] * invJ;

                B = zeros(3,8);

                for a = 1:4

                    dNx = dN(a,1);
                    dNy = dN(a,2);
```

```
                    col = (a-1) * 2;

                    B(1, col+1) = dNx;
                    B(2, col+2) = dNy;
                    B(3, col+1) = dNy;
                    B(3, col+2) = dNx;
                end

                B(3,:) = B0(3,:);

                Ke = Ke + (B' * D * B) * t * detJ ...
                    * wgp(i) * wgp(j);

                Nmat = zeros(2,8);

                for a = 1:4
                    Nmat(1, (a-1)*2 + 1) = N(a);
                    Nmat(2, (a-1)*2 + 2) = N(a);
                end

                Me = Me + rho * (Nmat' * Nmat) * t * detJ ...
                    * wgp(i) * wgp(j);
            end
        end

        gdofs = zeros(8,1);

        for a = 1:4
            node = nodes(a);
            gdofs((a-1)*2 + (1:2)) = (node-1)*2 + (1:2);
        end

        K(gdofs, gdofs) = K(gdofs, gdofs) + Ke;
        M(gdofs, gdofs) = M(gdofs, gdofs) + Me;
    end
end
```

In MATLAB, the function `eigs` is used for its efficiency and is shown in Listing 6.2. It is also preferred as it can compute the required number of eigenvalues and eigenvectors. This is more efficient than computing all eigenvalues when only the first few modes are needed in this study [26]. In this work, the option `'smallestabs'` is used to obtain the lowest eigenvalues, which correspond to the lowest natural frequencies.

Listing 6.2. Solution of the generalized eigenvalue problem.

```
[Phi_f, Lambda] = eigs(Kff, Mff, nModes, 'smallestabs', opts);

lambda = diag(Lambda);
```

```
lambda(lambda < 0 & abs(lambda) < 1e-10) = 0;

omega = sqrt(lambda);
freq  = omega / (2*pi);

[freq, sortIdx] = sort(freq, 'ascend');
omega = omega(sortIdx);
Phi_f = Phi_f(:, sortIdx);

Phi_full = zeros(nDOF, nModes);
Phi_full(freeDOFs,:) = Phi_f;
```

Small negative eigenvalues may appear due to numerical round-off errors in MATLAB. If these values are very close to zero, they are set equal to zero before computing the natural frequencies.

6.2.1 Mass Normalisation

The eigenvectors obtained from the generalised eigenvalue problem for different geometries do not have a consistent magnitude scaling. Therefore, a normalisation step is required before using the mode shapes for training the neural network. In this work, mass normalisation is applied. For each mode shape, the normalisation condition is:

$$\boldsymbol{\phi}_i^T \mathbf{M} \boldsymbol{\phi}_i = 1. \tag{6.4}$$

This gives a consistent scaling of the mode shapes which is more suitable for neural network training.

Listing 6.3. Mass normalisation of mode shapes.

```
for imode = 1:nModes
    modalMass = Phi_full(:,imode)' * M * Phi_full(:,imode);
    Phi_full(:,imode) = Phi_full(:,imode) / sqrt(modalMass);
end
```

6.2.2 Mode Ordering and Sign Correction

The eigenvalue solver returns the modes according to their eigenvalues, and the modes are sorted by increasing natural frequency:

$$\omega_1 < \omega_2 < \omega_3 < \cdots . \tag{6.5}$$

For moderate number of rectangular geometries the ordering is okay. For this work, with so many random geometries, the frequencies that are near each other can swap the order of their mode shape. This is known as **mode crossing** or **mode swapping**. To address this and build a uniform sorting to be used by the neural network to learn, this work adopts a frequency-based sorting.

Another important issue is **sign ambiguity**. If ϕ is an eigenvector, then $-\phi$ is also a valid eigenvector. They both are the same physical mode, but the network may not treat them the same if they are found with different signs in different samples. So, a sign correction is needed for the modal data to become consistent. It is done by identifying the degree of freedom with the largest absolute modal value. If this value is negative, the whole mode shape is multiplied by -1 as shown in Listing 6.4.

Listing 6.4. Sign correction of mode shapes.

```
for imode = 1:nModes
    [~, idxMax] = max(abs(Phi_full(:,imode)));

    if Phi_full(idxMax,imode) < 0
        Phi_full(:,imode) = -Phi_full(:,imode);
    end
end
```

This step does not change the physical shape of the mode, but it ensures the eigenvectors are assigned a consistent sign convention for use by the neural network for smoother mapping between geometry and modal response by the neural network.

6.3 Neural Network Architecture

A data-driven neural network is used to learn the relationship between the structural geometry and the corresponding mode shapes. The input vector is:

$$\mathbf{x} = \begin{bmatrix} B \\ H \end{bmatrix} . \tag{6.6}$$

The output vector contains the first three mode shapes. Since the mesh has 128 degrees of freedom for all the nodes and three modes are predicted, the total output size is:

$$n_{\text{out}} = n_{\text{DOF}} n_{\text{modes}} = 128 \times 3 = 384. \tag{6.7}$$

Therefore, the neural network maps two input parameters to 384 output values:

$$\mathbb{R}^2 \rightarrow \mathbb{R}^{384}. \tag{6.8}$$

The input parameters are normalised to the interval $[-1, 1]$. The output mode shapes $\boldsymbol{phi}$ are normalised using z-score normalisation.

A single feedforward neural network is used to predict all three mode shapes at once. The input layer has two neurons, corresponding to the normalised geometric parameters, B and H. The output layer has 384 neurons, corresponding to all displacement degrees of freedom for the three modes required. The network contains five hidden layers with 128 neurons each. A ReLU activation function is used after each hidden layer. The architecture is shown in Listing 6.5.

This architecture is finalised based on a method of trial and error. Increasing the number of hidden layers and neurons led to better accuracy for the third mode particularly. The Tanh activation function was also tested, but ReLU is chosen based on better modal prediction performance.

Listing 6.5. Neural network architecture for modal prediction.

```
nOut = nDOF * nModes;

layers = [
    featureInputLayer(2, "normalisation", "none")
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
```

```
    fullyConnectedLayer(nOut)
];

dlnet = dlnetwork(layerGraph(layers));
```

6.4 Training

To adequately prepare the data for training, the three-dimensional modal array is reshaped into a two-dimensional array. The normalised modal data originally has the size $n_{\text{DOF}} \times n_{\text{modes}} \times n_{\text{samples}}$ reshaped into$(n_{\text{DOF}} n_{\text{modes}}) \times n_{\text{samples}}$,using the MATLAB code shown in Listing 6.6.

Listing 6.6. Reshaping the normalised modal data for training.

```
Yn = reshape(PhiAll_norm, nDOF*nModes, nSamples);
```

Therefore, the training input and output sizes (with all the samples) become:

$$\mathbf{X}_n \in \mathbb{R}^{2\times 2000}, \qquad \text{and} \qquad \mathbf{Y}_n \in \mathbb{R}^{384\times 2000}. \tag{6.9}$$

The neural network is trained by minimising the mean squared error between the normalised finite element mode shapes and the normalised neural network predictions. The main training parameters used are summarised in Table 6.2.

Table 6.2. Training parameters used for the modal neural network.

Parameter	Value
Number of epochs	2000
Initial learning rate	1×10^{-3}
Mini-batch size	200
Learning rate decay epoch	$0.7\times$ total epochs
Learning rate decay factor	0.2

6.5 Inference

After training, the neural network is tested on a test geometry. The selected test case is as follows:

$$B_{\text{test}} = 1.25, \qquad H_{\text{test}} = 2.0. \tag{6.10}$$

The test input is first normalised using the same normalisation technique used during training. The neural network predicts the normalised modal output. The prediction is then denormalised back to the physical mode scale.

Because mode shapes can have arbitrary signs, sign alignment is applied between the FEM and NN mode shapes during inference. The sign is checked using:

$$s_i = \boldsymbol{\phi}_{i,\text{FEM}}^T \mathbf{M} \boldsymbol{\phi}_{i,\text{NN}}. \tag{6.11}$$

If $s_i < 0$, the predicted mode shape is multiplied by -1 to make it positive, as indicated in Listing 6.7. This makes the finite element and neural network resultant modes directly comparable.

Listing 6.7. Sign alignment during inference.

```
for imode = 1:nModes
    signCheck = Phi_FEM(:,imode)' * M * Phi_NN(:,imode);
    if signCheck < 0
        Phi_NN(:,imode) = -Phi_NN(:,imode);
    end
end
```

6.6 Results

6.6.1 Error Metrics

The **relative L^2 error** and the **relative infinity norm error** are calculated in a similar way as in the past cases. The **root mean square error** is computed as

$$\text{RMSE} = \frac{\|\boldsymbol{\phi}_{\text{FEM}} - \boldsymbol{\phi}_{\text{NN}}\|_2}{\sqrt{n}}, \tag{6.12}$$

where n is the number of degrees of freedom.

The **Modal Assurance Criterion** is used to compare the similarity between the FEM and NN mode shapes. It is defined as:

$$\text{MAC} = \frac{\left|\boldsymbol{\phi}_{\text{FEM}}^T \boldsymbol{\phi}_{\text{NN}}\right|^2}{\left(\boldsymbol{\phi}_{\text{FEM}}^T \boldsymbol{\phi}_{\text{FEM}}\right)\left(\boldsymbol{\phi}_{\text{NN}}^T \boldsymbol{\phi}_{\text{NN}}\right)}. \tag{6.13}$$

A MAC value close to 1 represents that the two mode shapes are very similar. A lower MAC value means that the predicted and reference mode shapes deviate more.

6.6.2 Error Evaluation

Table 6.3 shows the modal prediction errors for the test geometry $B = 1.25$ and $H = 2.0$.

Table 6.3. Modal prediction errors for the test geometry $B = 1.25$ and $H = 2.0$.

Mode	FEM frequency [Hz]	RMSE	Rel. L^2 error	Rel. ∞ error	MAC
1	4.072486×10^{-2}	1.194×10^{-3}	2.525×10^{-3}	2.755×10^{-3}	0.999995
2	1.260663×10^{-1}	2.958×10^{-3}	6.593×10^{-3}	1.037×10^{-2}	0.999968
3	1.412901×10^{-1}	1.258×10^{-2}	2.602×10^{-2}	5.294×10^{-2}	0.999450

The trend shows that the model accuracy slightly decreases for increasing modes. The accuracy is the best for the first mode with a relative L^2 error being only 2.525×10^{-3}. This shows that the neural network prediction is almost identical to the finite element mode shape.

The second mode is also predicted with a great accuracy of MAC value of almost 1, although the relative L^2 error increases to 6.593×10^{-3}. This means that the errors at the nodes are larger than in the first mode, but the overall modal deformation pattern is still captured well.

For the third mode, error increases slightly with the relative L^2 error value being 2.602×10^{-2}. The MAC value is still approximately 1, which indicates that the predicted mode shape is still being approximated well.

The surrogate model prediction and the finite element results for the mode shapes are plotted and overlaid for a visual and more intuitive comparison. A unitary scale is used for plotting the deformed mode shapes and for plotting the error too.

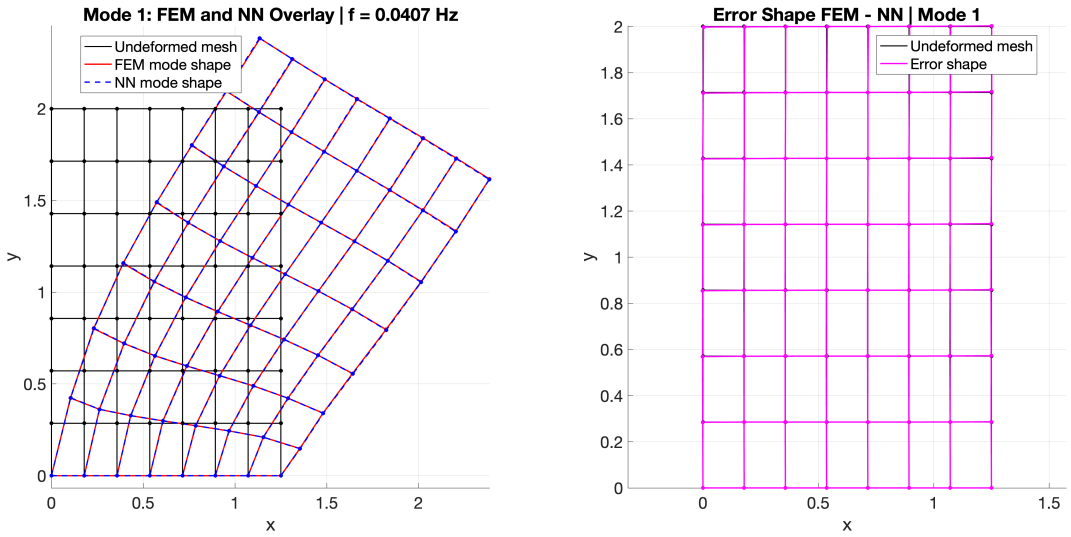


Figure 6.1. Comparison of FEM and neural network prediction with error plots for mode 1.

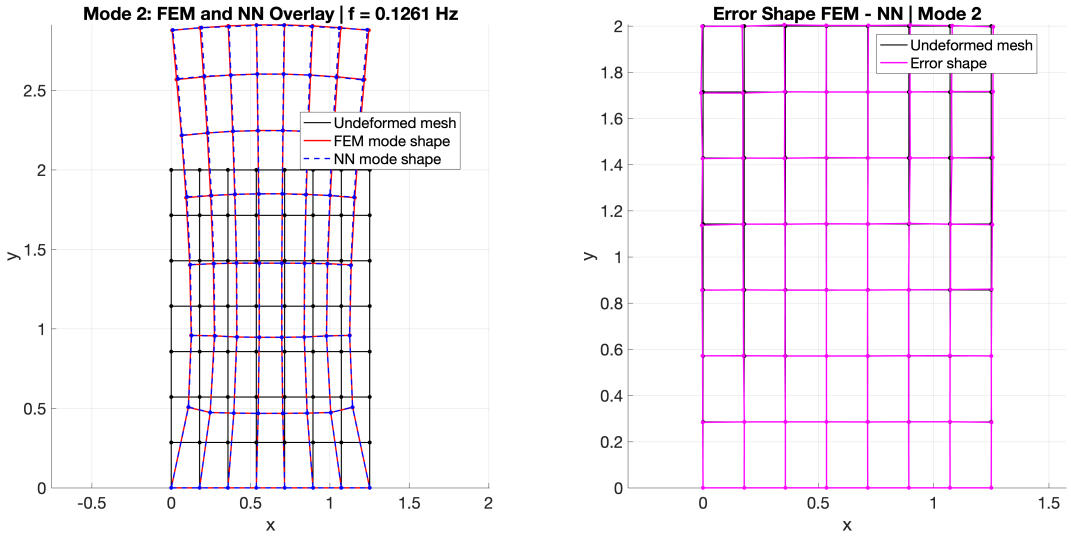


Figure 6.2. Comparison of FEM and neural network prediction with error plots for mode 2.

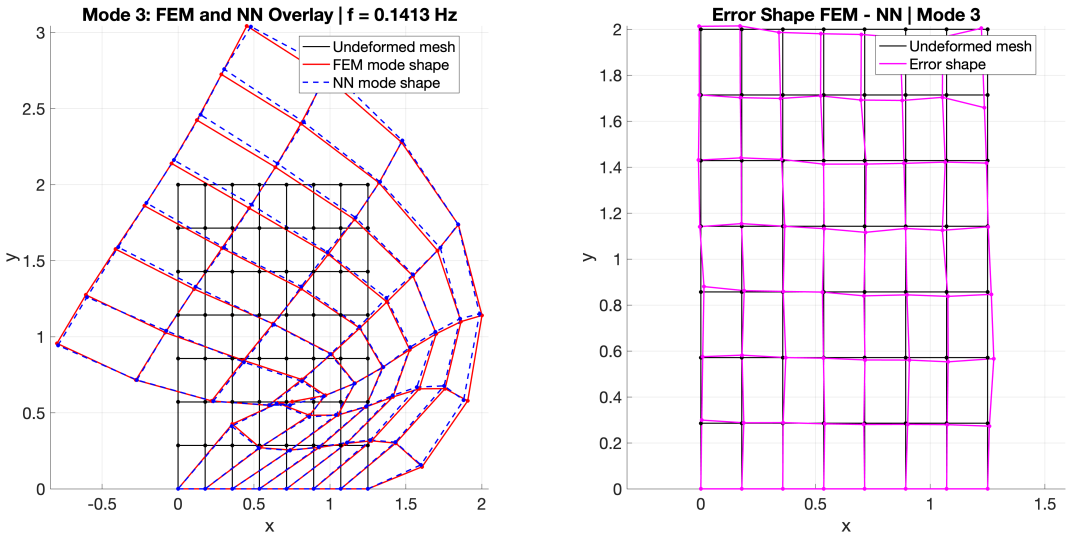


Figure 6.3. Comparison of FEM and neural network prediction with error plots for mode 3.

6.6.3 Computational Time

The average computational times recorded for the dynamic case are shown in Table 6.4.

Table 6.4. Average computational time for the dynamic analysis workflow.

Step	Time [s]
FEM data generation	17.433
Training of neural network	184.732
Inference	0.708

The FEM data generation step is relatively fast as the mesh is not as dense. The training step takes long as the neural networks are learning the relationship between the geometric parameters and the modal outputs. However, this is a one-time cost for a surrogate model. Once the network is trained, inference is very fast for the test geometry, indicating the significance of the model to obtain the mode shapes for new geometries without solving the entire eigenvalue problem again.

6.7 Discussion

The neural network predictions have indicated appropriate accuracy for the mode shapes of the test geometry of a parametrised plane-stress rectangular structure. The MAC is a good measure for comparing mode shapes, and the relative error criterion are better for indicating pointwise differences in the nodal mode values. Therefore, having both errors and MAC has been helpful to evaluate the modal predictions.

The three modes have MAC values almost equal to 1, which indicates that the shape of the mode is approximated well by the neural network. Though, the relative errors increase as the mode number increases. This is an indication of higher mode shapes being more difficult to predict, due to the increasing complexity of the mode shapes and increasing spatial variation than the lower modes.

In plane-stress conditions, out-of-plane modes are not represented. Hence, it is not accurate to simply describe the mode shapes as bending or torsional structure modes. The first mode of this plane-stress structure bounded at the bottom can be described as a lateral sway mode or an in-plane bending mode. The second mode indicates a more axial deformation or extension in the y-direction of the Cartesian plane. The third mode shows an in-plane twisting behaviour that is more complex than the previous two modes. In this case, out-of-plane bending and torsional modes of a structure are not represented because the finite element formulation contains only u_x and u_y degrees of freedom of displacement.

Overall, this extension has created an entry into the use of surrogate models to predict dynamic behaviour of structures. Future work could also be extended to include more training samples, training more complex geometries, or applying a mode-tracking method to reduce the impact of mode swapping.

Chapter 7

Comparison of Surrogate Modelling Approaches

This chapter evaluates different surrogate modelling cases of the two-dimensional plane-stress structures under body forces and imposed by prescribed boundary conditions. The two main approaches for developing the surrogate modelling include data-driven networks and physics-informed networks. Each case is affected by its hyperparameters, network architecture, training parameters, and problem formulation. However, a general comparison between the different cases, and between the two modelling approaches, can be drawn from the results obtained in this work.

7.1 Comparison of Data-Driven Cases

The cases covered in this chapter provide insight into the effects of different factors. The effect of mesh size, geometry range, geometric complexity, output size, and other hyperparameters can be analysed. Since all cases do not use the same mesh size, network architecture, or geometry type, a general comparison between all cannot be directly deduced, but general trends can be derived.

7.1.1 Prediction Accuracy

A summary of the main error values obtained for selected cases is given in Table 7.1. The relative errors listed give us a better insight for comparison.

The simple rectangular structure case gives the least error as the network uses two input parameters only, and the geometry domain is simple. Moving towards more complex shapes, such as the general quadrilateral case, increases the error, as the network now learns the mappings of five input parameters on the output displacement field.

The POD-reduced general quadrilateral case, as compared to the case predicting the complete displacement field directly, has a very slight error difference. Thereby, we can say they both are comparable in accuracy.

The rectangular structure with two holes has an added geometric complexity, holes, modelled as material change in the model, which creates a more complex displacement field variation across the domain for the model to learn, leading to error increase.

Table 7.1. Comparison of selected data-driven surrogate cases.

Case	Geometry & Inputs	RMSE	Relative L^2 error	Relative L^∞ error
Case 1	Rectangular structure, 2 inputs	8.373×10^{-2}	4.597×10^{-3}	9.784×10^{-3}
Case 4	Quadrilateral structure, 5 inputs	3.995×10^{-1}	7.773×10^{-2}	9.588×10^{-2}
Case 5	POD quadrilateral, 5 inputs	4.989×10^{-1}	9.708×10^{-2}	1.190×10^{-1}
Case 6	Two-hole structure, 8 inputs	8.630×10^{-1}	4.613×10^{-2}	7.608×10^{-2}

7.1.2 Computational Cost

The computational time in this study is divided into a general FEM data generation, training, and inference times. Generally, FEM data generation and training are the most computationally costly steps, and inference is very fast in comparison. The computational times for selected data-driven cases are compared in Table 7.2.

Table 7.2. Comparison of computational time for selected data-driven cases.

Case	Geometry & Inputs	FEM data gen. [s]	Training [s]	Inference [s]
Case 1	Rectangular structure, 2 inputs	11.876	73.547	0.755
Case 4	Quadrilateral structure, 5 inputs	30.264	346.100	0.673
Case 5	POD quadrilateral, 5 inputs	115.571	91.343	0.811
Case 6	Two-hole structure, 8 inputs	75.613	168.653	0.916

The rectangular case has the lowest data generation and training cost because it uses a simpler geometry and fewer input parameters. The quadrilateral and two-hole cases require more time as more samples are generated and geometry and mappings are more complex, hence training time is high too. The two-hole case uses a deeper network as well, thereby increasing the training time. The POD-based case for a general quadrilateral structure has the highest data generation time because it uses a larger number of FEM samples, but its training time is lower than the direct quadrilateral model because the output space is reduced.

7.1.3 Effect of Mesh Size

Generally, creating a denser mesh improves the network accuracy. However, the mesh-size study in Case 2 demonstrates that adding more nodes does not directly result in a lower neural network error. There are other factors that play a role simultaneously. Mesh refinement also increases the number of output degrees of freedom that must be predicted in this study. As a result, as the mesh is refined, the learning problem grows and becomes complicated.

This results in a more complex change in network accuracy related to mesh refinement. The 8×8 and 12×12 meshes, which are the intermediate mesh sizes in the test case study, end up giving the best overall performance. This suggests that mesh refinement should be accompanied by sufficient training data and suitable network capacity in order to improve results.

7.1.4 Effect of Geometry Range

The dimension-range study in Case 3 shows that the learning problem becomes more challenging as the range of geometric input parameters, B and H, increases. As a control, the number of samples remained the same, leading to data being more spread out in larger ranges. Consequently, the network has fewer examples for each region of the parameter domain, leading to less data for learning, creating a less accurate network as range increases.

The absolute RMSE also increases with the size of the geometry range. The reason for this is that larger structures generally produce larger displacement magnitudes under the same force, leading to larger errors being in greater magnitudes during displacement predictions.

7.1.5 Effect of General Quadrilateral Geometry

When extending from a rectangular structure (Case 1) to a general quadrilateral structure (Case 4), the input parameters also increase from two to five. A deeper network and larger training data size is required to better learn the mappings. The complexity in the mappings comes not just from the parameter increase, but also from the displacement complexity due to the complex edges and shape distortions in various quadrilaterals. This leads to a higher error in the general quadrilateral case, and a higher computation time as well.

7.1.6 Effect of POD Integration

The POD-reduced model in Case 5 is compared with the direct displacement prediction in Case 4. Both cases use the same geometric parametrisation and the test geometry, but the POD model predicts a smaller set of coefficients instead of the full displacement vector, leading to a smaller output size.

The accuracy of the POD-reduced case remains comparable to Case 4, with the error being only slightly higher. A slight increase is expected because the POD-based error includes both POD truncation error and neural network coefficient pre-

diction error. Therefore, POD successfully creates a balance between reconstruction accuracy and learning complexity. Additionally, as the network is trained in the reduced POD coefficient space, Case 5's training time is shorter than Case 4's despite using more FEM samples.

7.1.7 Effect of Circular Holes

The two-hole rectangular structure in Case 6 compared with the rectangular structure in Case 1 uses an increased input vector from two to eight parameters. The holes are modelled as less stiff materials, introducing local stiffness changes and stronger displacement gradients near the hole-domain boundaries. The holes are represented using low-stiffness material regions, which introduce local stiffness changes and stronger displacement gradients near the hole boundaries.

Compared with Case 1, the two-hole case gives a higher absolute error and higher relative errors. This is expected because the displacement field is no longer smooth over the entire domain. However, the relative L^2 error remains moderate, showing that the network still captures the overall displacement response. The higher relative L^∞ error indicates that the largest deviations are localised near the hole boundaries and in the region between the two holes, where the displacement field changes more rapidly.

The two-hole structure case demonstrates that data-driven models can be expanded to more complex geometries, but require more network capacity, higher mesh resolution, and denser training data to capture the local response around geometric discontinuities.

7.2 Comparison of Physics-Informed Cases

The strong-form and variational PINNs are compared using the same test geometry and body force. They can be compared based on prediction accuracy and computational time.

7.2.1 Prediction Accuracy

Table 7.3 shows the comparison of the errrors of the two approaches.

Table 7.3. Comparison of displacement prediction errors for the strong-form and variational PINNs.

Error measure	Strong-form PINN	Variational PINN
RMSE	4.718×10^{0}	4.802×10^{-1}
Relative L^2 error	2.645×10^{-1}	2.692×10^{-2}
Relative L^∞ error	2.877×10^{-1}	3.260×10^{-2}

The variational formulation gives lesser prediction errors than the strong-form PINN for the test geometry evaluated. The relative L^2 error decreases from approximately 26.5% to 2.7%, while the relative L^∞ error decreases from approximately 28.8% to 3.3%. This indicates that not only does the overall displacement field prediction improves but the maximum displacement is also predicted better. The RMSE is also reduced approximately one order of magnitude.

The difference in their prediction accuracy cannot be attributed to one factor, as it is affected by several factors simultaneously. One of the reasons could be that the variational formulation requires first-order derivatives of the displacement field, which is easier to optimise. The strong-form PINN requires the computation of stress derivatives, which makes the loss sensitive to higher-order derivative errors, making it much harder to predict.

Additionally, the variational formulation reduces the need for a traction-free loss term, making the prediction simpler than for the strong-form PINN. The traction loss term also has to be balanced against the PDE residual and the boundary loss terms, which makes the optimisation problem more sensitive to the relative weighting of the different loss terms.

While most of the other model parameters are kept the same in both cases, the boundary loss weight is different in each. This could also contribute to the difference in the prediction accuracy.

7.2.2 Computational Time

The training and inference time for both approaches is compared in Table 7.4.

Table 7.4. Comparison of computational time for the strong-form and variational PINNs.

Step	Strong-form PINN	Variational PINN
Geometry and collocation point generation	3.80 s	7.19 s
Training	14907.170 s	3010.14 s
Inference	0.363 s	0.194 s

The training time of the variational PINN is much lower than that of the strong-form PINN as expected. For this test case, the variational formulation is about 5.0 times faster during training. This is because a variational formulation is only computing first-order derivatives and penalising them.

Overall in this study, the variational PINN performs better for the present problem. It requires less training time and produces lower displacement prediction errors. This performance, however, depends on the network architecture, training parameters, loss weights, and current geometry range. The accuracy of the strong-form PINN may be enhanced by additional tuning, such as adding more collocation points, altering the network architecture, or modifying the loss weights.

7.3 Comparison of Data-driven and Physics-Informed Cases

For comparison of the two neural network approaches, the simple rectangular plane-stress structure cases are used. Therefore, Case 1 of the data-driven approach is compared with the similar rectangular structure used in the two physics-informed approaches: strong-form and variational energy.

This comparison is a generic overview for the case studied rather than strictly

one-to-one universal comparison. This is because the models use different training strategies; the data-driven model is trained using finite element labelled data, whereas the physics-informed models are trained using physics-based loss terms for the selected test problem. The models also differ in their training datasets, loss functions, network settings, and computational requirements. Therefore, the differences can not be attributed to a particular factor.

7.3.1 Prediction Accuracy

Table 7.5 compares the displacement prediction errors of data-driven model and the two physics-informed models. The data-driven model gives the lowest relative error among the tested cases. The variational physics-informed model gives the second lowest relative error and performs significantly better than the strong-form PINN.

Table 7.5. Comparison of displacement prediction errors for the selected data-driven and physics-informed models.

Case	RMSE	Relative L^2 error	Relative L^∞ error
Data-driven Case 1	8.373×10^{-2}	4.597×10^{-3}	9.784×10^{-3}
Strong-form PINN	4.718×10^{0}	2.645×10^{-1}	2.877×10^{-1}
Variational PINN	4.802×10^{-1}	2.692×10^{-2}	3.260×10^{-2}

The relative L^2 error of the data-driven model is approximately 0.46%, indicating that it reproduces the overall displacement response with high accuracy. The same error is approximately 2.7% for the variational PINN and 26.5% for the strong-form PINN. Similarly, the relative L^∞ errors show that the largest absolute displacement error is approximately 1.0%, 3.3%, and 28.8% of the maximum finite element displacement magnitude for the data-driven, variational, and strong-form models, respectively.

The higher accuracy of the data-driven method can be attributed to its direct training on finite element data. However, there is a computational cost to generating the training data required for this method. In contrast, the PINNs do not require

labelled displacement data during training, but their performance is more sensitive to collocation-point sampling, loss weighting, derivatives, boundary conditions, and optimisation.

For this case, the strong-form PINN does not perform as well as the other two models. This is mainly because the strong-form formulation requires the calculation of equilibrium residuals involving stress derivatives, which makes the loss function more difficult to optimise. In addition, the displacement boundary condition, traction-free boundary conditions, and PDE residuals are all imposed as separate soft loss terms, so the final accuracy depends strongly on how these terms are balanced. The variational PINN performs better in this comparison because it avoids the stress-derivative residuals and includes the traction-free boundaries naturally through the energy formulation.

The errors of the physics-informed models may be reduced through further tuning, such as using more collocation points, longer training, improved loss weighting, stronger boundary-condition enforcement, or larger network architectures. This comparison is used mainly to indicate the relative performance trend between the approaches for the current case study, rather than a universal evaluation of these approaches.

7.3.2 Computational Time

The computational times of the three models are compared in Table 7.6. By comparing the computation times of the two approaches, it can be observed that physics-informed models require less time for data generation than data-driven models.

In case of physics-informed models, the data generation constitutes the creation of the collocation and boundary points required for different samples. For data-driven cases, it is more intensive, as the finite element solver runs simulations for the number of samples determined. These stages are not computationally equivalent, hence the large difference.

In contrast, training time is significantly longer for physics-informed cases

Table 7.6. Comparison of computational times for the selected data-driven and physics-informed models.

Case	Data/geom. gen. [s]	Training [s]	Inference [s]	Total [s]
Data-driven Case 1	11.876	73.547	0.755	86.178
Strong-form PINN	3.795	14907.170	0.363	14911.328
Variational PINN	7.191	3010.137	0.194	3017.522

due to the computation required for automatic differentiation for the various derivatives. The strong-form PINN has the highest training cost because its loss evaluation involves repeated calculation of equilibrium residuals, second-order displacement derivatives, and traction-boundary residuals. The inference time is fast for both methods, indicating that all trained models can provide fast predictions once training is complete.

7.3.3 Summary

For the problems examined in this thesis, the data-driven models generally offer the highest accuracy and computational efficiency, particularly when there is an adequate amount of FEM data. Although the physics-informed models require less labelled FEM training data, their performance is more dependent on training settings, boundary-conditions, derivative computation, and loss function formulation. The variational physics-informed case outperforms the strong-form case in terms of accuracy and training time among the physics-informed cases. Thus, the data-driven approach is the best surrogate modelling strategy for the current work, with a great balance of prediction accuracy and computational time, although the variational physics-informed stays a viable second option in case labelled data is not available.

Table 7.7. Comparison summary of data-driven and physics-informed surrogate models.

Aspect	Data-driven models	Physics-informed models
Training data	Require FEM-generated labelled data	Require collocation and boundary points, but no labelled FEM solution for training
Accuracy in this work	Higher accuracy for the tested cases	Lower accuracy, especially for the strong-form PINN
Training cost	Lower training cost after data generation	Higher training cost due to automatic differentiation
Boundary conditions	Learned from the training data	Enforced through loss terms
Implementation	Simpler once FEM data is available	More sensitive to PDE residuals, loss weights, and derivative scaling
Best use case	Repeated predictions when FEM data can be generated	Problems where labelled data is limited or physics constraints are important

Conclusions

The aim of this thesis is to explore the use of data-driven and physics-informed neural networks as surrogate models for two-dimensional structural analysis problems. The main interest is in linear elastic plane-stress structures under applied body forces and prescribed boundary conditions. Different surrogate modelling strategies are investigated for the prediction of displacement fields and modal analysis based on this reference solution.

The data-driven models show that with sufficient training data, neural networks can approximate the finite element displacement response with a good accuracy. For simple rectangular geometries, the predicted displacement fields are found to be close to the finite element solutions with low relative errors. The mesh study revealed that the higher the mesh resolution, the more expensive the learning problem would be and due to the larger dimension of the output of the network, the prediction accuracy might not always be improved. The geometry range study reveal that the wider the range of possible dimensions, the more complex the mapping, and the greater the prediction error. Test geometries are best suited to be tested within the same range as those used during training.

The extension to more general quadrilaterals geometries shows that more flexible geometry parametrisations can also be used with neural-network surrogates. However, this makes the input-output relationship more complicated, leading to a decrease in the accuracy of the surrogate model. The use of Proper Orthogonal Decomposition decreases the dimension of the output and increases computational efficiency during training. The POD-based model accelerates the training while giv-

ing an accuracy comparable to the original complete displacement field prediction method.

In addition to data-driven models, physics-informed neural networks are also investigated. The strong-form PINN can approximate the general displacement field without requiring a large finite element training dataset. However, its accuracy is significantly lower than the data-driven model and the variational PINN for the case evaluated in this work. This is mainly due to the difficulty of training the strong-form formulation, where the PDE residual, Dirichlet boundary condition, and traction boundary condition must be balanced carefully. The model is also sensitive to the choice of loss weights, collocation points, network architecture, and training parameters. Although different network architectures and training settings are tested, the improvement is limited, while the training time and computational cost increases considerably.

The lower prediction errors reveal the variational formulation to be more accurate and stable than strong-form PINN for the present problem. This could be because it does not have the same complexity as strong-form PINNs. However, it still relies on the derivative calculations, the enforcement of the boundary conditions, and the training setup, which is intensive.

An initial extension to the dynamic analysis is also performed, by solving a modal analysis problem. The natural frequencies and mode shapes of the rectangular structure are obtained by solving a finite element eigenvalue problem. The trained neural networks are then applied to the prediction of the modal response for different rectangular geometries. The results shows that surrogate modelling is an effective solution to the prediction of the modal quantities. The lower modes are well predicted while for the higher modes the accuracy slightly decreases. This work can be further developed for more complex transient problems.

In general, the results demonstrate that for the test problem dimensions should be within the training range and the high-quality finite element data should be available for data-driven neural networks to be efficient and accurate. Data-driven surrogate models are useful if repeated structural analysis is required for different ge-

ometries. While physics-informed neural networks can be trained using less labelled finite element data, they are more difficult to train and can be more sensitive to the tuning. Furthermore, the strong-form PINN did not show as good performance in this work as anticipated, indicating that maybe more collocation points, larger networks and longer training times are needed to improve the accuracy of the strong-form PINN, which will also increase the computational cost. Thus, although PINNs offer a physically constrained learning framework, their usage in practice is highly dependent on the formulation and training procedure.

The study is limited to two dimensional linear elastic plane-stress problems having relatively simple loading conditions and boundary conditions. The material properties are fixed, and the models are trained for specific geometry parameterisations. The test of the neural networks is conducted on specific geometry cases, rather than a large test set. The boundary conditions is difficult to enforce, and the training setup affects the PINNs' accuracy. Furthermore, forced vibration and time-dependent response are not included in the dynamic portion of the work, which is restricted to free-vibration modal analysis.

The current work can be expanded for future research. Non-linear applications and three dimensional geometries can be tested. More complex geometries, material parameters, loading cases and boundary conditions can be included as inputs. Larger dataset and more complex architectures can be used to increase the generalisation of the data-driven models. For the physics-informed models, adaptive loss weighting, adaptive collocation strategy (including hybrid sampling), and hard boundary conditions can be investigated. The modal analysis can be further expanded to transient dynamic problems and frequency-response analysis. Finally, a hybrid approach can be tested, which uses both finite element data and physics-informed formulation.

Overall, this thesis shows the ability to create a neural network surrogate model for a finite element solution to a parametric structural analysis problem with speed and reasonable accuracy. These models are not as versatile as finite element analysis yet but can be used to reduce the computational effort when repeated simulations, design testing, or parametric design analysis is required.

Bibliography

[1] J. Kudela and R. Matousek, "Recent advances and applications of surrogate models for finite element method computations: a review," *Soft Computing*, vol. 26, pp. 13709–13733, 2022.

[2] H. Adeli, "Neural networks in civil engineering: 1989–2000," *Computer-Aided Civil and Infrastructure Engineering*, vol. 16, no. 2, pp. 126–142, 2001.

[3] J. Ghaboussi, J. H. Garrett, and X. Wu, "Knowledge-based modeling of material behavior with neural networks," *Journal of Engineering Mechanics*, vol. 117, no. 1, pp. 132–153, 1991.

[4] E. Kharazmi, Z. Zhang, and G. E. Karniadakis, "Variational physics-informed neural networks for solving partial differential equations," *arXiv preprint arXiv:1912.00873*, 2019.

[5] W. E and B. Yu, "The deep ritz method: A deep learning-based numerical algorithm for solving variational problems," *Communications in Mathematics and Statistics*, vol. 6, no. 1, pp. 1–12, 2018.

[6] S. P. Timoshenko and J. N. Goodier, *Theory of Elasticity*. New York: McGraw-Hill, 3 ed., 1970.

[7] M. H. Sadd, *Elasticity: Theory, Applications, and Numerics*. Academic Press, 2 ed., 2009.

[8] T. J. R. Hughes, *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Englewood Cliffs, NJ: Prentice-Hall, 1987.

[9] M. J. Turner, R. W. Clough, H. C. Martin, and L. J. Topp, "Stiffness and deflection analysis of complex structures," *Journal of the Aeronautical Sciences*, vol. 23, no. 9, pp. 805–823, 1956.

[10] R. W. Clough, "The finite element method in plane stress analysis," in *Proceedings of the 2nd ASCE Conference on Electronic Computation*, (Pittsburgh, Pennsylvania), pp. 345–378, 1960.

[11] O. C. Zienkiewicz, R. L. Taylor, and J. Z. Zhu, *The Finite Element Method: Its Basis and Fundamentals*. Butterworth-Heinemann, 7 ed., 2013.

[12] K.-J. Bathe, *Finite Element Procedures*. Klaus-J"urgen Bathe, 2006.

[13] R. W. Clough and J. Penzien, *Dynamics of Structures*. New York: McGraw-Hill, 2 ed., 1993.

[14] A. K. Chopra, *Dynamics of Structures: Theory and Applications to Earthquake Engineering*. Pearson, 5 ed., 2017.

[15] L. Meirovitch, *Fundamentals of Vibrations*. New York: McGraw-Hill, 2001.

[16] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

[17] M. Leshno, V. Y. Lin, A. Pinkus, and S. Schocken, "Multilayer feedforward networks with a nonpolynomial activation function can approximate any function," *Neural Networks*, vol. 6, no. 6, pp. 861–867, 1993.

[18] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[19] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[20] E. Kharazmi, Z. Zhang, and G. E. Karniadakis, "hp-vpinns: Variational physics-informed neural networks with domain decomposition," *Computer Methods in Applied Mechanics and Engineering*, vol. 374, p. 113547, 2021.

[21] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, vol. 9 of *Proceedings of Machine Learning Research*, pp. 249–256, 2010.

[22] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning*, pp. 807–814, 2010.

[23] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[24] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. arXiv:1412.6980.

[25] The MathWorks, Inc., *Deep Learning Toolbox Documentation*. The MathWorks, Inc., 2026. Accessed: 2026-07-06.

[26] The MathWorks, Inc., *`eigs`: Subset of eigenvalues and eigenvectors*. The MathWorks, Inc., 2026. MATLAB Documentation. Accessed: 2026-07-06.

Acknowledgements

This thesis would not have been possible without the guidance of my esteemed supervisors, Prof. Domenico Magisano, guided me throughout this work, introduced me to many new concepts, and was always available whenever I needed help, and Prof. Giovanni Garcea, who welcomed me with warmth and supported me throughout my time here. I am also grateful to everyone in the DIMES lab for their support throughout my time here, especially Andrea Leonetti and Prof. Leonardo Leonetti.

I also extend my gratitude to all my professors and colleagues from my Erasmus Mundus journey, who helped me gain the knowledge and skills that formed the foundation of this work. I am equally thankful to the wonderful people and friends I met along the way in Lille, Wrocław, Athens, and Cosenza who taught me invaluable lessons about life and the world.

Above all, my deepest gratitude goes to my mother, who taught me that I could do anything I set my mind to, and to my father, who supported me in every dream I pursued. I owe the most to my brother, Asad, and sister-in-law, Saba, for being the backbone I could always rely on. I also extend my heartfelt thanks to my entire family, especially Aapa, because it truly took a village to get me where I am today.

My friends have helped shape me into the person I am today, and I am most grateful to my closest friends, Khuzaima, Alvina, Bareerah, Furhana, Ahmed, and Zaha for their unwavering belief in me, even when I struggled to believe in myself.

Finally, I thank Allah Almighty for all the opportunities that came my way, for the incredible people I met on my Erasmus journey, and for showing me light on my darkest days.